ADDIS ABABA UNIVERSITY

COLLEGE OF NATURAL AND COMPUTATIONAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

AGRINET

CoSc 4122: FINAL PROJECTI

BY:

1. Lisan Geberetensay UGR/1712/15
2. Tola Anbesu UGR/7995/15
3.Yohannes Gizaw UGR/6435/15
4.Nathnael Hailemariam UGR/9299/16

PROJECT ADVISOR:Ashenafi K

Nov 2025

DECLARATION

This is to declare that this project work which is done under supervision of Mr. Ashenafi K and having the title "Agrinet(Agricultural Networking) is the solcontribution of:

1. Lisan Geberetensay
2. Tola Anbesu
3. Yohannes Gizaw
4. Nathnael Hailemariam

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: _____

Group Members:

Full Name	Signature
_____	_____
_____	_____
_____	_____
_____	_____

CERTIFICATE

I certify that this BSc final project report entitled “AgriNet(Agricultural Networking)” by:

1. Lisan Geberetensay
2. Tola Anbesu
3. Yohannes Gizaw
4. Nathnael Hailemariam

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

_____ _____

Date Name and Signature of Supervisor

ACKNOWLEDGEMENT

Our heartfelt gratitude goes to Mr. Ashenafi K. for his unwavering support, insightful guidance, and invaluable feedback. his expertise and mentorship have been instrumental in shaping the direction and quality of our collective work.

We owe a special thanks to the senior students who generously shared their experiences and knowledge with us. Their insights and advice significantly contributed to the development and execution of our project. Their willingness to lend a helping hand has been truly appreciated.

TABLE OF CONTENTS

TABLE OF FIGURES

LIST OF TABLE

ACRONYMS

✓ SID - Scenario ID

CHAPTER ONE – PROJECT PROPOSAL

1.1. Introduction

Introducing Agrinet**,** a groundbreaking digital agricultural platform designed to revolutionize how farmers and buyers connect, trade, and grow together. In a world where traditional agricultural systems still rely on middlemen, fragmented markets, and limited access to information, the platform emerges as a seamless and transformative solution.

Our motivation stems from the recognition that existing agricultural market structures often leave farmers underpaid, isolated, and disconnected from consumers and modern opportunities. With this project, we aim to simplify and digitize the entire process empowering farmers with a user-friendly platform that eliminates unnecessary intermediaries while ensuring transparency, fair pricing, and accessibility for all.

What truly sets Agrinet apart is its commitment to go beyond a simple marketplace. By harnessing the power of digital technology and data-driven insights**,**the platform transforms traditional farming interactions into smart, efficient, and connected experiences. The platform enables farmers to list and sell crops and livestock directly to consumers, wholesalers, and retailers while accessing expert guidance on crop management, disease detection, and sustainable farming practices.

In addition to its commercial and logistical strengths, the project fosters avibrant agricultural communitya digital hub where farmers can exchange experiences, share knowledge, and collaborate to solve challenges. This collaborative forum transforms individual progress into collective growth, cultivating a culture of learning and empowerment across the agricultural landscape.

In essence, Agrinet transcends the limitations of traditional farming systems, offering a dynamic, data-driven, and community-centered platform that empowers farmers, connects markets, and promotes sustainable agricultural development. Through innovation, inclusivity, and collaboration, the platform envisions a future where technology and tradition work hand in hand to secure fair opportunities, improve livelihoods, and strengthen food security for all.

1.2. Statement of the Problem

Agriculture remains the backbone of many economies, providing livelihoods for millions of farmers. However, despite its importance, the sector continues to face persistent challenges that limit productivity, profitability, and sustainability. Many traditional agricultural systems still rely on outdated methods of trade and communication, leaving farmers disconnected from modern markets and vital information. These gaps create inefficiencies in the agricultural value chain, affecting both farmers and consumers alike.

One of the most pressing issues is the dominance of middlemen, who often dictate prices and reduce farmers' profits. This lack of transparency prevents farmers from receiving fair compensation for their hard work. In addition, many farmers struggle with limited access to expert guidance on crop and livestock management, disease prevention, and modern farming techniques, resulting in lower yields and income. Transportation barriersfurther restrict farmers from reaching larger markets efficiently, while the absence of a centralized communication and knowledge-sharing platform isolates them from collaboration and innovation.

To address these challenges, Agrinet proposes a comprehensive digital solution that transforms how farmers and buyers interact. The platform enables direct connections between farmers and buyers, ensuring fair trade without intermediaries. It provides expert agricultural advice to help farmers make informed decisions and integrated transport support to ensure timely and safe delivery of goods. Furthermore, Agrinet creates an interactive community space where farmers can share experiences, seek guidance, and learn from one another. Through these combined efforts, the platform aims to empower farmers, promote transparency, and foster sustainable growth across the agricultural sector.

The Problems We Want to Address:

- **Middlemen dominance:** Farmers are often forced to sell their products through intermediaries who dictate prices, reducing farmers' profit margins and causing unfair market practices.
- **Limited access to expert knowledge:** Many farmers lacks reliable information on modern farming techniques, crop and livestock management, and disease control, which negatively impacts productivity and sustainability.
- **Lack of communication and collaboration:** The absence of a centralized digital hub prevents farmers from sharing knowledge, discussing challenges, or learning from one another.

Proposed Solutions:

- **Direct marketplace:**Agrinet connects farmers directly with buyers' consumers, wholesalers, and retailers eliminating middlemen and ensuring fair prices.
- **Expert advisory system:** The platform offers professional guidance on crop and livestock management, disease detection, and sustainable farming practices, helping farmers make informed decisions.
- **Collaborative online community:** The platform incorporates a discussion forum where farmers can interact, share experiences, and seek advice, fostering innovation and community growth.

1.3. Objectives

1.3.1 General Objectives

The general objective of Agrinet is to create a modern, efficient, and transparent digital agricultural marketplace that directly connects farmers with buyers. In simple terms, it aims to bridge the gap between producers and consumers by removing unnecessary middlemen who often exploit farmers and distort fair pricing.

At its core, the general objective focuses on empowering farmers not only by giving them direct access to markets but also by providing them with the tools, knowledge, and support they need to thrive in a competitive agricultural economy. Agrinet is designed to make trading easier, faster, and more profitable for farmers while offering buyers direct access to high-quality agricultural products.

Beyond trading, the general objective also includes enhancing productivity and sustainability. Through expert guidance on crop and livestock management, disease detection, and modern farming techniques, the platform supports informed decision-making and encourages farmers to adopt better practices.

Furthermore, Agrinet's goal extends to improving logistics and communication in agriculture. Its transportation support system ensures that goods are delivered safely and efficiently, reducing market inefficiencies. The inclusion of a knowledge-sharing community further strengthens collaboration among farmers, enabling them to share experiences, learn from each other, and stay connected to new agricultural trends and innovations.

1.3.2.Specific Objective

- **To gather and analyze detailed user requirements** from farmers, buyers, and experts through observation, interviews, and document review, ensuring the system fully reflects real-world agricultural marketplace needs.
- **To design a scalable system architecture** that supports marketplace functionalities, exert advisory services, transportation management, and community interaction modules.
- **To develop a responsive and user-friendly interface (UI/UX)** that allows farmers and byers to easily register, list products, communicate, and access advisory content.
- **To design and implement a secure and optimized database** using PostgreSQL and Prisma ORM, ensuring efficient management of user data, product listings, transactions, discussions, and system logs.

- **To implement core system functionalities** such as product listing, user authentication, expert consultation, transportation request handling, and discussion forum using modern software frameworks.
- **To integrate reliable data validation and security mechanisms** (authentication, authorization, input validation, encryption) to protect user data and ensure safe transactions.
- **To perform comprehensive testing activities** (unit testing, integration testing, system testing, usability testing, and user acceptance testing) to ensure that the system is functional, secure, and user-friendly.
- **To deploy the system on a suitable hosting environment** ensuring accessibility, performance optimization, and proper configuration for real-world usage.
- **To provide system documentation** including requirement specifications, system design documents, user manuals, and maintenance guidelines.
- **To establish a maintenance plan** ensuring the system receives continuous improvements, bug fixes, security updates, and feature enhancements after deployment.

1.4.Scope and limitations

The scope of the project encompasses the development and implementation of an integrated online agricultural marketplace that directly connects farmers with various categories of buyers, including wholesalers, retailers, and end consumers. The platform is designed to facilitate the trade of a diverse range of agricultural commodities such as crops, livestock, and other farm-related products.

Beyond its core marketplace functionality, the platform extends its services to include a range of value-added features aimed at empowering the agricultural community. These features include expert advisory services on crop and livestock disease detection, best practices in modern and sustainable farming, and up-to-date market insights to guide informed decision-making. The platform also integrates communication and collaboration tools to promote knowledge exchange among farmers, thereby fostering innovation, problem-solving, and community growth within the agricultural sector.

Furthermore, Agrinet includes a transportation support module that assists farmers and buyers in managing the logistics of product delivery. This ensures that agricultural goods are transported efficiently, safely, and in a timely manner, enhancing trust and reliability in transactions.

In addition to marketplace functionality, the system integrates several essential modules to enhance the overall user experience and agricultural productivity. These include:

- **Expert Advisory Module:** Provides farmers with access to real-time guidance on crop and livestock disease detection, modern agricultural practices, and effective farm management techniques.
- **Knowledge Sharing Platform:** Facilitates communication and collaboration among farmers, enabling the exchange of ideas, experiences, and solutions to common challenges within the agricultural community.
- **Market Insight Module:** Offers current market trends, pricing data, and demand analysis to support informed decision-making.

Overall, the scope of project extends beyond digital trade facilitation—it aims to build a comprehensive agricultural ecosystem that promotes transparency, sustainability, and inclusivity while addressing key challenges faced by farmers in accessing markets, information, and logistics.

Limitations

- **Limited Internet Access in Rural Areas**: Many farmers—especially in remote areas—may have weak or unreliable internet connectivity, which can limit their ability to fully use the platform's features such as product listing, communication, or accessing expert advice.
- **Low Digital Literacy Among Farmers**: A significant portion of farmers may not be familiar with smartphones, computers, or digital platforms. This may reduce user adoption unless adequate training and awareness programs are provided.
- **Dependence on Accurate User Input**: The quality of marketplace data (such as product prices, quantities, and descriptions) depends on users entering correct and up-to-date information. Incorrect inputs can affect product visibility, buyer trust, and overall system reliability.
- **Lack of Real-Time Market Data Integration**: Since the platform may not have full integration with national agricultural market systems, real-time price fluctuations, supply changes, and market demands may not always be perfectly updated.
- **Transportation and Logistics Challenges**: Although the system includes transportation support, the real-world delivery process depends on external transport providers. Delays, shortages, or poor infrastructure can impact the effectiveness of the logistics module.

1.5. System Development Methodology

The **Project** follows a **System Development Life Cycle (SDLC)** approach to ensure a structured, efficient, and high-quality development process. The chosen methodology is the **Waterfall Model**, which provides a clear, sequential flow of activities, ensuring that each development phase is thoroughly completed before moving on to the next. This approach was selected due to its suitability for projects with well-defined requirements and deliverables.

The following phases outline the methodology used in developing the project:

- **PlanningPhase**

  This phase involved identifying the overall objectives, scope, and feasibility of the project. Detailed project planning was conducted to determine resource allocation, timelines, risk management strategies, and the expected outcomes of the system. Stakeholder needs—including farmers, buyers, and administrators—were analyzed to align the project's goals with user requirements.

- **System Analysis Phase**

  During this stage, a thorough analysis of the existing agricultural market challenges was conducted. The functional and non-functional requirements of the Agrinet system

were identified, such as user registration, product listing, communication, expert consultation, and logistics support. Data flow diagrams (DFDs) and use case models were developed to visualize system interactions and workflows.

- **System Design Phase**

  In this phase, both high-level and detailed designs of the platform were created. The system architecture, database schema, and user interface layouts were designed to ensure scalability, usability, and efficiency. Design considerations also included security protocols, navigation flow, and system integration with external modules like transportation support.

- **Development Phase**

  The development phase involved coding and implementing the system components according to the approved design specifications. Modern programming languages, frameworks, and database systems were used to build the front-end, back-end, and database layers. Emphasis was placed on writing clean, maintainable, and well-documented code.

- **Testing Phase**

  After development, comprehensive testing was conducted to ensure system functionality, reliability, and security. Various testing methods such as unit testing, integration testing, system testing, and user acceptance testing (UAT) were applied to identify and correct any errors or inconsistencies in the system.

- **Implementation Phase**

  The tested system was deployed for initial use. User accounts were set up, and the system was configured in a live environment. Training sessions were provided to ensure users could effectively navigate the system and utilize all available features.

- **Maintenance Phase**

  Post-implementation, continuous monitoring and maintenance activities were established to ensure system stability and performance. Feedback from users was collected to guide future updates, bug fixes, and feature enhancements, ensuring the system remains relevant and effective in meeting evolving agricultural needs.

1.5.1.Investigation (Fact-finding) Methods

To ensure the successful development of the **project**, a thorough investigation was conducted to collect accurate and relevant information about the existing agricultural marketplace processes, user challenges, and technological requirements. The fact-finding process aimed to identify inefficiencies in current systems and gather the necessary insights to design a platform that meets the practical needs of farmers and buyers.

The following methods were employed during the investigation phase:

1. Observation

Direct observation was carried out at agricultural markets, trading centers, and farming communities to gain firsthand understanding of how transactions are conducted between farmers, buyers, and intermediaries. The observation focused on how agricultural goods are priced, exchanged, and transported. This method helped the development team identify key operational challenges such as lack of transparency in pricing, logistical inefficiencies, and communication gaps between stakeholders. Insights from observation guided the formulation of core system features, including direct trading, transportation support, and knowledge-sharing functions.

2. Document Review

Existing documents and records related to agricultural trading, market pricing, and government agricultural policies were carefully reviewed. Reports on agricultural production, market trends, and digital agriculture initiatives were analyzed to understand the broader context and regulatory framework in which the Agrinet System would operate. This method provided valuable information about existing market structures, transaction procedures, and the potential for digital transformation in the agricultural sector.

3. Online Research

Comprehensive online research was conducted to examine existing agricultural e-commerce and digital marketplace platforms. The study involved analyzing the design, features, and operational models of similar systems in both local and international contexts. Through this research, the development team identified best practices, common system limitations, and innovative solutions that could be adapted and improved upon in the Agrinet platform. This helped shape the system's functionality, user interface design, and integration strategy.

1.5.2.System Development Tools

The development of the platform required the use of various software tools, programming languages, and technologies to ensure efficient design, implementation, testing, and deployment. The selected tools were chosen based on their reliability, scalability, ease of integration, and suitability for building a secure and user-friendly online marketplace platform.

- **Front-end**: HTML5, CSS3, JavaScript, Tailwind, React.js
- **Back-end**: Next.js, postman
- **Database**: Postgres, Prisma ORM
- **Version Control System**: Git
- **Collaboration**: GitHub

1.6.Significance

The project plays a significant role in addressing key challenges faced by the agricultural sector through the integration of technology, market access, and knowledge sharing. Its implementation contributes not only to improving the economic well-being of farmers but also to enhancing the efficiency, transparency, and sustainability of agricultural trade.

The significance of the project can be viewed from multiple perspectives:

1. Empowerment of Farmers

Agrinet empowers farmers by providing direct access to a digital marketplace where they can sell their products without the interference of middlemen. This ensures that farmers receive fair prices for their goods, increasing their income and financial independence. The platform also allows them to reach a wider range of buyers, including wholesalers, retailers, and end consumers, thereby expanding their market opportunities.

2. Promotion of Transparency and Fair Trade

The system promotes transparency in agricultural trading by displaying real-time pricing information and eliminating hidden costs often associated with traditional market intermediaries. This helps build trust between farmers and buyers while ensuring fairness and accountability in every transaction.

3. Knowledge Sharing and Capacity Building

Agrinet serves as an information hub where farmers can access expert advice on modern farming practices, disease management, and market trends. By facilitating communication and knowledge exchange among farmers, the system encourages learning, collaboration, and innovation within the agricultural community.

4. Improvement of Supply Chain Efficiency

With its integrated transportation support module, Agrinet enhances the delivery process of agricultural products. It ensures that goods reach buyers safely and efficiently, reducing post-harvest losses and minimizing logistical challenges. This leads to more reliable supply chains and improved customer satisfaction.

5. Technological Advancement in Agriculture

The project contributes to the digital transformation of the agricultural sector by introducing an innovative, technology-driven solution for trading and knowledge management. It demonstrates how information technology can be leveraged to modernize agriculture, bridge the digital divide, and support sustainable rural development.

6. Contribution to Economic and Social Development

By increasing farmer income, reducing market inefficiencies, and improving agricultural productivity, the platform indirectly contributes to the overall economic growth of rural communities. The system supports poverty reduction, food security, and social inclusion—aligning with broader national and global development goals.

1.7. Beneficiaries

The projectis designed to benefit multiple stakeholders within the agricultural ecosystem by addressing their specific needs through digital innovation and connectivity. The system aims to create a mutually beneficial environment for farmers, buyers, and other participants involved in agricultural production and trade.

The key beneficiaries of the project include:

1. Farmers

Farmers are the primary beneficiaries of the Agrinet System. The platform enables them to:

- Sell their agricultural products directly to buyers, reducing dependence on middlemen.
- Access fair and competitive market prices for their produce.

- Gain expert advice on modern farming techniques, disease management, and sustainable agricultural practices.
- Connect with other farmers for knowledge exchange and collaboration.
- Improve their income, productivity, and overall livelihood.

By empowering farmers digitally, Agrinet contributes to rural development and enhances their role in the agricultural value chain.

2. Buyers (Wholesalers, Retailers, and Consumers)

Buyers also benefit greatly from the Agrinet System. The platform allows them to:

- Access a wide variety of fresh and high-quality agricultural products directly from farmers.
- Eliminate extra costs added by intermediaries, leading to better pricing.
- Build direct and trustworthy relationships with producers.
- Enjoy improved transparency in product sourcing, pricing, and delivery processes.

This direct connection between buyers and farmers enhances market efficiency and ensures a more reliable food supply chain.

3. Agricultural Experts and Advisors

Agricultural experts, agronomists, and technical advisors benefit from the platform by using the platform as a medium to:

- Share valuable knowledge, training materials, and advisory content with farmers.
- Provide remote consultations on farming techniques, pest control, and productivity improvement.
- Engage with a larger audience of farmers, thereby expanding their outreach and impact.

This collaboration helps in disseminating modern agricultural knowledge and promoting sustainable farming practices.

4. The Community and Economy

The wider community and national economy also benefit from Agrinet's implementation.

- Rural communities gain from increased employment opportunities and income circulation.

- Agricultural productivity and market efficiency improve, contributing to food security.
- The overall economy benefits from digital inclusion, technological advancement, and sustainable development within the agricultural sector.

5. Developers

- Improve skills in system analysis, requirements gathering, and architectural design.
- Enhance problem-solving, debugging, and software testing skills.
- Build teamwork, collaboration, and professional readiness for real-world software engineering careers.

1.8. Time Schedule

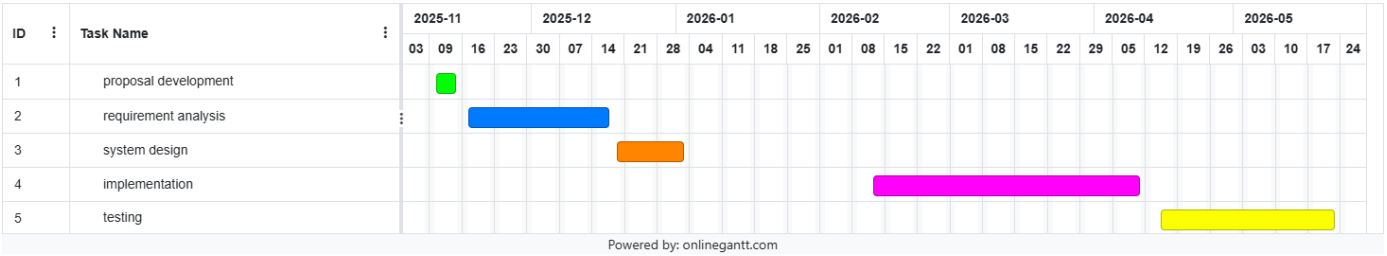


Figure 1 Gant Chart for time schedule

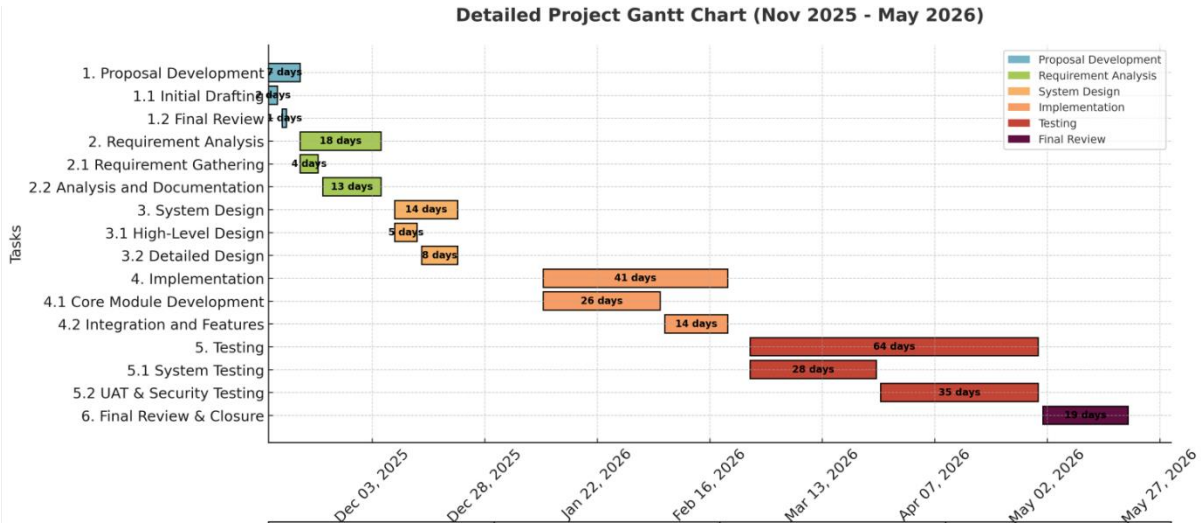


Figure 2 Time Management Plan

2.CHAPTER TWO – REQUIREMENT ANALYSIS

2.1. Introduction

Agrinet is an online platform designed to bridge the gap between farmers, buyers, and agricultural service providers. In many farming communities, access to reliable markets, up-to-date information, and trusted suppliers is often limited. The platform aims to address these challenges by creating a digital space where farmers can showcase their products, buyers can easily find what they need, and both parties can interact more efficiently. By bringing agricultural stakeholders together on one platform, the project supports fair trade, improves market visibility, and helps modernize traditional farming practices through technology.

Purpose of the System:

The purpose of the project system is to provide a convenient, accessible, and efficient platform that connects farmers with potential buyers and service providers. The system aims to:

- Simplify the buying and selling process for agricultural goods.
- Give farmers better exposure to wider markets.
- Enable buyers to find quality products at competitive prices.
- Provide a reliable communication channel between users.
- Support agricultural communities through improved information sharing and accessibility.

2.2. Current System

2.2.1. Current System Description

At present, most agricultural transactions in the target community are carried out through traditional, manual methods. Farmers typically rely on local marketplaces, personal networks, or middlemen to sell their products. This often results in limited market reach, inconsistent pricing, and reduced profit for farmers. Buyers, on the other hand, face challenges in finding reliable suppliers, comparing product quality, and accessing updated information on available crops or services.

Communication between farmers and buyers is mostly informal, making it difficult to ensure transparency, timely updates, or organized record-keeping. There is no centralized platform where agricultural stakeholders can interact, share information, or conduct business efficiently. Because of this, opportunities for growth, fair competition, and wider market access are often missed.

2.2.2. Problem of Current System

The current agricultural trading process relies heavily on traditional, manual methods. While these methods have been used for years, they come with several limitations that affect both farmers and buyers. The lack of a centralized digital platform creates gaps in communication, market access, and pricing transparency. These issues make it difficult for stakeholders to trade efficiently and fairly.

Key Problems in the Current System:

- **Limited Market Reach**: Farmers are often restricted to selling within their local area, reducing their chances of finding better buyers or competitive prices.
- **Dependence on Middlemen**: Intermediaries often control pricing and product movement, which can lower farmers' profit margins.
- **Lack of Pricing Transparency**: Prices vary greatly from one location to another, and farmers have no reliable way to compare or confirm fair market rates.
- **Inefficient Communication**: Buyers and farmers rely on phone calls, word of mouth, or physical visits, causing delays and miscommunication.
- **Poor Information Accessibility**: Farmers have limited access to updated information on market trends, product demand, and available services.
- **No Centralized Record Keeping**: Transactions are mostly unrecorded, leading to issues in tracking sales, resolving disputes, or analyzing business performance.
- **Time-Consuming Processes**: Manual buying and selling require physical presence and coordination, resulting in delays and inefficiencies.

The existing agricultural trading process is slow, fragmented, and heavily dependent on manual interaction. Farmers have limited market reach and often rely on middlemen, resulting in reduced profits and inconsistent pricing. Communication between farmers and buyers is inefficient, leading to delays and misunderstandings. With no centralized platform for information or record-keeping, stakeholders struggle to access reliable market data, compare prices, or track transactions. Overall, the current system lacks transparency, accessibility, and efficiency—hindering fair trade and the growth of the agricultural community.

2.3. Requirement Gathering

2.3.1. Requirement Gathering Methodologies

To ensure the Agrinet system accurately reflects the needs of its end users, several requirement-gathering methodologies were applied. These methods help collect reliable information and better understand the challenges faced by farmers and buyers.

The following methods:

1. Observation

Real-world market activities and interactions were observed to understand how agricultural trading currently works. This provided a better understanding of workflow patterns and common difficulties faced during transactions.

2. Document Analysis

Existing documents such as market reports, transaction records, and agricultural guidelines were reviewed. This helped in identifying gaps, recurring problems, and areas where digitalization could bring improvements.

3. Brainstorming Sessions

Collaborative discussions were held with the project team to explore potential solutions and refine system requirements. This ensured that the system design aligns with both users needs and technical possibilities.

4. Online Research

Online resources, including agricultural websites, digital marketplaces, and research articles, were studied to understand modern solutions and industry trends. This helped in comparing existing platforms and identifying useful features for Agrinet.

Online resources, including agricultural websites, digital marketplaces, and research articles, were explored to understand modern solutions and identify useful features for the proposed system.

5. Advisor Consultancy

Consultation sessions were held with our lecturer, who acted as the project advisor. Their guidance helped validate gathered requirements, refine system goals, and ensure our proposed solution aligns with academic standards and real-world needs.

2.3.2. Results Found

The requirement gathering process confirmed the critical challenges outlined in the current system analysis and yielded specific, actionable insights for the Agrinet system. The key results are as follows:

- **High Dependence on Intermediaries:** A significant majority of farmers reported that middlemen control market access, consistently leading to reduced profit margins and a feeling of powerlessness in price negotiation.
- **Critical Need for Pricing Transparency:** Both farmers and buyers expressed a strong need for a centralized source of real-time, localized market prices to ensure fair trade and enable informed decision-making.
- **Demand for a Centralized Marketplace:** There is a clear consensus for a single, reliable platform to connect farmers directly with a wider range of buyers, including retailers, wholesalers, and institutional buyers, to break geographical limitations.
- **Inefficiency in Communication:** The current reliance on phone calls and physical meetings was identified as a major source of delay, inefficiency, and potential for miscommunication, creating a need for integrated, in-platform messaging and notification systems.
- **Digital Record-Keeping is a Priority:** Users highlighted the difficulties of manual transaction tracking. A strong requirement emerged for automated digital records of sales, purchases, and communications to simplify management and resolve disputes.
- **Feature Prioritization from Market Research:** Analysis of existing platforms validated the need for core features such as user profiles with ratings/reviews, a searchable product catalog, and a secure transaction framework. Additionally, potential for advanced features like demand forecasting and logistics coordination was identified.
- **Validation of System Purpose:** The findings directly validate the proposed purpose of the platform, confirming that a platform designed to simplify trade, improve market exposure, and provide a reliable communication channel is not only desirable but necessary to modernize the agricultural trading ecosystem.

2.4. Proposed System

2.4.1. Overview

Agrinet is a comprehensive digital agricultural platform designed to transform traditional farming trade and communication by leveraging modern web technologies. It serves as an integrated online ecosystem that directly connects farmers with buyers, eliminates middlemen, and provides access to expert knowledge, logistics support, and a collaborative community.

The overarching goal of Agrinet is to create a modern, efficient, and transparent digital ecosystem that empowers farmers by connecting them directly with buyers and providing them with the tools and knowledge needed to thrive.

This general objective breaks down into several key aspirations:

- **Bridge the Market Gap:** To eliminate the physical and informational distance between farmers and consumers/retailers/wholesalers.
- **Disintermediate the Supply Chain:** To remove the dominance of middlemen who reduce farmers' profits, ensuring fairer prices for producers and better prices for buyers.
- **Empower Through Knowledge:** To go beyond being a marketplace by becoming an information hub, providing expert advice on farming techniques, disease control, and market trends.
- **Modernize Agricultural Practices:** To leverage technology (digital platforms, data-driven insights) to transform traditional, inefficient farming and trading methods.
- **Build an Agricultural Community:** To foster collaboration and knowledge-sharing among farmers, creating a supportive network that drives collective growth.

2.4.2. Functional Requirements

- **User Registration and Profile Management (FR1)**

Description: The system must allow all users including farmers, buyers, and agricultural experts to register for an account by providing essential details.

Benefit: This creates a verified and accountable user community, which forms the foundation for trust and security on the platform. It also enables personalized user experiences, such as tailored product suggestions and relevant advisory content.

- **User Authentication and Authorization (FR2)**

Description: A secure login system must be implemented to verify user identities, typically through an email and password combination.

Benefit: This protects sensitive user data and financial transactions, ensuring that only authorized individuals can perform critical actions, thereby safeguarding the entire ecosystem.

- **Product Listing and Catalog Management (FR3)**

Description: The system must enable farmers to easily add, update, and remove products available for sale. These products must then be displayed in a centralized, searchable online catalog.

Benefit: This provides farmers with a digital storefront, dramatically expanding their market reach beyond local boundaries and directly addressing the problem of limited exposure identified in the current system.

- **Product Search, Filter, and Browse (FR4)**

Description: Buyers must be able to efficiently locate products by using search functions and applying filters based on criteria such as product category, price range, geographical location, and quality ratings.

Benefit: This significantly reduces the time and effort buyers spend sourcing agricultural goods, enhancing the user experience and making the platform a more attractive and practical tool for trade.

- **Shopping Cart and Order Management (FR5)**

Description: The system must provide buyers with a digital shopping cart to select multiple products and a streamlined process for placing orders.

Benefit: This formalizes and simplifies the transaction process, reducing manual errors and coordination delays, and creating a clear, auditable record for reference and dispute resolution.

- **Expert Advisory Portal (FR6)**

Description: The system must include a dedicated section where agricultural experts can publish articles, guides, and market alerts. In an advanced implementation, a feature for farmers to submit specific queries to experts should be available.

Benefit: This democratizes access to specialized knowledge, aiding in improving crop yields, managing livestock health, and adopting modern practices, ultimately leading to increased productivity and income.

- **Admin Dashboard and Moderation (FR7)**

Description: A comprehensive administrative dashboard must be available for system administrators to manage users, moderate content in the marketplace and forum, resolve disputes, and monitor system analytics.

Benefit: This ensures the platform remains a safe, orderly, and trustworthy environment through proactive management and oversight, which is essential for long-term user retention and platform integrity.

2.4.3. Non-Functional Requirements

- **Usability:** The platform features an intuitive and user-friendly interface designed for users with varying levels of digital literacy, ensuring ease of navigation and task completion.
- **Accessibility:** The system is accessible across various devices and compatible with different operating systems and browsers.
- **Performance:** The platform responds quickly to user interactions, ensuring a seamless and responsive user experience.
- **Reliability:** The system maintains consistent availability and operates without unexpected failures under normal usage conditions.
- **Security:** Robust security measures are implemented to protect user data, including secure authentication methods and transparent privacy settings.
- **Scalability:** The architecture is designed to support scalability as the user base grows, ensuring consistent performance.
- **Maintainability:** The system is designed with modular components and clear documentation to facilitate easy updates and maintenance.
- **Compatibility:** The platform is compatible with popular web browsers and mobile devices to ensure a consistent experience for all users.
- **Data Integrity:** The system ensures accurate and consistent storage and processing of all user data and transaction information.
- **Cost-Effectiveness:** The platform utilizes resources efficiently to minimize operational costs while maintaining required performance standards.

2.4.3.1.User Interface and Human Factors

1. Intuitive Navigation

- The interface employs clear visual hierarchies, familiar icons, and a consistent layout
- Primary navigation menu remains accessible across all pages
- Benefits users with low digital literacy by reducing learning time and cognitive load

2. Mobile-First Responsive Design

- Interface adapts seamlessly to various screen sizes (mobile, tablet, desktop)
- Touch-friendly buttons and controls for mobile users
- Ensures accessibility for farmers who primarily use mobile devices

3. Localized Content Presentation

- Uses culturally appropriate symbols and color schemes
- Makes the platform welcoming and understandable to diverse user groups

4. Accessible Visual Design

- High contrast ratios for text readability in various lighting conditions
- Appropriate font sizes and spacing for users with visual impairments
- Color-blind friendly palettes that don't rely solely on color to convey information

5. Progressive Disclosure

- Complex features are revealed gradually as users become more proficient
- Simplified workflows for common tasks (e.g., quick product listing)
- Prevents overwhelming new users while providing advanced functionality

6. Contextual Help and Guidance

- Inline tooltips and help text for unfamiliar features
- Reduces support needs and builds user confidence

7. Error Prevention and Recovery

- Clear error messages with suggested solutions
- Confirmation dialogs for critical actions (e.g., order cancellation)

8. Consistent Interaction Patterns

- Uniform button styles and interaction feedback across the platform

- Predictable placement of common actions and controls
- Creates a cohesive experience that builds user trust and efficiency

9. Offline Consideration

- Graceful handling of poor network conditions
- Clear indicators when connectivity is required for specific actions
- Accommodates users in areas with unreliable internet access

10. Performance Feedback

- Loading indicators for operations that take longer than 500ms

- Immediate visual feedback for user interactions
- Manages user expectations and reduces frustration during wait times

2.4.3.2. Documentation

1. User Documentation

- **User Manual:** Comprehensive guide covering registration, navigation, and all platform features with step-by-step instructions
- **Quick Start Guide:** Simplified booklet for immediate platform usage, focusing on essential functions
- **Video Tutorials:** Visual demonstrations of key processes like product listing and order management
- **FAQ Section:** Answers to common questions and troubleshooting guidance

2. Technical Documentation

- **System Requirements Specification:** Detailed functional and non-functional requirements
- **System Design Document:** Architecture diagrams, database schema, and component specifications
- **API Documentation:** Complete reference for all system endpoints and integration points
- **Database Documentation:** Entity-relationship diagrams and data dictionary

3. Development Documentation

- **Source Code Documentation:** Inline code comments and module descriptions
- **Setup Guide:** Development environment configuration and deployment instructions
- **Testing Documentation:** Test cases, results, and quality assurance procedures
- **Version Control Guide:** Branching strategy and commit standards

4. Administrative Documentation

- **System Administration Guide:** Daily operations, monitoring, and maintenance procedures
- **Backup and Recovery Guide:** Data protection strategies and disaster recovery plans
- **Security Protocol:** Access controls, security measures, and incident response procedures
- **User Management Guide:** Administrative functions for managing user accounts and content

5. Project Documentation

- **Project Proposal:** Initial concept, objectives, and scope definition
- **Requirement Analysis:** User needs assessment and requirement gathering methodology
- **Design Mockups:** Wireframes and UI/UX design specifications
- **Implementation Plan:** Development timeline and resource allocation

6. Maintenance Documentation

- **Update Procedures:** Guidelines for applying patches and system upgrades
- **Bug Tracking Documentation:** Error logging and resolution processes
- **Performance Monitoring Guide:** System health checks and optimization techniques
- **Change Management:** Procedures for implementing system modifications

7. Training Documentation

- **Training Manuals:** Materials for user training sessions
- **Presentation Slides:** Educational content for workshops and demonstrations
- **Hands-on Exercises:** Practical activities for user skill development
- **Assessment Materials:** Tools for evaluating user understanding and platform adoption

8. Compliance and Standards Documentation

- **Privacy Policy:** Data handling and protection compliance documentation
- **Terms of Service:** Platform usage rules and regulations
- **Accessibility Compliance:** Documentation of WCAG adherence
- **Industry Standards:** Alignment with agricultural and technology standards

2.4.3.3.Hardware Consideration

1. Server Infrastructure

- **Cloud-Based Servers:** Utilization of scalable cloud services (AWS, Azure, or Google Cloud) for flexible resource allocation
- **Load Balancers:** Distribution of traffic across multiple servers to ensure optimal performance
- **Redundant Storage Systems:** Implementation of RAID configurations or cloud storage replication for data safety

- **Backup Power Supplies:** UPS systems and generator backups for continuous operation during power outages

2. Client-Side Hardware Requirements

- **Mobile Devices:** Compatibility with Android and iOS smartphones with minimum 2GB RAM
- **Tablets:** Support for various screen sizes and touch interfaces
- **Desktop Computers:** Compatibility with standard Windows, macOS, and Linux systems
- **Minimum Specifications:** 4GB RAM, dual-core processor, and 500MB storage space for app installation

3. Network Infrastructure

- **Internet Connectivity:** Minimum 3G connectivity for mobile users, broadband for desktop users
- **Data Transfer Capacity:** Sufficient bandwidth to handle image uploads and real-time messaging
- **CDN Integration:** Content Delivery Network for faster loading of images and static content
- **Network Security:** Firewalls and encryption protocols to secure data transmission

4. Peripheral Device Support

- **Camera Integration:** Mobile device cameras for product photography and documentation
- **GPS Functionality:** Location services for mapping and logistics coordination
- **Printing Capabilities:** Support for local printing of transaction receipts and records
- **External Storage:** Ability to export data to external storage devices

5. Data Center Requirements

- **Climate Control:** Proper cooling systems to maintain optimal server operating temperatures
- **Physical Security:** Secure facilities with access control and surveillance systems
- **Redundant Internet Connections:** Multiple ISP connections for uninterrupted service
- **Disaster Recovery Site:** Secondary location for business continuity planning

6. Power Management

- **Energy-Efficient Hardware:** Selection of hardware with lower power consumption
- **Power Backup Systems:** Automatic failover to backup power sources
- **Voltage Regulation:** Protection against power surges and fluctuations
- **Remote Management:** Ability to monitor and control hardware power states remotely

7. Environmental Considerations

- **Dust Protection:** Hardware enclosures suitable for rural environments
- **Temperature Tolerance:** Equipment capable of operating in varying climate conditions
- **Portable Solutions:** Mobile device compatibility for field use in farming environments
- **Low-Power Options:** Support for energy-efficient operation in areas with unreliable electricity

8. Maintenance Hardware

- **Diagnostic Tools:** Hardware monitoring and troubleshooting equipment
- **Replacement Parts:** Inventory of critical components for quick repairs
- **Testing Equipment:** Network and hardware performance testing tools
- **Spare Devices:** Backup hardware for critical system components

2.4.3.4 Performance Characteristics

The system is engineered to deliver consistently fast and responsive performance across all user interactions, ensuring quick page loads, seamless navigation, and efficient transaction processing even during periods of high user activity. It maintains high availability and reliability through robust architecture that minimizes downtime and supports smooth operation under varying loads. The platform is designed to scale effectively to accommodate growing numbers of users and transactions without compromising speed or functionality. Performance is optimized for diverse devices and network conditions, providing a fluid user experience while efficiently managing system resources and maintaining strong security measures without impacting responsiveness.

2.4.3.5Error Handling

The platform implements comprehensive error handling mechanisms to ensure system stability and user confidence. The system proactively validates user inputs to prevent common data entry errors and provides clear, actionable error messages in plain language to guide users toward resolution. For technical failures, the system employs graceful

degradation, maintaining core functionality even when non-essential features experience issues. All errors are automatically logged with complete contextual information for developer analysis while protecting users from exposure to technical details. The platform includes automated recovery procedures for transaction failures, ensuring data consistency through rollback mechanisms and retry protocols. Network connectivity issues are managed through intelligent queuing of operations for synchronization when connection is restored, and users receive appropriate feedback about offline status. For authentication and authorization errors, the system provides specific guidance without compromising security. System-level errors trigger immediate alerts to administrators while maintaining a professional user interface that reassures users and provides alternative pathways to complete their tasks.

2.4.3.6.Quality Issues

1. Functional Quality

- **Requirement Compliance:** Ensuring all system features fully align with documented user requirements and expectations
- **Feature Reliability:** Guaranteeing consistent performance of all platform functions without unexpected failures
- **Transaction Integrity:** Maintaining accurate and complete processing of all financial and trade transactions
- **Data Accuracy:** Ensuring all stored information reflects correct user inputs and system calculations

2. Performance Quality

- **Response Consistency:** Maintaining uniform system responsiveness across different usage scenarios and load conditions
- **Resource Efficiency:** Optimizing hardware and software resource utilization without excessive consumption
- **Scalability Maintenance:** Preserving system performance standards as user base and transaction volumes increase
- **Connection Stability:** Providing reliable service across varying network conditions and connection qualities

3. Security Quality

- **Data Protection:** Implementing comprehensive measures to safeguard user information and business data
- **Access Control:** Maintaining strict authentication and authorization mechanisms

- **Vulnerability Management:** Regularly identifying and addressing potential security weaknesses
- **Privacy Compliance:** Adhering to data protection regulations and privacy standards

4. Usability Quality

- **Interface Consistency:** Ensuring uniform design language and interaction patterns throughout the platform
- **Navigation Clarity:** Providing intuitive pathways for users to accomplish their tasks efficiently
- **Accessibility Standards:** Meeting requirements for users with diverse abilities and device capabilities
- **Learning Curve:** Minimizing the time and effort required for new users to become proficient

5. Compatibility Quality

- **Browser Support:** Maintaining consistent functionality across different web browsers and versions
- **Device Adaptation:** Ensuring proper rendering and operation on various mobile devices and screen sizes
- **Operating System Compatibility:** Supporting different operating environments without functional degradation
- **Network Adaptation:** Performing reliably under various internet connection speeds and qualities

6. Maintainability Quality

- **Code Quality:** Following established coding standards and best practices for long-term sustainability
- **Documentation Completeness:** Providing comprehensive and up-to-date technical and user documentation
- **Update Management:** Facilitating smooth deployment of patches, updates, and new features
- **Troubleshooting Support:** Enabling efficient diagnosis and resolution of system issues

7. Reliability Quality

- **System Availability:** Ensuring the platform remains accessible and operational as expected

- **Error Recovery:** Implementing effective mechanisms to detect, report, and recover from system failures
- **Data Integrity:** Preventing corruption or loss of critical user and transaction data
- **Backup Effectiveness:** Maintaining reliable data backup and restoration procedures

8. Support Quality

- **User Assistance:** Providing effective help resources and responsive customer support
- **Feedback Processing:** Establishing efficient channels for user feedback and feature requests
- **Problem Resolution:** Ensuring timely and effective solutions to user-reported issues
- **Training Effectiveness:** Delivering comprehensive user education and training materials

2.4.3.7. System Modifications

1. Change Management Process

- **Modification Request System:** Formal procedure for submitting, reviewing, and approving system change requests
- **Impact Analysis:** Comprehensive assessment of how modifications affect existing functionality, performance, and security
- **Approval Workflow:** Structured authorization process involving technical leads, project managers, and stakeholder representatives
- **Version Control:** Systematic tracking of all changes through dedicated branching and merging strategies

2. Update Deployment Procedures

- **Staged Rollouts:** Gradual deployment of changes through development, staging, and production environments
- **Rollback Mechanisms:** Automated procedures to revert changes in case of deployment failures or unexpected issues
- **Database Migration:** Controlled scripts for schema changes and data transformations with backup protection
- **Dependency Management:** Careful coordination of updates to third-party libraries and external service integrations

3. Feature Enhancement Framework

- **Modular Architecture:** Component-based design allowing independent updates to specific system features

- **API Versioning:** Backward-compatible interface management for seamless third-party integration updates
- **Configuration Management:** Externalized settings for easy feature tuning without code modifications
- **A/B Testing Infrastructure:** Controlled rollout of new features to specific user segments for performance validation

4. Maintenance Modifications

- **Security Patching:** Regular updates to address vulnerabilities and maintain compliance standards
- **Performance Optimization:** Continuous improvements to database queries, caching strategies, and resource utilization
- **Third-Party Updates:** Scheduled integration of updates from external services and platform dependencies
- **Code Refactoring:** Ongoing restructuring to improve maintainability and technical debt reduction

5. Scalability Modifications

- **Infrastructure Scaling:** Planned capacity increases to handle user growth and seasonal demand fluctuations
- **Architecture Evolution:** Strategic upgrades to system architecture supporting long-term growth
- **Database Optimization:** Periodic restructuring and indexing for maintaining performance with expanding data
- **Load Balancing Adjustments:** Dynamic distribution of traffic across available system resources

2.4.3.8. Physical Environment

The system is designed to operate effectively across diverse physical environments, from controlled data centers with advanced climate and security systems to rural agricultural settings with potential exposure to dust, humidity, and variable temperatures. The platform supports usage on mobile devices in outdoor conditions with sunlight-readable displays and resilience to weather elements, while remaining functional in areas with unreliable power or intermittent internet connectivity. Administrative workspaces are configured for ergonomic comfort and efficient operation, and all hardware selections prioritize durability, energy efficiency, and environmental sustainability to ensure reliable performance across the varied conditions encountered by farmers, field agents, and support staff.

2.4.3.9. Security Issues

The comprehensive security concerns through robust data protection measures including strong encryption for all stored and transmitted information, multi-factor authentication and role-based access controls to prevent unauthorized system entry, and rigorous input validation and API security to defend against application-level attacks. The platform implements network security protocols including firewalls and intrusion detection systems, ensures secure payment processing with fraud monitoring, and maintains physical security for server infrastructure, while establishing clear audit trails for all transactions and providing users with transparent privacy controls to manage their data.

2.4.3.10. Resource Issues

The platform addresses several critical resource considerations to ensure sustainable operation and accessibility. Given the primary user base of farmers, many in rural areas, the system is designed for efficient bandwidth usage and low data consumption to accommodate limited internet connectivity.

Server resources are optimized through scalable cloud infrastructure that can handle fluctuating demand during harvest seasons and market peaks without performance degradation. The application maintains minimal hardware requirements to ensure compatibility with older mobile devices and tablets commonly used in agricultural communities. Storage resources are managed through automated data archiving and compression strategies to control costs while maintaining quick access to active transactions and user data. Development resources follow a modular approach, allowing efficient maintenance and feature updates without requiring complete system overhauls. Energy consumption is minimized at both server and client levels, supporting longer battery life for mobile devices used in field conditions. Human resource needs are reduced through intuitive interfaces that minimize training requirements and automated administrative tools that streamline platform management.

The system is strategically designed to navigate critical resource constraints by prioritizing low data consumption and efficient bandwidth use for users in areas with limited internet connectivity. It employs a scalable cloud infrastructure to manage fluctuating demand cost-effectively and is built as a lightweight application to ensure compatibility with older mobile devices and tablets. This focus on resource efficiency ensures the platform remains accessible, affordable, and performant for its core user base in the agricultural community.

2.5. Constraints/Pseudo Requirements

1. Technical Constraints

- **Platform Compatibility:** Must operate on Android and iOS mobile devices with limited hardware capabilities
- **Network Limitations:** Must function effectively in low-bandwidth and intermittent connectivity environments
- **Data Usage:** Must minimize mobile data consumption for cost-sensitive users
- **Storage Limits:** Must optimize media storage and implement automatic cleanup protocols

2. Operational Constraints

- **User Literacy:** Interface must accommodate users with varying levels of digital literacy and education
- **Language Support:** Must provide content in multiple local languages and dialects
- **Power Availability:** Must consider limited electricity access in rural operating areas
- **Maintenance Access:** Must account for limited technical support availability in remote regions

3. Business Constraints

- **Budget Limitations:** Development and operational costs must remain within constrained project funding
- **Deployment Timeline:** Must meet academic and seasonal agricultural deadlines
- **Scalability Requirements:** Must support gradual user adoption without initial over-provisioning
- **Third-Party Dependencies:** Must minimize reliance on paid external services and APIs

4. Regulatory Constraints

- **Data Privacy:** Must comply with local data protection and privacy regulations
- **Agricultural Standards:** Must adhere to national agricultural trading policies and guidelines
- **Financial Compliance:** Must follow relevant regulations for transaction processing
- **Accessibility Requirements:** Must meet basic accessibility standards for diverse user abilities

5. Environmental Constraints

- **Device Durability:** Must account for exposure to dust, moisture, and variable temperatures
- **Power Conservation:** Must optimize battery usage for extended field operation
- **Network Reliability:** Must withstand inconsistent cellular coverage in rural areas
- **Physical Access:** Must consider limited hardware availability and repair services

6. Implementation Constraints

- **Development Resources:** Limited to available team members with specific technical skills
- **Testing Limitations:** Restricted access to diverse real-world user environments for testing
- **Documentation Requirements:** Must provide comprehensive documentation within time constraints
- **Integration Boundaries:** Limited control over third-party service reliability and performance

7. Cultural Constraints

- **Social Acceptance:** Must align with local agricultural practices and community norms
- **Trust Building:** Must overcome traditional resistance to digital trading platforms
- **Knowledge Transfer:** Must accommodate varied levels of technology adoption readiness
- **Community Integration:** Must complement existing social and trading networks

2.6. System Model

The system model provides a structured representation of how the Agrinet system behaves, communicates, and responds to user interactions. It does not describe the internal code or algorithms, but rather how different parts of the system work together from the user's point of view.

For this project, the system model focuses on showing:

- How farmers, buyers, administrators, and the system itself interact
- The sequence of actions taken to achieve a specific goal
- The conditions that must be true before a process begins
- The expected system response after the process completes

By modeling these interactions, the project gains clarity about what the system should do, how users will engage with it, and how the system should behave under different conditions. This ensures that all requirements are clearly mapped to real-world workflows, reducing ambiguity during system development.

2.6.1. Scenario

A scenario represents a realistic, step-by-step narrative of how a particular functionality in the system will be used. It identifies the actor involved, the goal they want to achieve, and the orderly flow of actions that take place during the interaction.

Table 1: Scenario – User Registration

SID	001
Scenario Name	User Registration
Participating Actor Instance	Abebe(user)
Pre-condition	Abebe is not registered in the system
Flow of Events	1. Abebe navigates to the registration page. 2. Abebe enters required details and submits the form. 3. System validates all inputs. 4. System creates a new account. 5. Abebe receives a successful registration confirmation
Alternative Flow(Failure)	1. Abebeenters incomplete or invalid information. 2. System validation fails 3. System displays an error message indicating incorrect fields. 4. Registration is not completed until Abebe corrects the information.

Table 2: Scenario – User Login

SID	002
Scenario Name	User Login

Participating Actor Instance	Abebe (Registered user)
Pre-condition	Abebehas a valid account
Flow of Events	1. Abebe opens the login page. 2. Abebeenters credentials. 3. System verifies credentials. 4. Abebeis granted access to their homepage.
Alternative flow(failure)	1.Abebe enters incorrect credentials 2.System rejects the login attempt 3. System displays “Invalid username or password” 4. Abebe remains on the login page.

Table 3: Scenario – Post a Product for Sale

SID	003
Scenario Name	Product Posting
Participating Actor Instance	Abebe(Farmer / Seller)
Pre-condition	Abebe is logged in
Flow of Events	1. Abebe selects “Post Product” 2. Abebeenters product details and uploads images. 3. System validates data. 4. Product is published and visible to buyers.
Alternative flow(failure)	1. Abebe submits incomplete or invalid product details. 2.System validation fails 3. System displays error messages for required or incorrect fields 4.Product is not published until corrections are made.

Table 4: Scenario – Browse Products

SID	004
Scenario Name	Product Browsing
Participating Actor Instance	Abebe (Buyer)
Pre-condition	Abebeis logged in
Flow of Events	1. Abebegoes to the homepage or marketplace page. 2. System displays available products. 3. Abebe scrolls, filters or searches products.
Alternative Flow(Failure)	1. System fails to load products OR no products exist. 2. System displays a “No products available” message or an error. 3. Abebe cannot browse products.

Table 5: Scenario – View Product Details

SID	005
Scenario Name	Product Detail Viewing
Participating Actor Instance	Abebe (Buyer)
Pre-condition	Products exist in the system
Flow of Events	1. Abebeselects a product. 2. System selects a product. 3. Abebe decides whether to contact the seller.
Alternative Flow(Failure)	1. Product has been removed, unpublished, or is temporarily unavailable. 2. System fails to load the product details. 3. System displays an error or redirects Abebe back to the product list.

Table 6: Scenario – Chat Between Buyer and Seller

SID	006
Scenario Name	Buyer-Seller Chat
Participating Actor Instance	Abebe (Buyer), Kebede(Seller)
Pre-condition	Abebeis logged in and viewing a product
Flow of Events	1. Abebe clicks “Chat with kebede” 2. System opens the chat interface. 3. Abebe sends a message. 4. Kebede receives and responds. 5. Conversation continues until the goal is reached
Alternative Flow(Failure)	1. Abebe attempts to open the chat. 2. System fails to start chat due to network issues or seller being unavailable/blocked 3. System displays error message. 4.No conversation begins

Table 7: Scenario – Report a User

SID	007
Scenario Name	Repost User
Participating Actor Instance	Abebe(any user buyer, seller or expert)
Pre-condition	Abebeencounters suspicious behavior

Flow of Events	1. Abebeopens the suspicious profile. 2. Abebeselects “Report”. 3. Abebe enters a reason. 4. System stores report and notifies admin.
Alternative Flow(Failure)	1.Abebe enters an invalid or empty reason 2. System validation fails. 3. System displays error message like “Reason required to submit report”. 4. Report is not submitted.

Table 8: Scenario – Admin Manages Users

SID	008
Scenario Name	User Management
Participating Actor Instance	Abebe(Admin)
Pre-condition	Abebe is logged in
Flow of Events	1. Abebe opens user management panel. 2. Abebe views list of users. 3. Abebe blocks, unblocks or deletes a user. 4. System updates the user’s status.
Alternative Flow(Failure)	1. Abebe attempts to load the user management panel. 2. System fails due to server or database error. 3. User list does not load. 4.Abebe cannot manage users until issue is resolved

Table 9: Scenario – Edit User Profile

SID	009
Scenario Name	Profile Editing
Participating Actor Instance	Abebe (Registered User)
Pre-condition	Abebe is logged in
Flow of Events	1. Abebeopens their profile page. 2. Abebeedits details. 3. System validates the inputs. 4. Profile is updated successfully.
Alternative Flow(Failure)	1. Abebe enters invalid or missing information. 2. System validation fails. 3. System displays error messages for invalid fields. 4. Profile remains unchanged.

Table 10: Scenario – Verification of Users

SID	0010
Scenario Name	User Verification
Participating Actor Instance	Abebe (Seller / Admin)
Pre-condition	User Submitted verification data
Flow of Events	1. Abebe uploads verification documents. 2. Admin reviews documents. 3. Admin approves or rejects verification. 4. System marks account as “verified” if approved.

Alternative Flow(Failure)	1. Abebe uploads unclear or invalid verification documents 2. Admin reviews the submission. 3. Admin rejects the verification request. 4. System marks the account as “Verification Failed”.

Table 11: Scenario – Edit Product

SID	0011
Scenario Name	Edit Product Information
Participating Actor Instance	Abebe (Farmer/Seller)
Pre-condition	Abebe is logged in and has posted at least one product
Flow of Events	1. Abebe navigates to his products. 2. Abebe selects a product to edit. 3.Abebe modifies product details (price, description, quantity) 4. System validates the updated information. 5.System saves the changes and displays a success message.
Alternative Flow (Failure)	1. Abebe enters invalid or incomplete data. 2. System validation fails. 3. System displays error messages. 4. Product details remain unchanged.

Table 12: Scenario – Delete Product

SID	0012
Scenario Name	Delete Product
Participating Actor Instance	Abebe (Farmer / Seller)
Pre-condition	Abebe is logged in and has listed products

Flow of Events	1. Abebe navigates to his products. 2. Abebe selects a product and clicks “Delete. 3.System asks for confirmation 4. Abebe confirms deletion. 5.System removes the product from the marketplace.
Alternative Flow(Failure)	1. Abebe cancels the deletion. 2. System aborts the delete operation. 3. Product remains available.

Table 13: Scenario – Add Product to Cart

SID	0013
Scenario Name	Add Product to Cart
Participating Actor Instance	Abebe (Buyer)
Pre-condition	Abebe is logged in and viewing product details
Flow of Events	1. Abebe clicks “Add to Cart”. 2. System checks product availability. 3. System adds the product to the shopping cart. 4. System displays a confirmation message.
Alternative Flow (Failure)	1. Product is out of stock. 2. System displays an “Out of stock” message. 3. Product is not added to the cart.

Table 14: Scenario – Place Order

SID	0014
Scenario Name	Place Order
Participating Actor Instance	Abebe (Buyer)
Pre-condition	Abebe has items in the cart
Flow of Events	1. Abebe reviews items in the cart. 2. Abebe confirms the order. 3. System processes payment. 4. System confirms the order and generates an order ID.
Alternative Flow (Failure)	1. Payment processing fails 2. System displays a payment failure message. 3. Order is not placed

Table 15: Scenario – View Orders

SID	0015
Scenario Name	View Orders
Participating Actor Instance	Abebe (Farmer/Seller)
Pre-condition	Abebe has received at least one order
Flow of Events	1. Abebe navigates to the orders page. 2. System displays a list of orders. 3. Abebe selects an order to view details
Alternative Flow (Failure)	1. No orders exist. 2. System displays a “No orders available” message.

Table 16: Scenario – Access Expert Advisory

SID	0016
Scenario Name	Access Expert Advisory
Participating Actor Instance	Abebe (Farmer/Seller)
Pre-condition	Abebe is logged in
Flow of Events	1. Abebe navigates to the expert advisory section. 2. System displays available articles and guides. 3. Abebe reads selected advisory content.
Alternative Flow (Failure)	1. Advisory content fails to load. 2. System displays an error message.

Table 17: Scenario – Admin Monitors System Analytics

SID	0017
Scenario Name	Admin Monitors System Analytics
Participating Actor Instance	Abebe (Admin)
Pre-condition	Abebe is logged in as admin
Flow of Events	1. Abebe navigates to the analytics dashboard. 2. System displays statistics for users, products, and orders. 3. Abebe reviews system performance metrics.
Alternative Flow (Failure)	1. Analytics data fails to load. 2. System displays a retry or error message.

Table 18: Scenario – Expert Responds to Farmer Queries

SID	0018
Scenario Name	Respond to Farmer Queries
Participating Actor Instance	Kebede (Agricultural Expert)
Pre-condition	Queries from farmers exist

Flow of Events	1. Kebede opens the farmer queries section. 2. Kebede selects a query. 3. Kebede provides recommendations. 4. System delivers the response to the farmer.
Alternative Flow (Failure)	1. Message delivery fails. 2. System displays a notification to retry.

2.6.2 Use Case Model

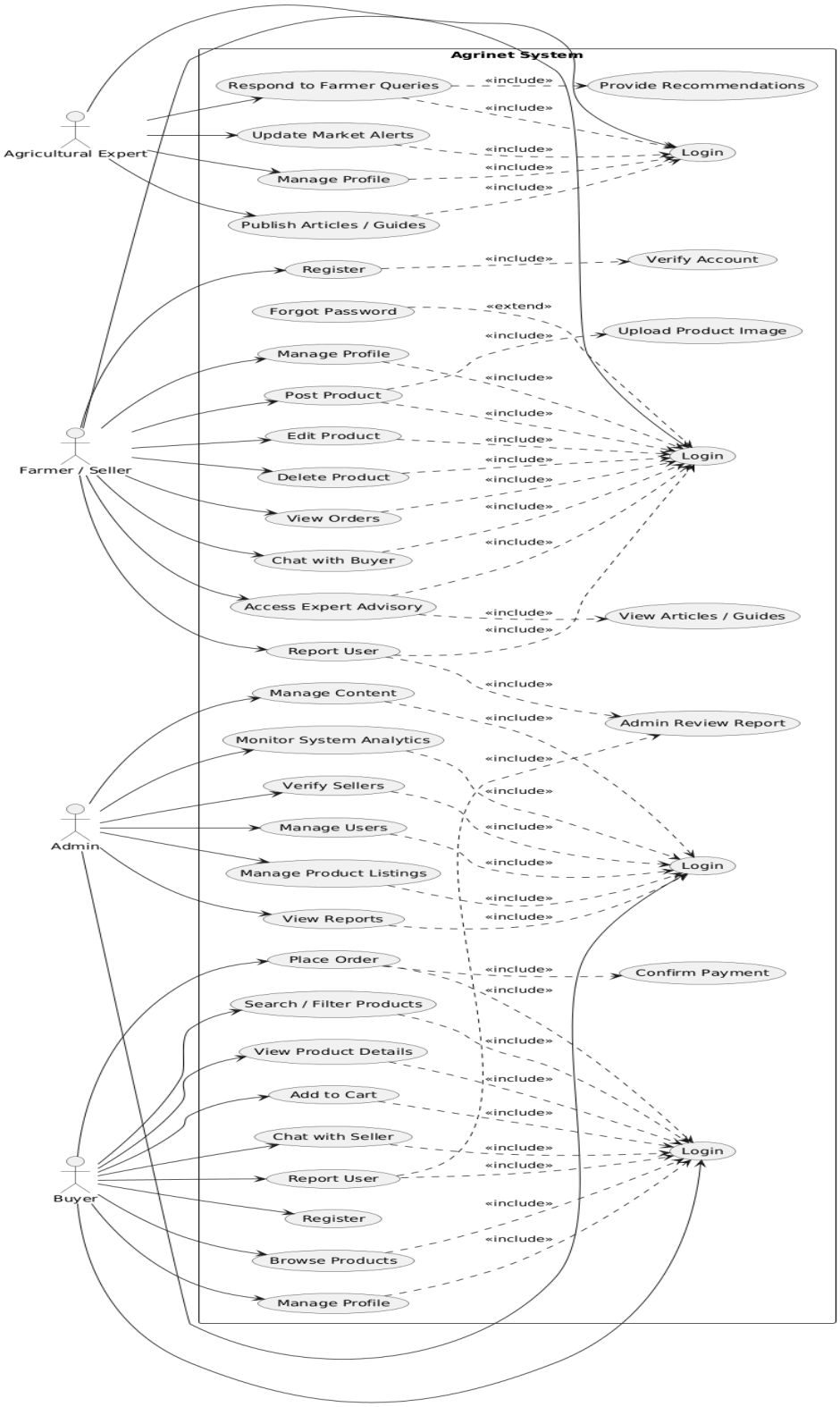


Figure 3 **use case diagram**

Description of Use-Case Model

Table 19 Use Case Account Registration

Use Case ID	UC001
Use Case Name	Register
Participation Actors	Farmer / Buyer / Expert
Entry Condition	User is not registered
Flow of Events	1. User navigates to the registration page 2. Userprovides necessary personal and business details. 3. System validates information. 4.System creates a new account. 5. User receives a confirmation message.
Alternate Flow	3.1. If information is invalid, the system prompts for correction. 4.1. Account creation fails(server error), the system prompts user to retry later.
Exit Conditions	User account is successfully registered.

Table 20 Use Case Login

Use Case ID	UC002
Use Case Name	Login
Participation Actors	Farmer / Buyer / Expert / Admin
Entry Condition	User has a registered account
Flow of Events	1. User navigates to login page 2. User enters username and password. 3. System verifies credentials. 4.System grants access to the system dashboard.
Alternate Flow	3.1. If credentials are invalid, system displays an error message. 3.1. Account locked, system displays lockout message.
Exit Conditions	User is successfully logged in.

Table 21 Use Case Manage Profile

Use Case ID	UC003
Use Case Name	Manage Profile

Participation Actors	Farmer / Buyer / Expert / Admin
Entry Condition	User is logged in
Flow of Events	1. User navigates to profile settings 2. Userupdates personal or business information. 3. System validates changes. 4.System saves the updated profile.
Alternate Flow	3.1. Invalid inputs prompt the user to correct data. 4.1. Save fails, system displays retry message
Exit Conditions	Profile is successfully updated.

Table 22 Use Case Post Product

Use Case ID	UC004
Use Case Name	Post Product
Participation Actors	Farmer / Seller
Entry Condition	Farmer is logged in
Flow of Events	1.Farmernavigates to “Post Product” page. 2.Farmer enters product details. 3. Farmeruploads product image. 4.System validates information. 5.System posts the product on the marketplace
Alternate Flow	4.1. Missing or invalid data prevents posting. 5.1. Posting fails(network/server error), system prompts retry.
Exit Conditions	Product is successfully listed.

Table 23 Use Case Edit Product

Use Case ID	UC005
Use Case Name	Edit Product
Participation Actors	Farmer / Seller
Entry Condition	Farmer has listed products
Flow of Events	1.Farmerselects a product to edit 2. Farmermodifies product information. 3. System validates updates. 4.System saves changes.
Alternate Flow	3.1. Invalid data, system requests correction. 4.1. Save fails, system prompts retry.
Exit Conditions	Product information is successfully updated

Table 24 Use Case Delete Product

Use Case ID	UC006
Use Case Name	Delete Product
Participation Actors	Farmer / Seller
Entry Condition	Farmer has listed products
Flow of Events	1.Farmerselects a product to delete 2.System requests confirmation. 3. Farmer confirms deletion. 4.System removes the product.
Alternate Flow	3.1. Farmer cancel, System aborts deletion. 4.1. Delete fails, System displays error.
Exit Conditions	Product is successfully deleted

Table 25 Use Case View Orders

Use Case ID	UC007
Use Case Name	View Orders
Participation Actors	Farmer / Seller
Entry Condition	Farmer has orders
Flow of Events	1.Farmernavigates to orders page 2.System displays a list of orders. 3. Farmer views order details.
Alternate Flow	2.1. No orders found, System displays empty-state message. 2.2. Load failure, System prompts retry.
Exit Conditions	Orders are successfully displayed.

Table 26 Use Case Chat with Buyer

Use Case ID	UC008
Use Case Name	Chat with Buyer
Participation Actors	Farmer / Seller
Entry Condition	Farmer has received a message or wants to

	contact buyer
Flow of Events	1.Farmeropens chat interface 2.Farmerselects a buyer. 3. Farmersends message. 4.System delivers the message.
Alternate Flow	4.1. Message not delivered, System notifies delivery failure.
Exit Conditions	Chat communication is successfully sent and received

Table 27 Use Case Access Expert Advisory

Use Case ID	UC009
Use Case Name	Access Expert Advisory
Participation Actors	Farmer / Seller
Entry Condition	Farmer is logged in
Flow of Events	1.Farmernavigates to expert advisory section 2.System displays articles and guides.
Exit Conditions	Advisory accessed

Table 28 Use Case Report User

Use Case ID	UC010
Use Case Name	Report User
Participation Actors	Farmer / Buyer
Entry Condition	User logged in
Flow of Events	1. Userselects a user to report. 2. User provides reason for report. 3. System forwards report to admin for review.
Alternate Flow	2.1. Missing reason, system requests input. 3.1. Report submission fails, system prompts retry.
Exit Conditions	Report is successfully submitted

Table 29 Use Case Browse Products

Use Case ID	UC011

Use Case Name	Browse Products
Participation Actors	Buyer
Entry Condition	Buyer is logged in
Flow of Events	1.Buyer navigates to product catalog. 2. System displays available products. 3. Buyer selects products for more information.
Alternate Flow	2.1. No products available, system shows “No products found.” 2.2. Loading error, system prompts user to retry.
Exit Conditions	Buyer browses products successfully.

Table 30 Use Case View Product Details

Use Case ID	UC012
Use Case Name	View Product Details
Participation Actors	Buyer
Entry Condition	Buyeris browsing products
Flow of Events	1.Buyer selects a product. 2.System displays product information and images.
Alternate Flow	2.1. Product details fail to load, system displays error and retry option.
Exit Conditions	Buyer successfully views product details.

Table 31 Use Case Add to Cart

Use Case ID	UC013
Use Case Name	Add to Cart
Participation Actors	Buyer
Entry Condition	Buyer has selected a product
Flow of Events	1.Buyer clicks “add to cart” 2.System adds the product to the shopping cart.
Alternate Flow	2.1. Product out of stock, system displays error. 2.2. Add-to-cart fails (server error), system prompts retry.
Exit Conditions	Product is successfully added to the cart.

Table 32 Use Case Place Order

Use Case ID	UC014
Use Case Name	Place Order
Participation Actors	Buyer
Entry Condition	Buyer has items in the cart
Flow of Events	1.Buyer reviews cart items. 2.Buyer confirms the order. 3. System processes payment 4.System confirms the order.
Alternate Flow	3.1. Payment failure, system prompts retry. 3.2. Network error during payment, system shows incomplete-payment warning.
Exit Conditions	Order is successfully placed.

Table 33 Use Case Chat with Seller

Use Case ID	UC015
Use Case Name	Chat with Seller
Participation Actors	Buyer
Entry Condition	User wants to communicate with seller
Flow of Events	1.Buyer opens chat interface. 2.Buyer selects a seller. 3. Buyer sends message. 4.System delivers the message.
Alternate Flow	4.1. Message delivery fails, system shows “Failed to send.”
Exit Conditions	Chat communication is successfully sent and received.

Table 34 Use Case Verify Sellers

Use Case ID	UC016
Use Case Name	Verify Sellers
Participation Actors	Admin
Entry Condition	Sellers have registered accounts

Flow of Events	1.Admin views pending seller registrations 2.Admin verifies documents and details. 3. System updates seller status.
Exit Conditions	Seller accounts are verified or rejected.

Table 35 Use Case Manage Users

Use Case ID	UC017
Use Case Name	Manage Users
Participation Actors	Admin
Entry Condition	Admin is logged in
Flow of Events	1.Admin views the user list. 2. Admin updates, disables, or deletes user accounts as necessary.
Alternate Flow	2.1. Action fails (server error), system prompts admin to retry. 2.2. Invalid update, system displays error.
Exit Conditions	Admin accounts are successfully managed.

Table 36 Use Case Manage Product Listings

Use Case ID	UC018
Use Case Name	Manage Product Listings
Participation Actors	Admin
Entry Condition	Admin is logged in
Flow of Events	1.Admin views product listings. 2. Admin edits, approves, or removes product listings.
Exit Conditions	Product listings are successfully managed

Table 37Use Case View Reports

Use Case ID	UC019
Use Case Name	View Reports
Participation Actors	Admin
Entry Condition	Admin is logged in

Flow of Events	1.Adminselects reports to view. 2. System displays reports of users, orders, or products.
Alternate Flow	2.1. Report fails to load, system prompts refresh. 2.2. No data in report, system displays “No data available.”
Exit Conditions	Reports are successfully accessed

Table 38 Use Case Monitor System Analytics

Use Case ID	UC020
Use Case Name	Monitor System Analytics
Participation Actors	Admin
Entry Condition	Admin is logged in
Flow of Events	1.Admin navigates to analytics dashboard. 2.System displays statistics for users, products, and orders
Exit Conditions	Analytics data is successfully monitored.

Table 39Use Case Publish Articles/ Guides

Use Case ID	UC021
Use Case Name	Publish Articles/ Guides
Participation Actors	Agricultural Expert
Entry Condition	Expertis logged in
Flow of Events	1.Expertcreates article or guide content. 2.System validates and publishes content.
Exit Conditions	Articles or guides are successfully published.

Table 40 Use Case Respond to Farmer Queries

Use Case ID	UC022
Use Case Name	Respond to Farmer Queries
Participation Actors	Agricultural Expert
Entry Condition	Expertis logged in and queries exist
Flow of Events	1.Expertviews farmer queries.

	2.Expert provides recommendations. 3.System delivers responses to farmers.
Alternate Flow	3.1. Message delivery fails, system notifies expert. 1.1. No queries, system displays empty list.
Exit Conditions	Farmer queries are successfully answered.

Table 41 Use Case Update Market Alerts

Use Case ID	UC023
Use Case Name	Update Market Alerts
Participation Actors	Agricultural Expert
Entry Condition	Expertis logged in
Flow of Events	1.Expert updates market alert information. 2.System validates and posts alerts.
Exit Conditions	Market alerts are successfully updated.

2.6.3. Object Model

2.6.3.1. Data Dictionary

A data dictionary is a file that holds meta-data about the objects in a system. It includes the name type and description of each class. The Data Dictionary of the class diagram is described for each class in a tabular form.

Table 42User (General User Information)

Attribute Name	Description	Data Type	Constraints
User Id	Unique identifier for each user	INT	Primary Key, Auto Increment
Full Name	User's full legal name	VARCHAR (100)	Not Null
Email	User's email address	VARCHAR (100)	Unique, Not Null
Phone Number	User's phone number	VARCHAR (20)	Not Null
PasswordHash	Encrypted password	VARCHAR (255)	Not Null
User Role	Defines user type (Farmer, Buyer, Admin, Expert)	ENUM	Not Null
Profile Image	Profile picture	VARCHAR (255)	Optional

	link/path		
Address	User location/address	VARCHAR (255)	Optional
Registration Date	Date user created account	DATETIME	Default: Current Time
Account Status	Active, Suspended, Pending Verification	ENUM	Default: Active

Table 43 Farmer/Seller Profile Table

Attribute Name	Description	Data Type	Constraints
Farmer ID	Links to User Id	INT	Primary Key, Foreign Key(userId)
Farmer Name	Name of the farmer/business	VARCHAR (150)	Optional
Farmer Type	Type of farming (crop, livestock, etc.)	VARCHAR (50)	Optional
Verification Status	Indicates admin verification	ENUM	Pending/Verified/Rejected
Bio	Short description of the farmer	TEXT	Optional

Table 44 Buyer Profile

Attribute Name	Description	Data Type	Constraints
Buyer ID	Links to User Id	INT	Primary Key, Foreign Key
Preferred Categories	Buyer's interest categories	VARCHAR (255)	Optional

Table 45 Agricultural Expert

Attribute Name	Description	Data Type	Constraints
Expert ID	Links to User Id	INT	Primary Key, Foreign Key
Qualification	Expert qualifications	VARCHAR (255)	Not Null
Specialization	Expert area of expertise	VARCHAR (100)	Optional

Experience Years	Years of experience	INT	Optional

Table 46 Product

Attribute Name	Description	Data Type	Constraints
Product Id	Unique product identifier	INT	Primary Key
Farmer Id	Owner of the product	INT	Foreign Key
Product Name	Name of the product	VARCHAR (100)	Not Null
Category	Category (e.g. vegetables, fruits)	VARCHAR (50)	Not Null
Description	Product description	TEXT	Optional
Price	Price per unit	DECIMAL (10,2)	Not Null
Quantity Available	Stock available	INT	Not Null
ImagePath	Product image link	VARCHAR (255)	Optional
Date Posted	When product was posted	DATETIME	Default: Current Time
Status	Available/ out of Stock	ENUM	Default: Available

Table 47 Cart

Attribute Name	Description	Data Type	Constraints
Cart ID	Unique cart Id	INT	Primary Key
Buyer ID	Owner of the cart	INT	Foreign Key
Date Created	Date cart was created	DATETIME	Default Current Time

Table 48 Cart Items

Attribute Name	Description	Data Type	Constraints
Cart Item ID	Unique cart item Id	INT	Primary Key
Cart ID	Associated cart	INT	Foreign Key
Product ID	Product added to cart	INT	Foreign Key

Quantity	Quantity selected	INT	Not Null

Table 49 Orders

Attribute Name	Description	Data Type	Constraints
Order Id	Unique order identifier	INT	Primary Key
Buyer Id	Buyer placing the order	INT	Foreign Key
Farmer Id	Farmer selling the product	INT	Foreign Key
Total Amount	Total price of order	DECIMAL (10,2)	Not Null
Order Date	Date of order	DATETIME	Default Current Time
Payment Status	Pending/Paid / Failed	ENUM	Default Pending
Order Status	Active / Completed / Cancelled	ENUM	Default Active

Table 50 Order Items

Attribute Name	Description	Data Type	Constraints
Order Item ID	Unique identifier	INT	Primary Key
Order ID	Related order	INT	Foreign Key
Product ID	Product purchased	INT	Foreign Key
Quantity	Quantity of product	INT	Not Null
Price	Price per unit at purchase time	DECIMAL (10,2)	Not Null

Table 51 Chat Table

Attribute Name	Description	Data Type	Constraints
Chat ID	Unique identifier	INT	Primary Key
Sender ID	Sender of message	INT	Foreign Key
Receiver ID	Receiver of message	INT	Foreign Key
Message	Chat message content	TEXT	Not Null
Timestamp	Time of message	DATETIME	Default current time

Table 52 Reports

Attribute Name	Description	Data Type	Constraints
Report Id	Unique report identifier	INT	Primary Key
Reporter Id	User who submits the report	INT	Foreign Key
Reported User Id	User being reported	INT	Foreign Key
Report Reason	Reason for report	TEXT	Not Null
Report Date	Date submitted	DATETIME	Default Current Time
Admin Response	Admin action summary	TEXT	Optional
Status	Pending / Reviewed / Resolved	ENUM	Default Pending

Table 53 Market Alerts

Attribute Name	Description	Data Type	Constraints
Alert ID	Unique alert identifier	INT	Primary Key
Expert ID	Expert issuing the alert	INT	Foreign Key
Title	Alert title	VARCHAR (150)	Not Null
Description	Alert details	TEXT	Not Null
Alert Date	Date issued	DATETIME	Default current time

Table 54 Articles/Guides

Attribute Name	Description	Data Type	Constraints
Article Id	Unique article identifier	INT	Primary Key
Expert Id	Author Expert	INT	Foreign Key
Title	Article title	VARCHAR (150)	Not Null
Content	Content of the article	TEXT	Not Null
Category	Article Category	VARCHAR (50)	Optional
Publish Date	Publishing Date	DATETIME	Default Current Time

Table 55 Payment

Attribute Name	Description	Data Type	Constraints
Payment Id	Unique payment identifier	INT	Primary Key, Auto Increment
Order Id	Related order identifier	INT	Foreign Key
Amount	Amount paid	DECIMAL (10,2)	Not Null
Payment Method	Method of payment (Mobile Money, Cash, etc.)	ENUM	Not Null
Payment Status	Status of payment	ENUM	Pending / Paid / Failed
Payment Date	Date and time of payment	DATETIME	Default: Current Time

Table 56 System Log

Attribute Name	Description	Data Type	Constraints
Log Id	Unique log identifier	INT	Primary Key
User Id	User performing the action	INT	Foreign Key
Action	Description of action performed	VARCHAR (255)	Not Null
Timestamp	Date and time of action	DATETIME	Default: Current Time

2.6.3.2 Class Modeling

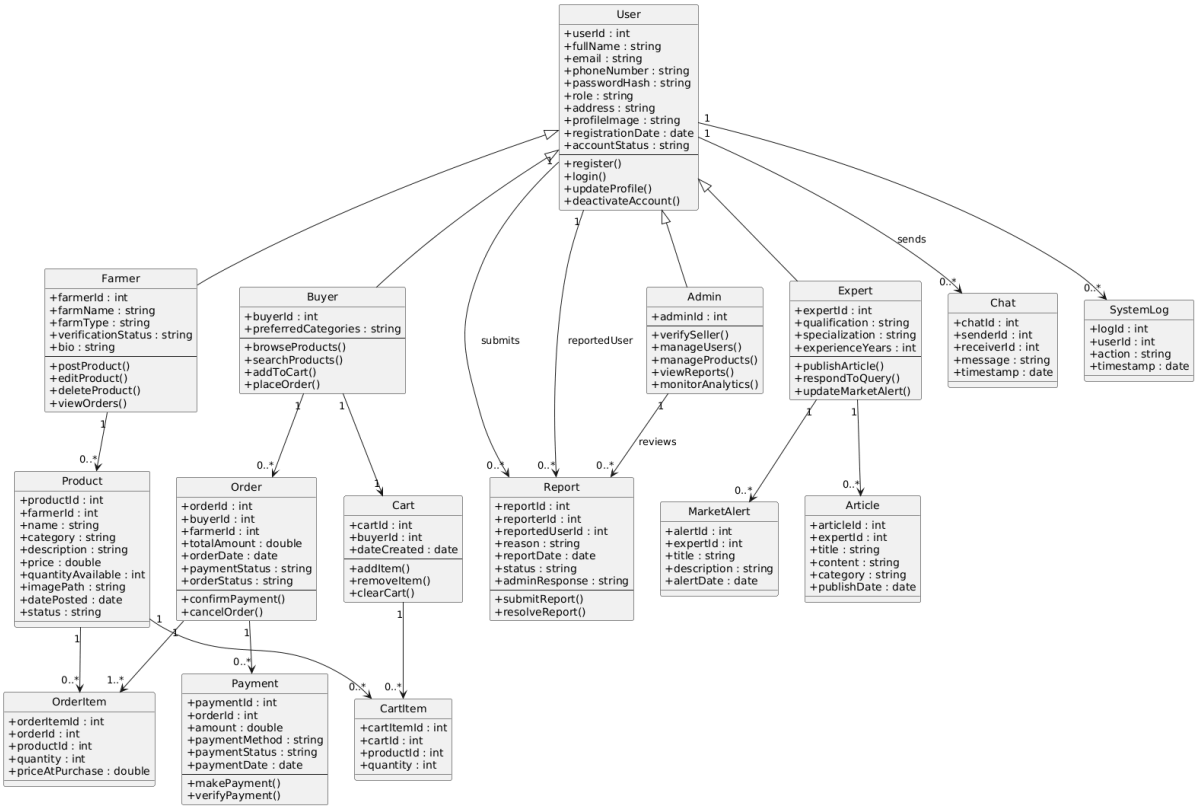


Figure 4 Class Diagram

2.6.4. Dynamic Modeling

The dynamic model is used to express and model the system's behavior over time. Sequence Diagrams and Activity Diagrams are used to represent it. Activity Diagrams depict the system's internal behavior, whereas Sequence Diagrams describe behavior as a sequence of messages exchanged among a set of objects

2.6.4.1 Sequence Diagram

A sequence diagram is a type of interaction diagram that shows how and in what order objects function in a system. A sequence diagram includes class roles or participants, messages, lifelines, activation or execution occurrences etc. It is used to show the interactions between objects and the sequential order in which they occur by the flow of messages from one object to another. It is useful for requirement analysis.

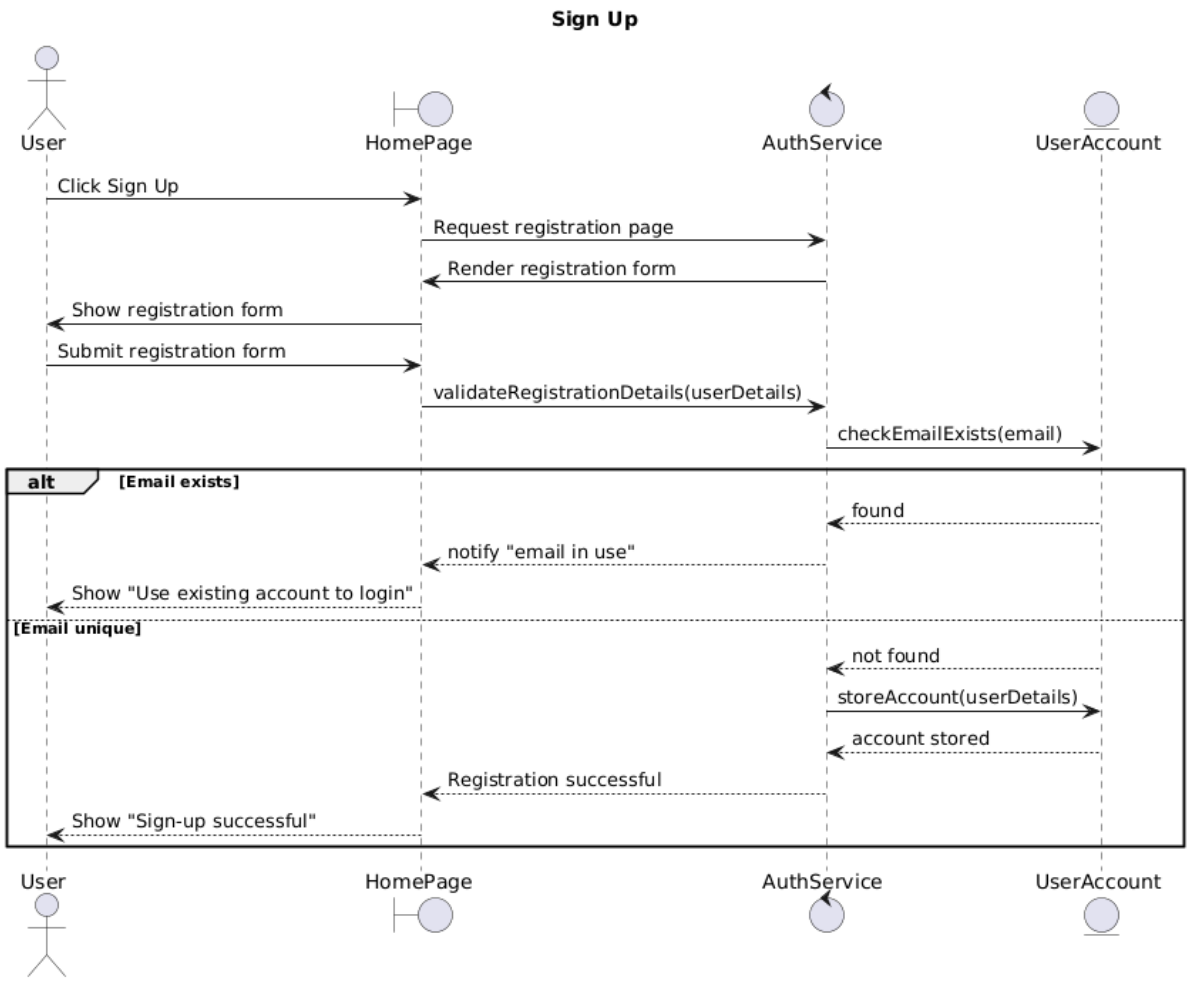


*Figure 5*Signup sequence diagram

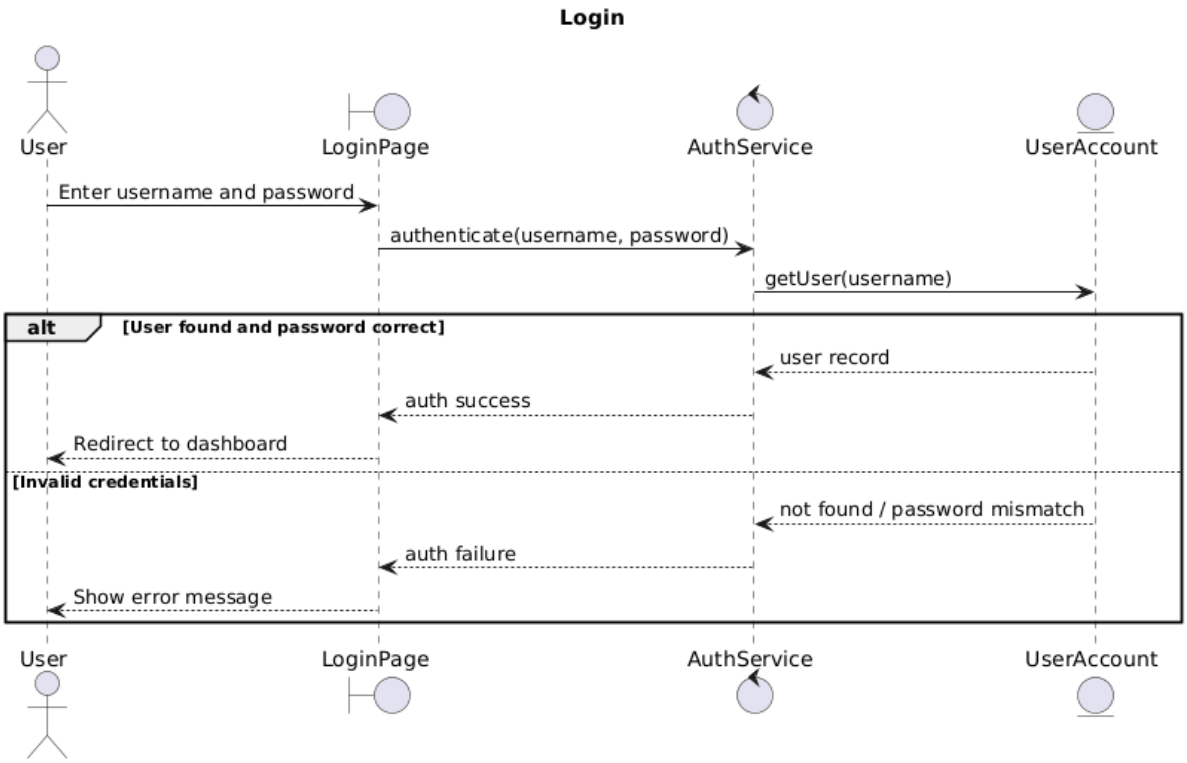


*Figure 6*Login sequence diagram

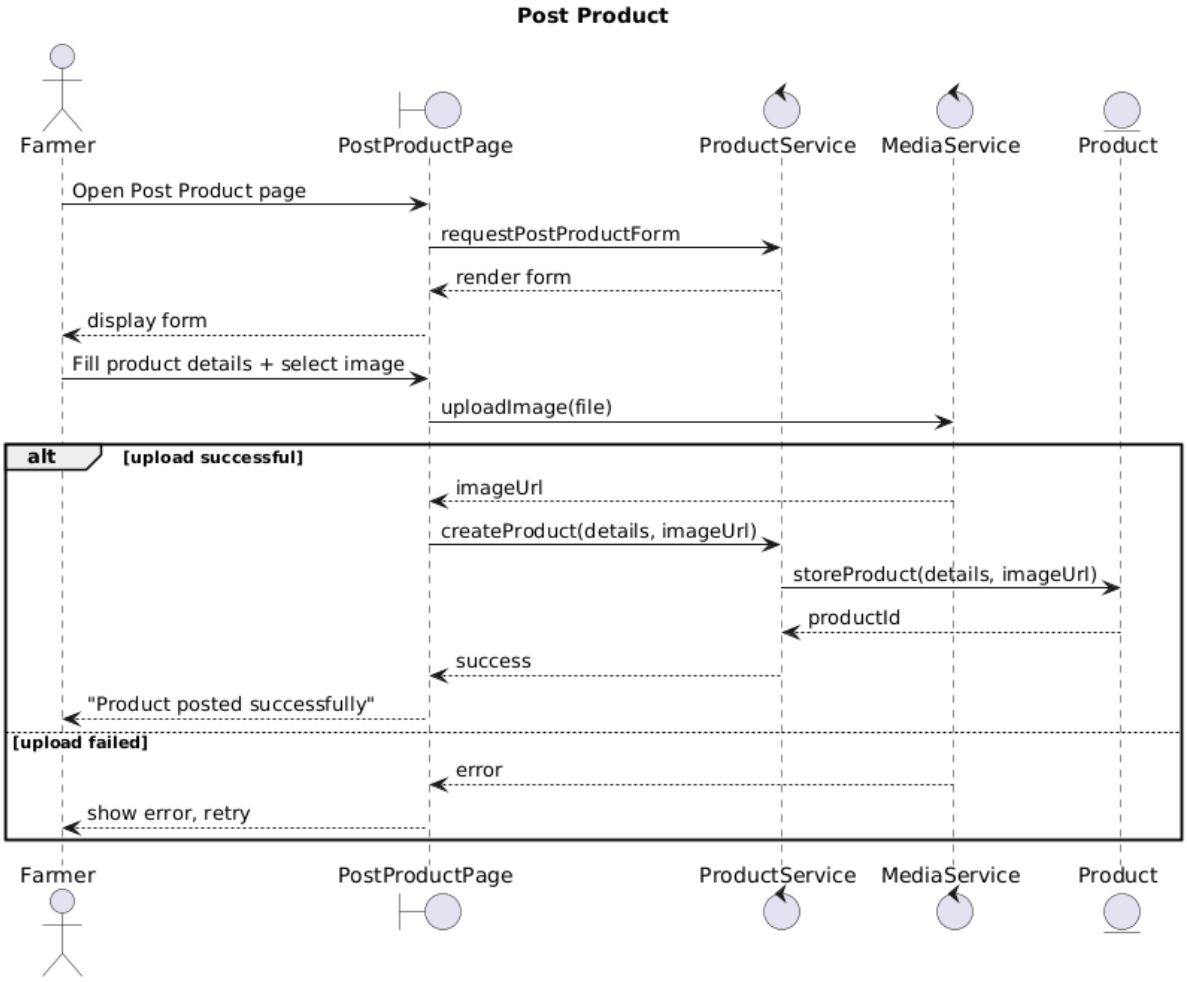


Figure 7Post Product sequence diagram

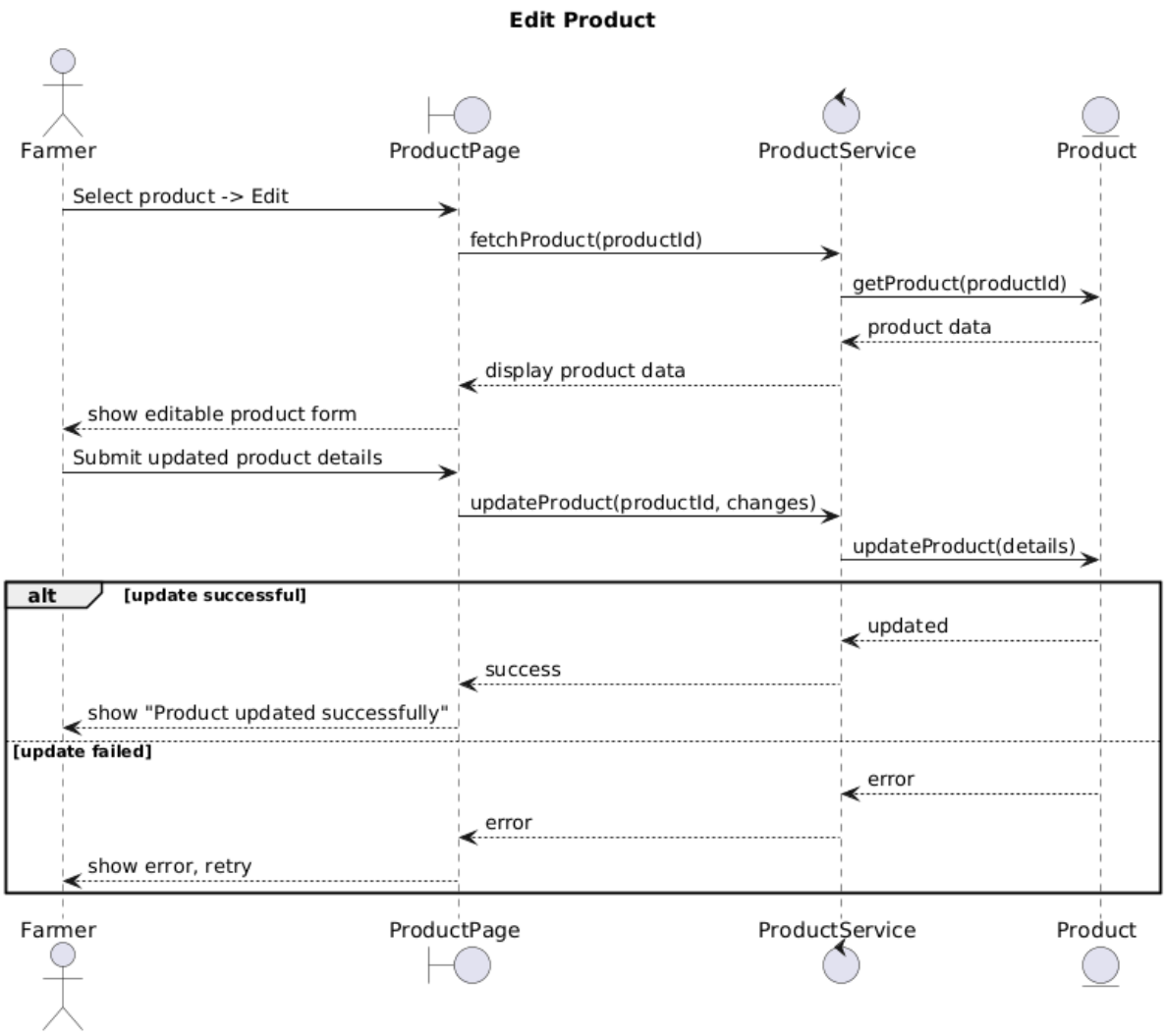


*Figure 8*Edit Product sequence diagram

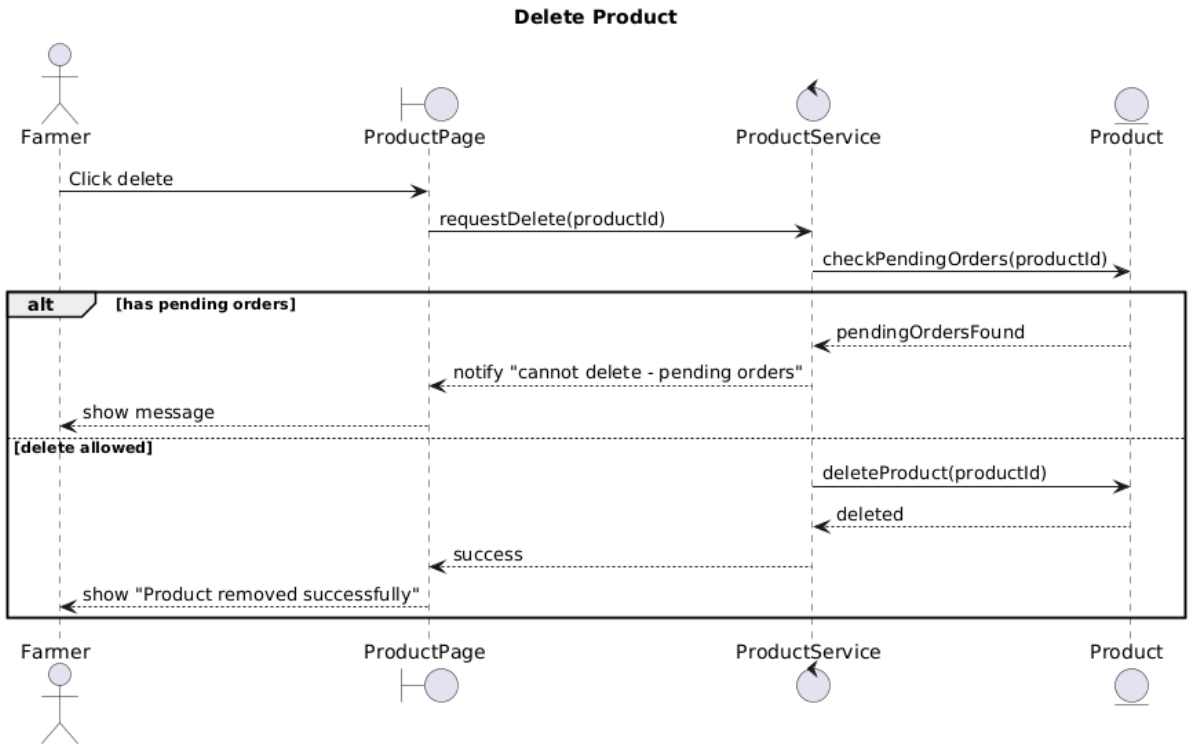


Figure 9 Delete product sequence diagram

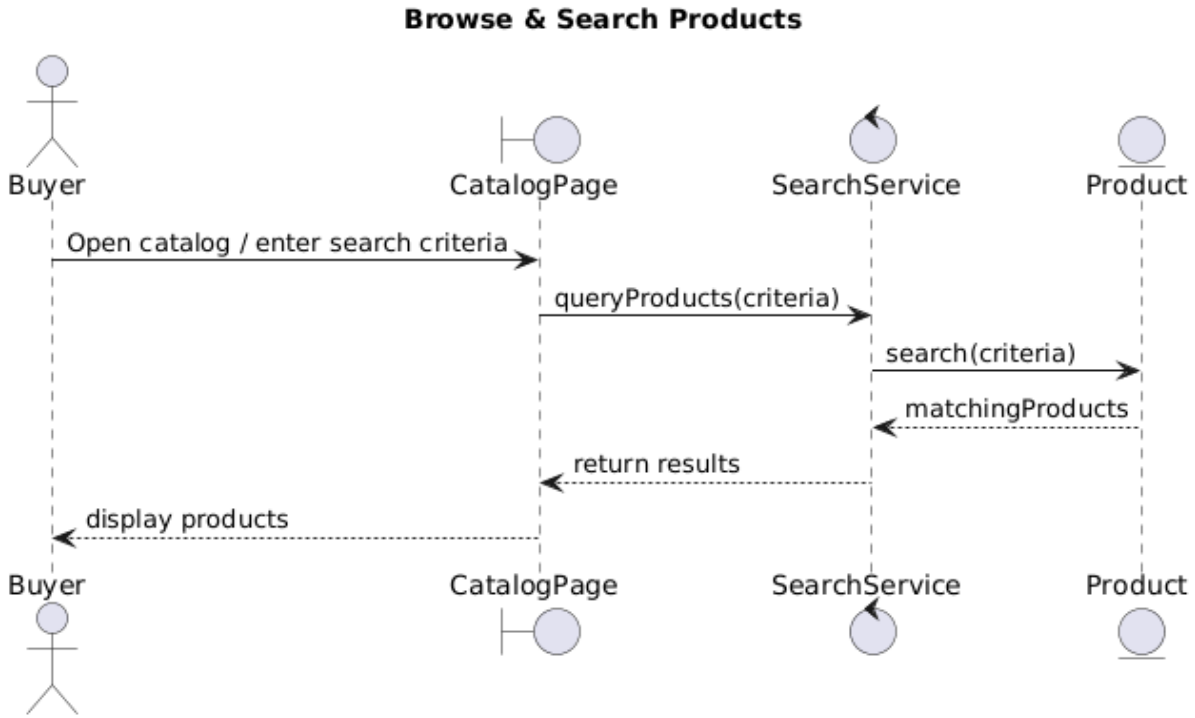


Figure 10 Search product sequence diagram

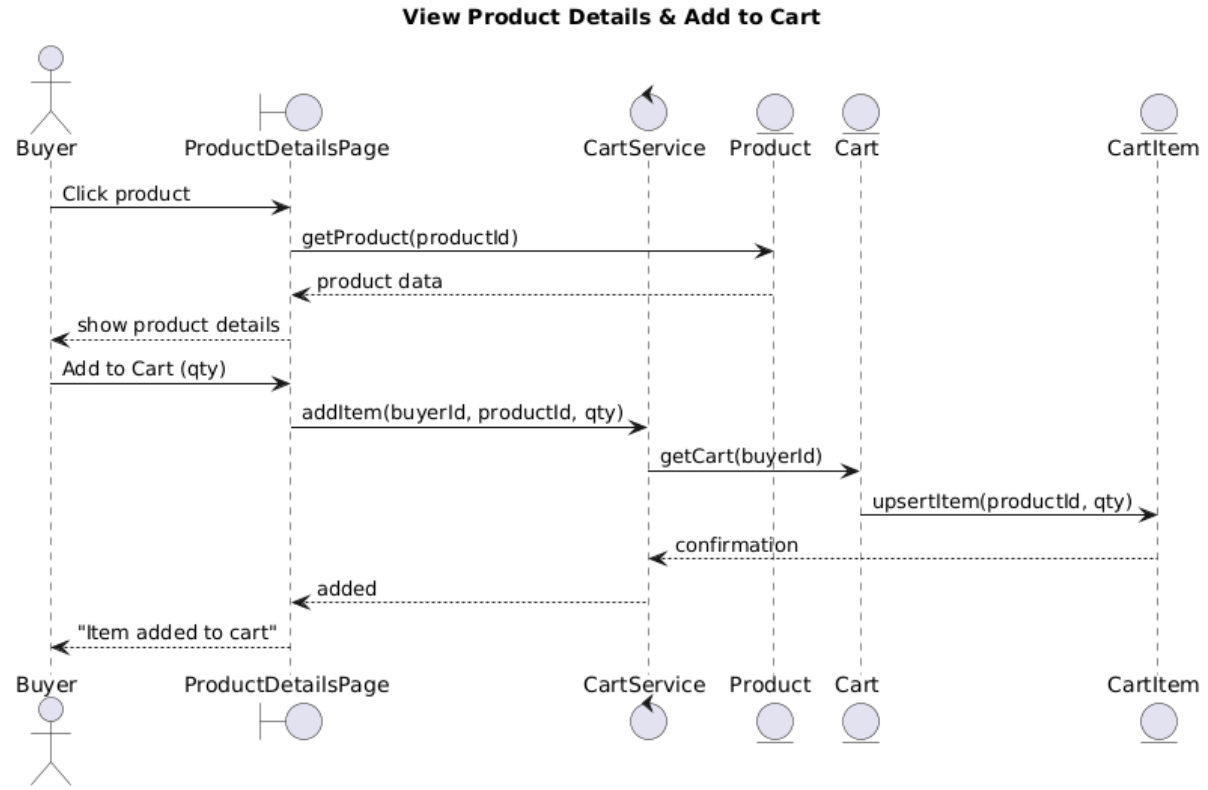


Figure 11 View details and add to cart sequence diagram

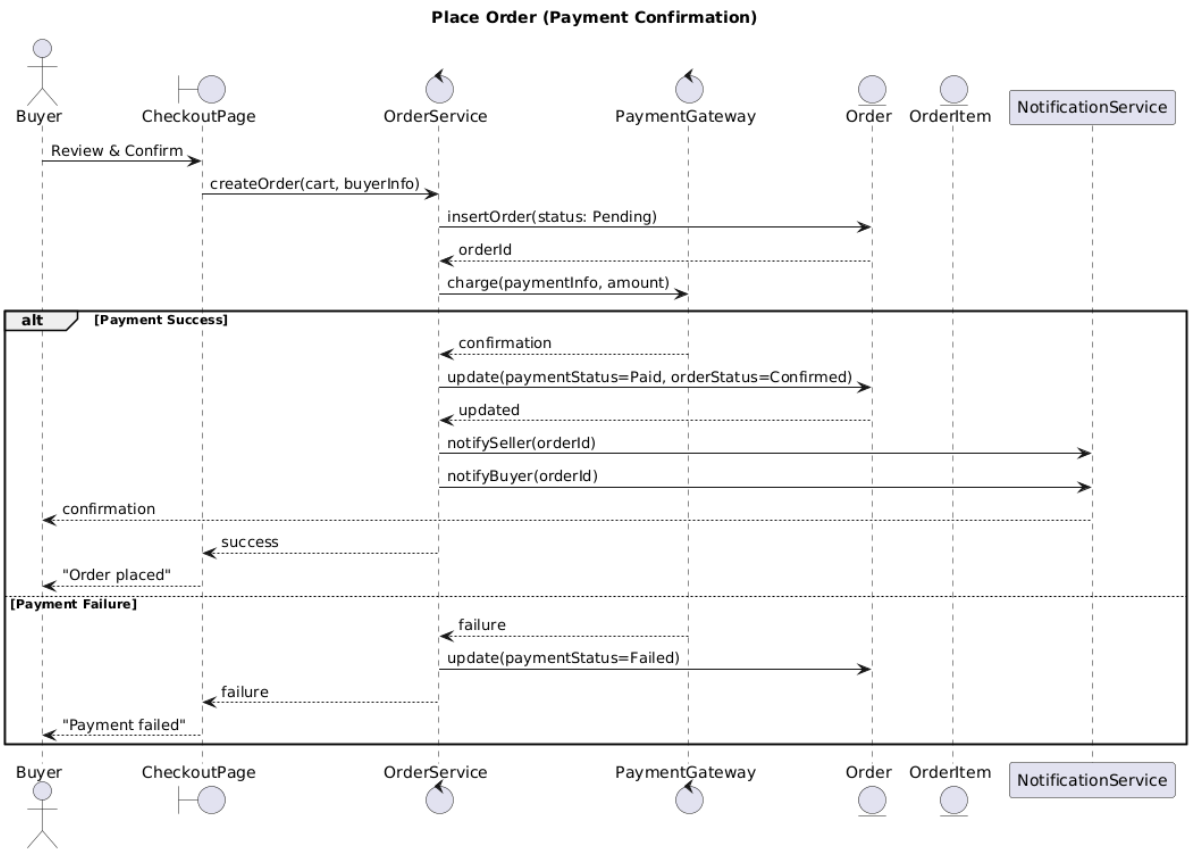


Figure 12 Place order sequence diagram

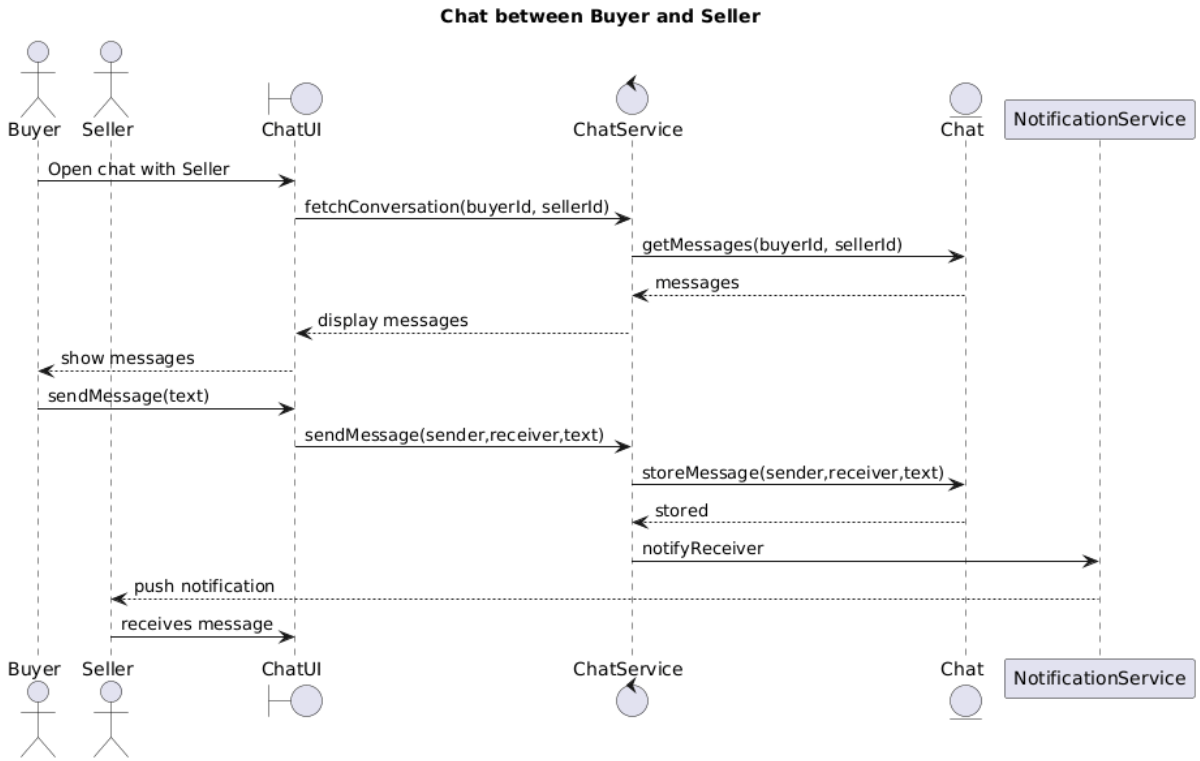


Figure 13 Chat sequence diagram

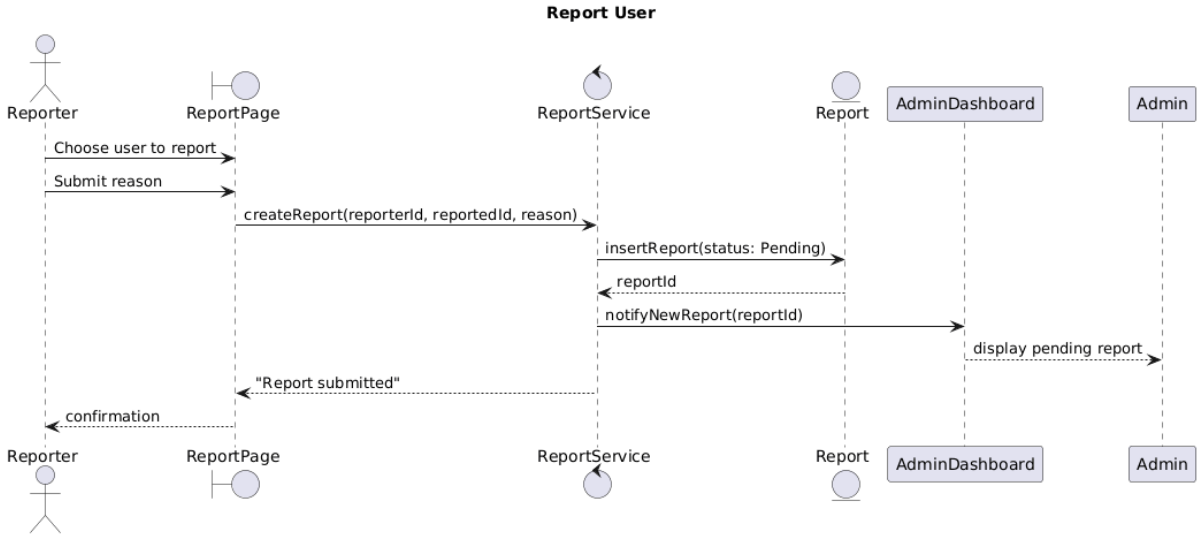


Figure 14 Report user sequence diagram

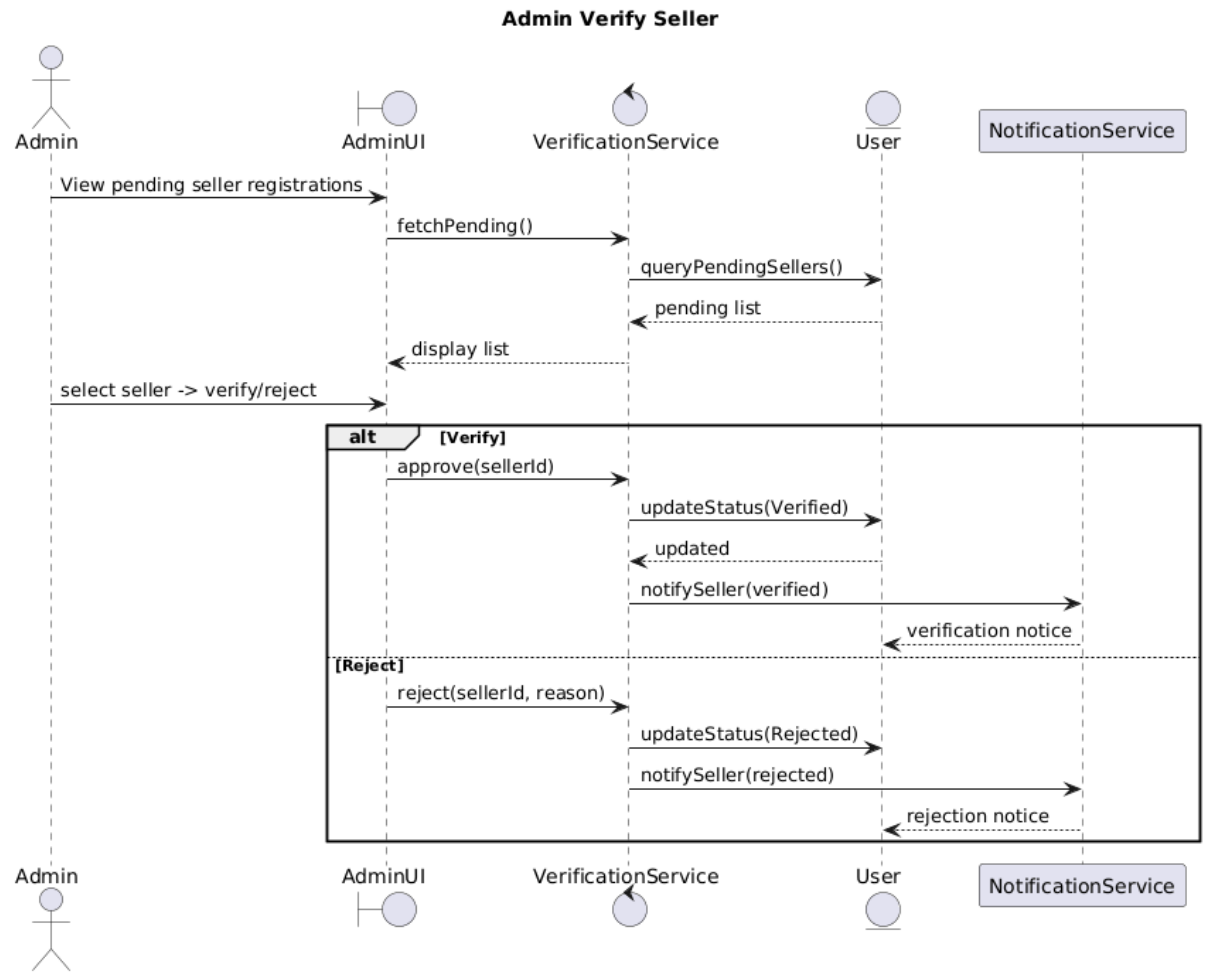


Figure 15 Verify Seller sequence diagram

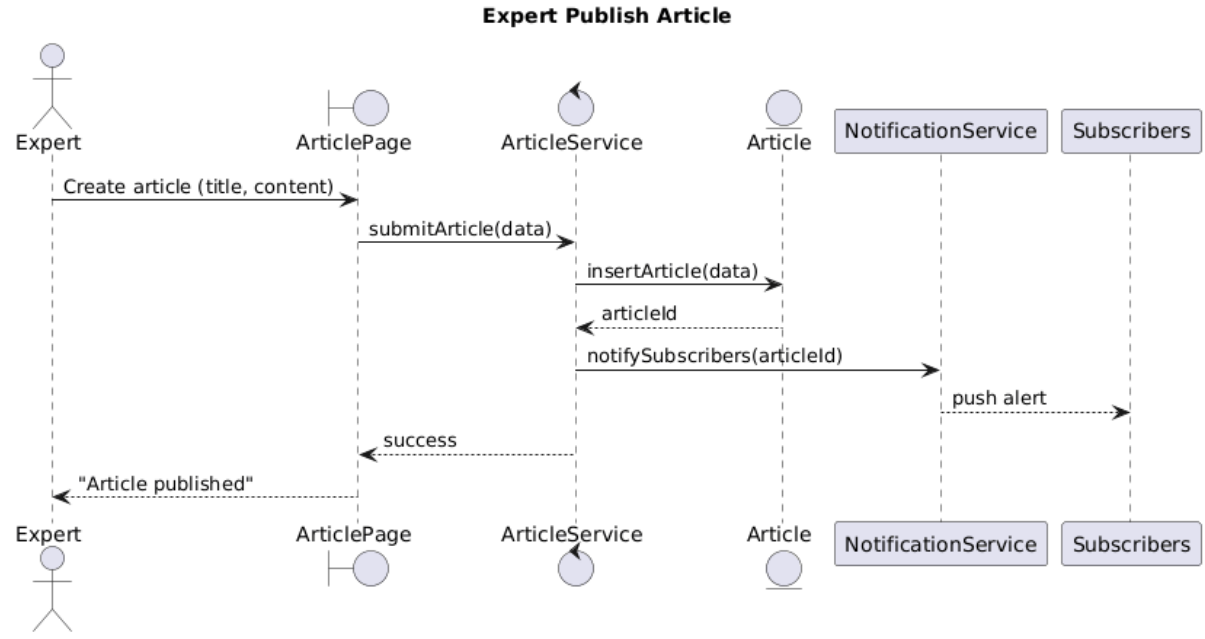


Figure 16 Expert publish guide sequencediagram

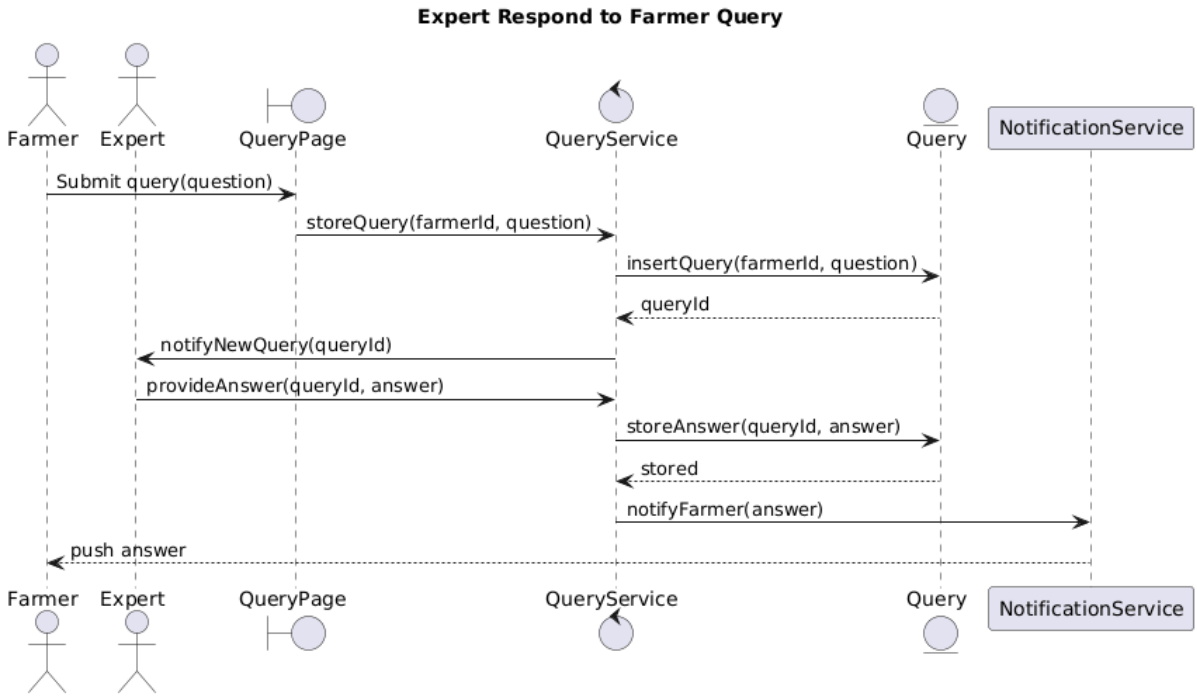


Figure 17 Expert respond to Farmer's query sequence diagram

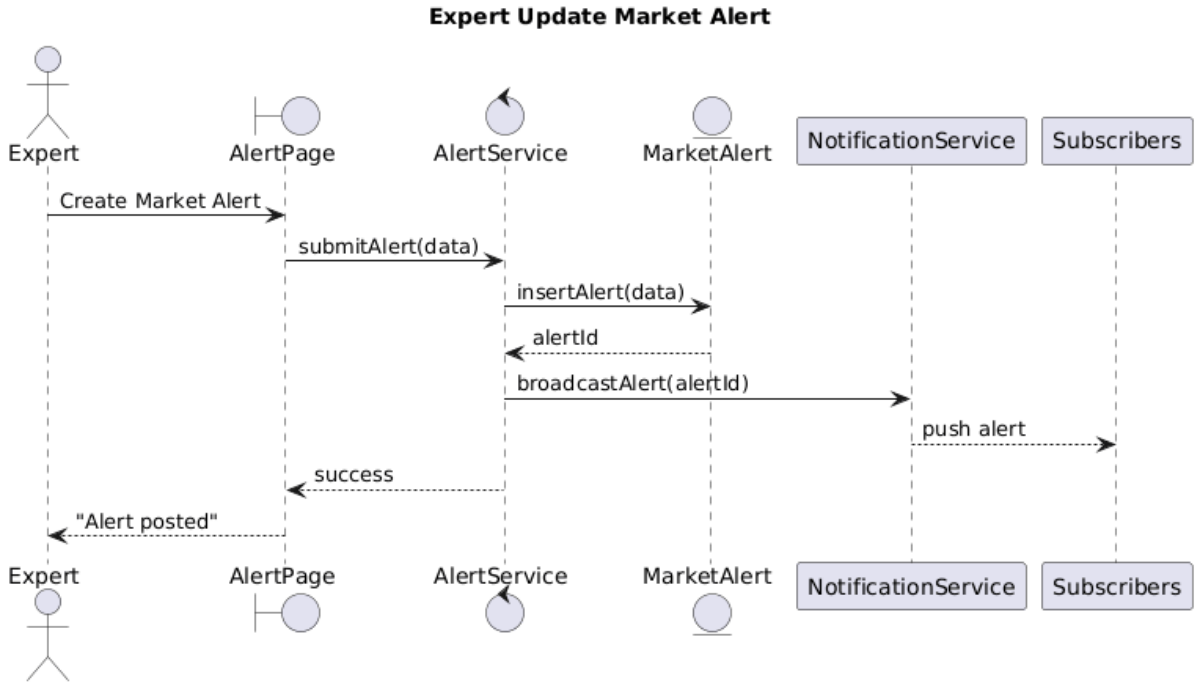


Figure 18 Expert update Market Alert sequence diagram

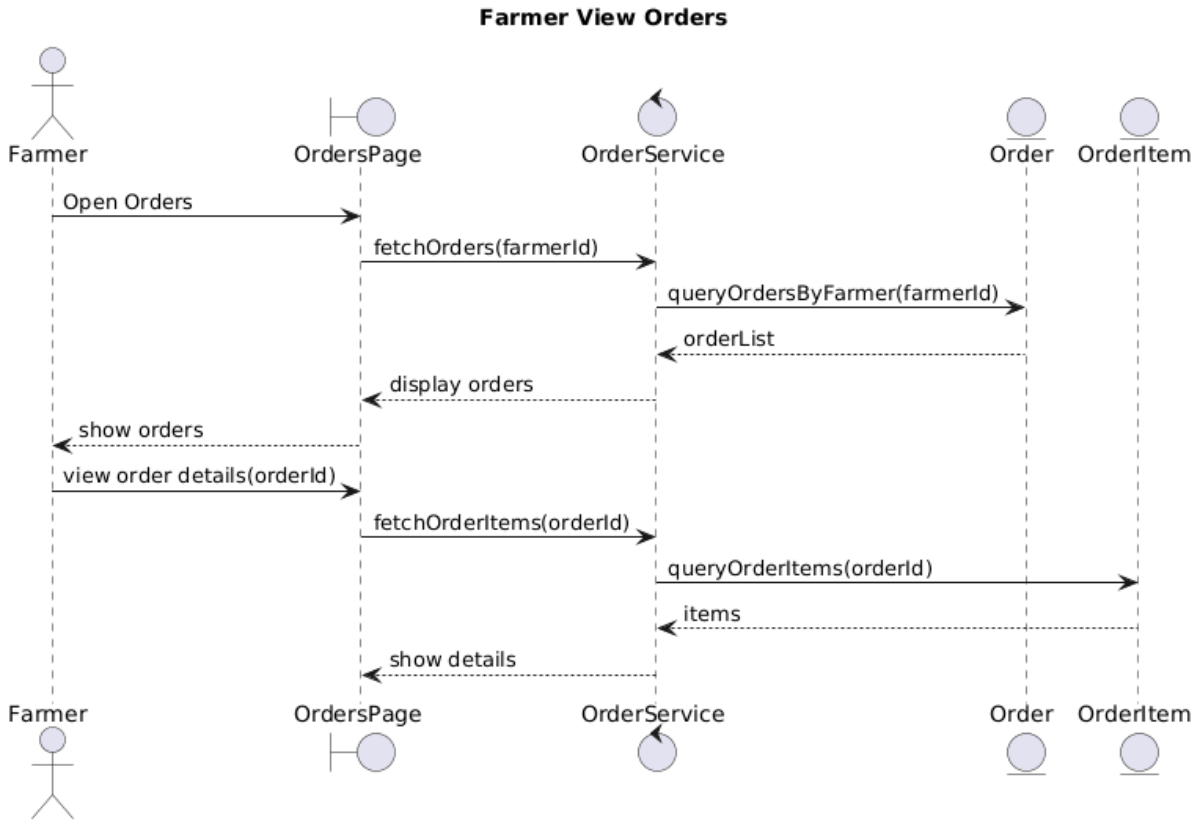


Figure 19 Farmers View Orders sequence diagram

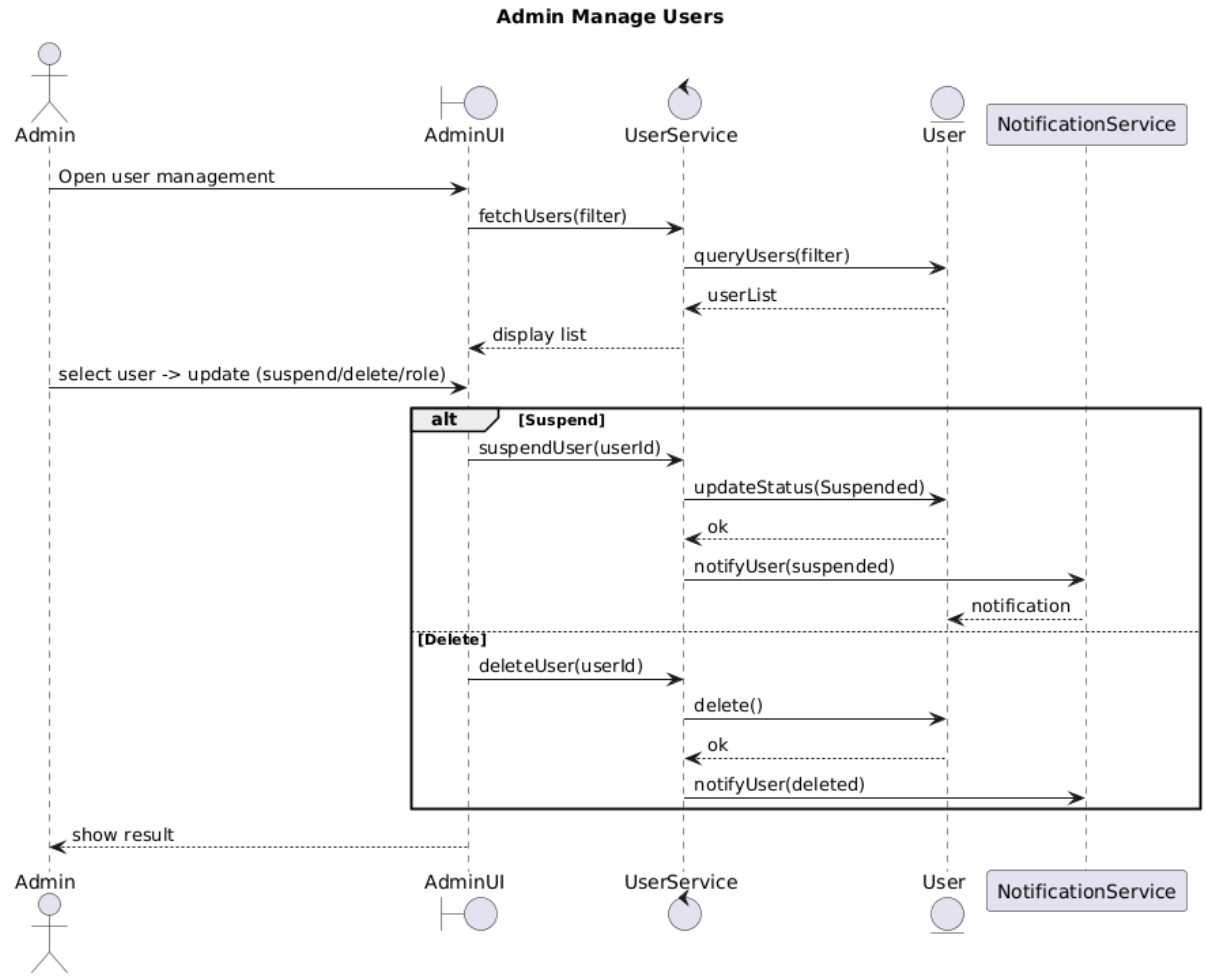


Figure 20 Admin Manage Users sequence diagram

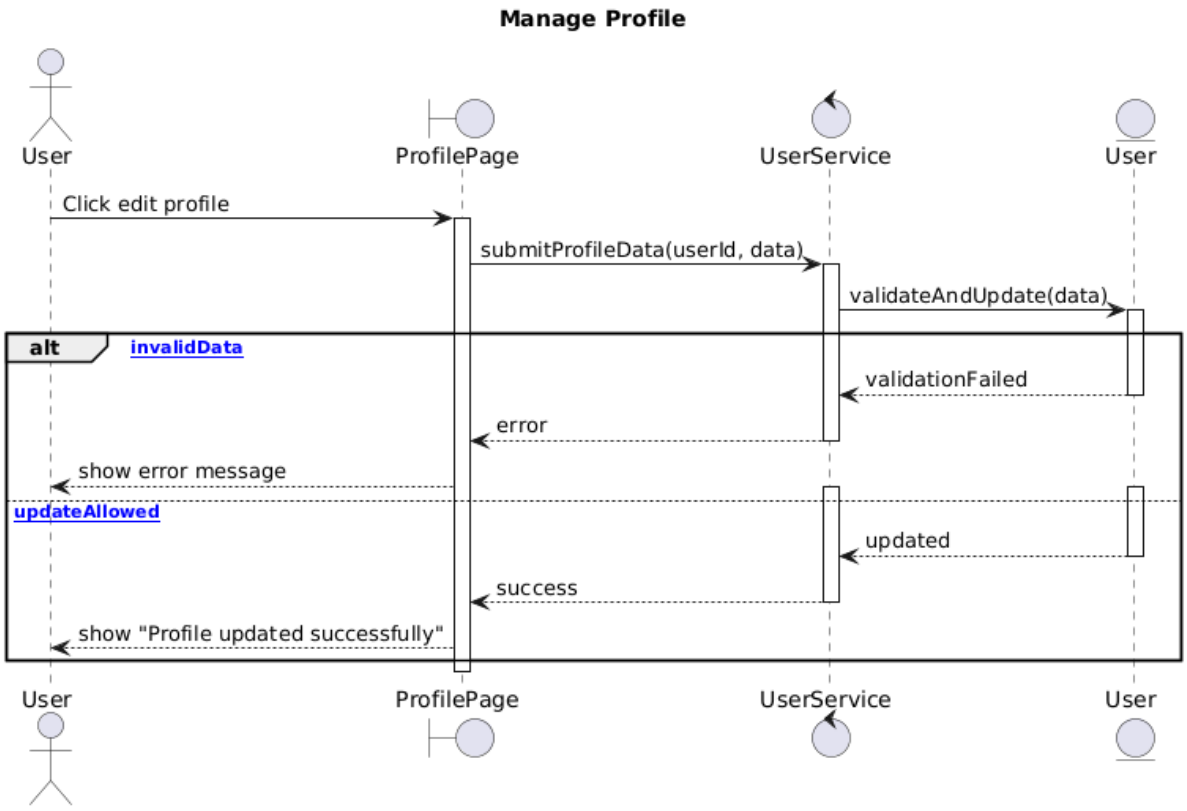


Figure 21 Manage Profile sequence diagram

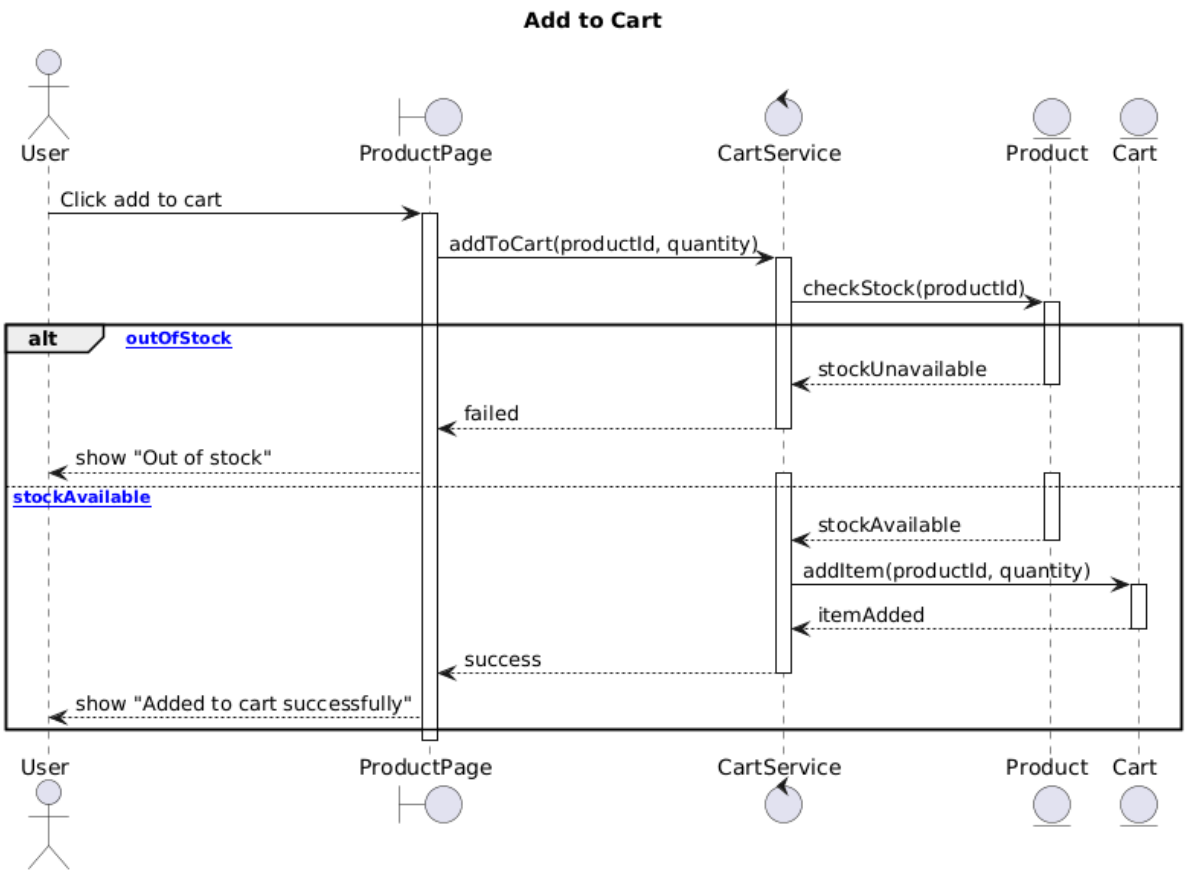


Figure 22 Add to Cart sequence diagram

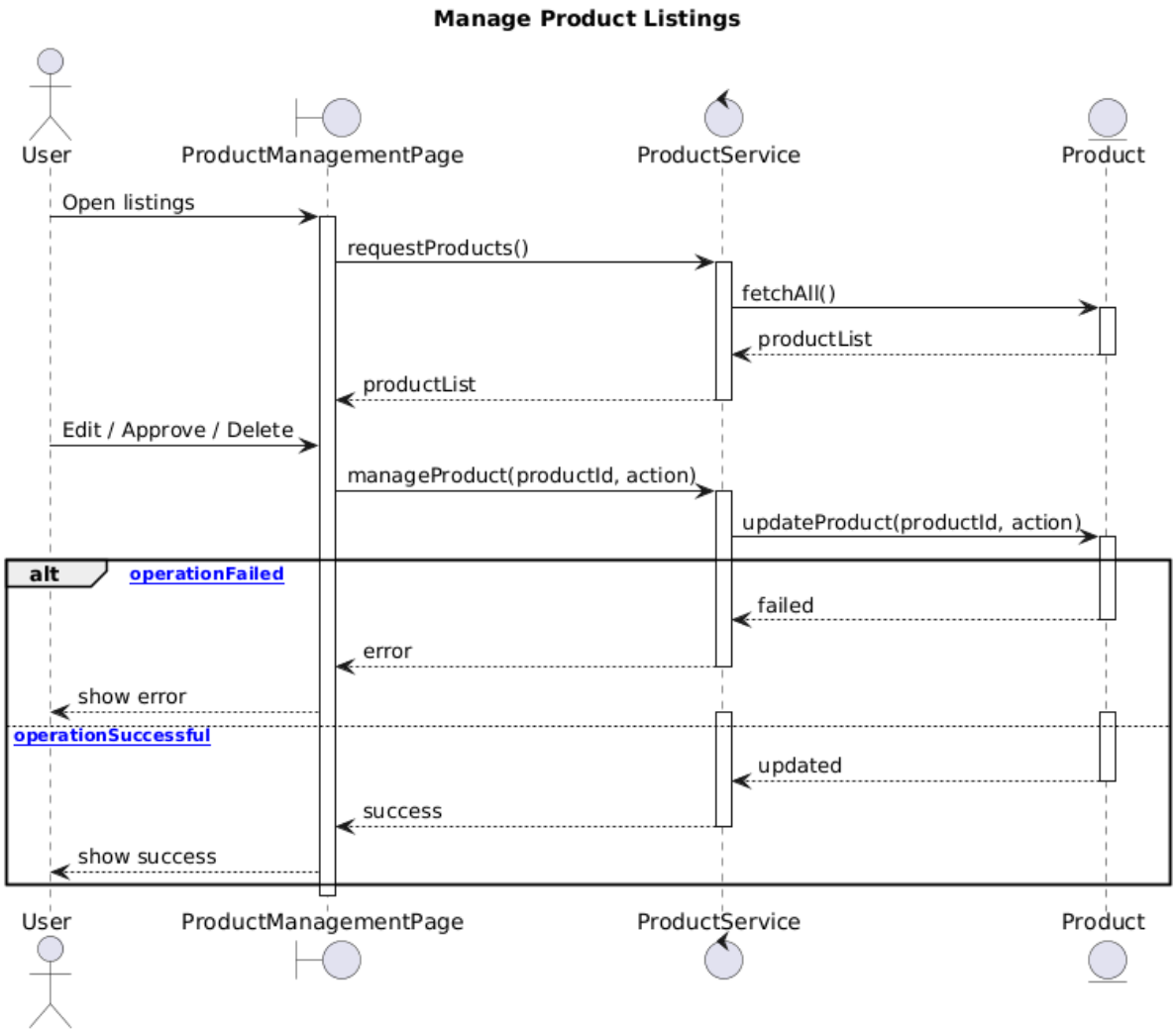


Figure 23 Manage Product Listing sequence diagram

2.6.4.2. State Diagram

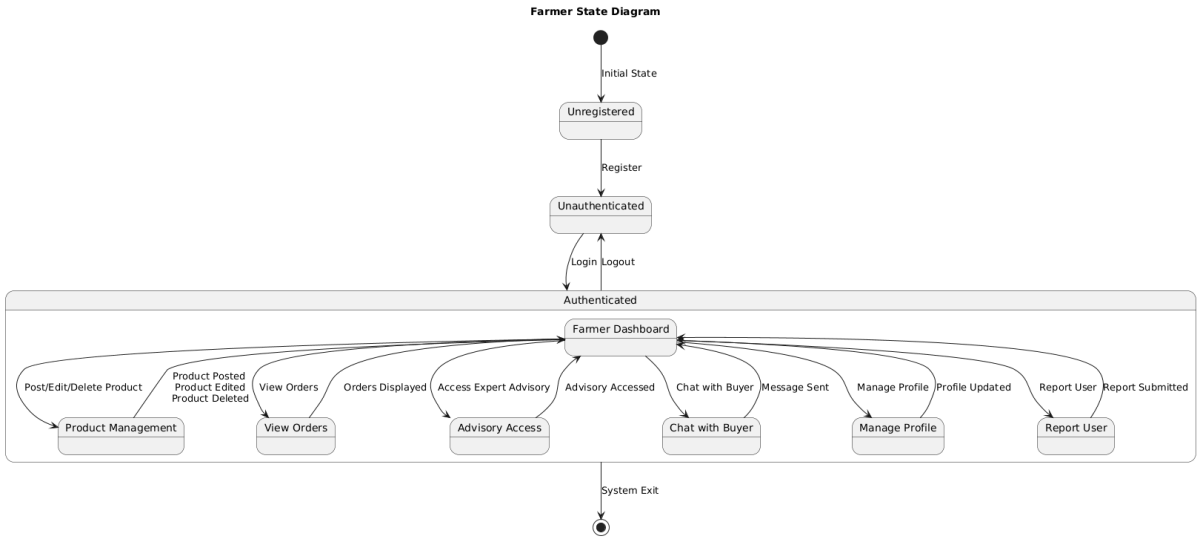


*Figure 24*State Diagram Farmer State

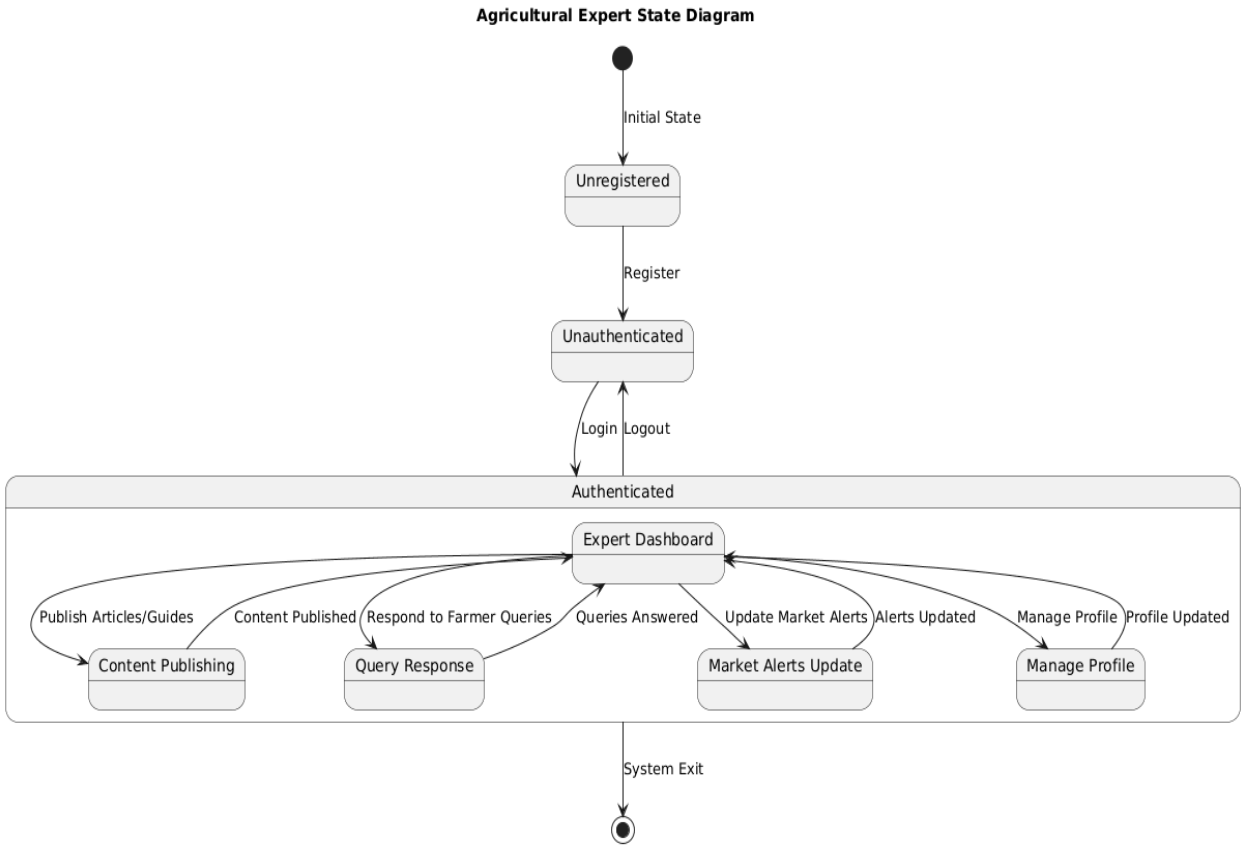


Figure 25 State Diagram ExpertState

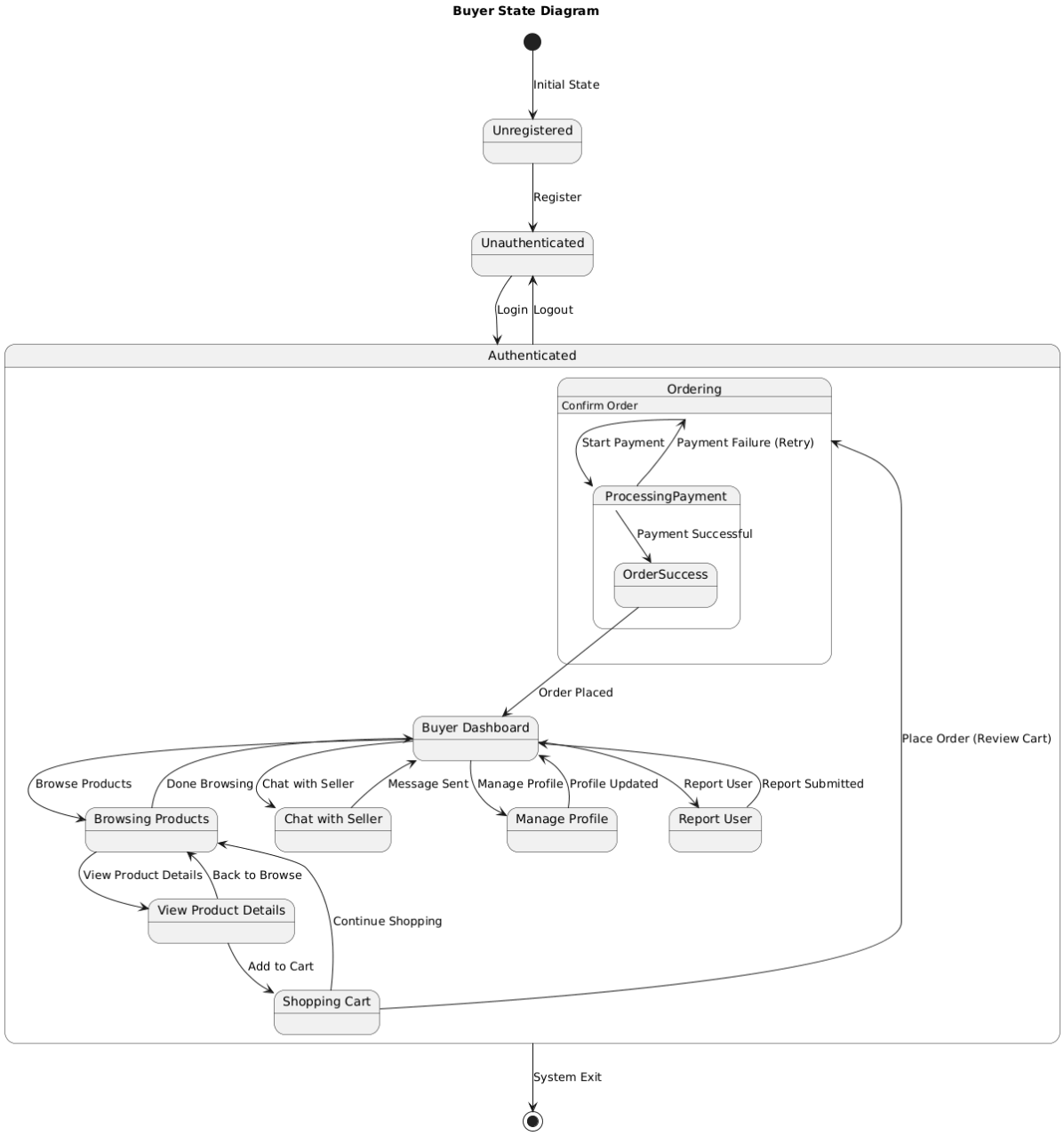


*Figure 26*State Diagram BuyerState

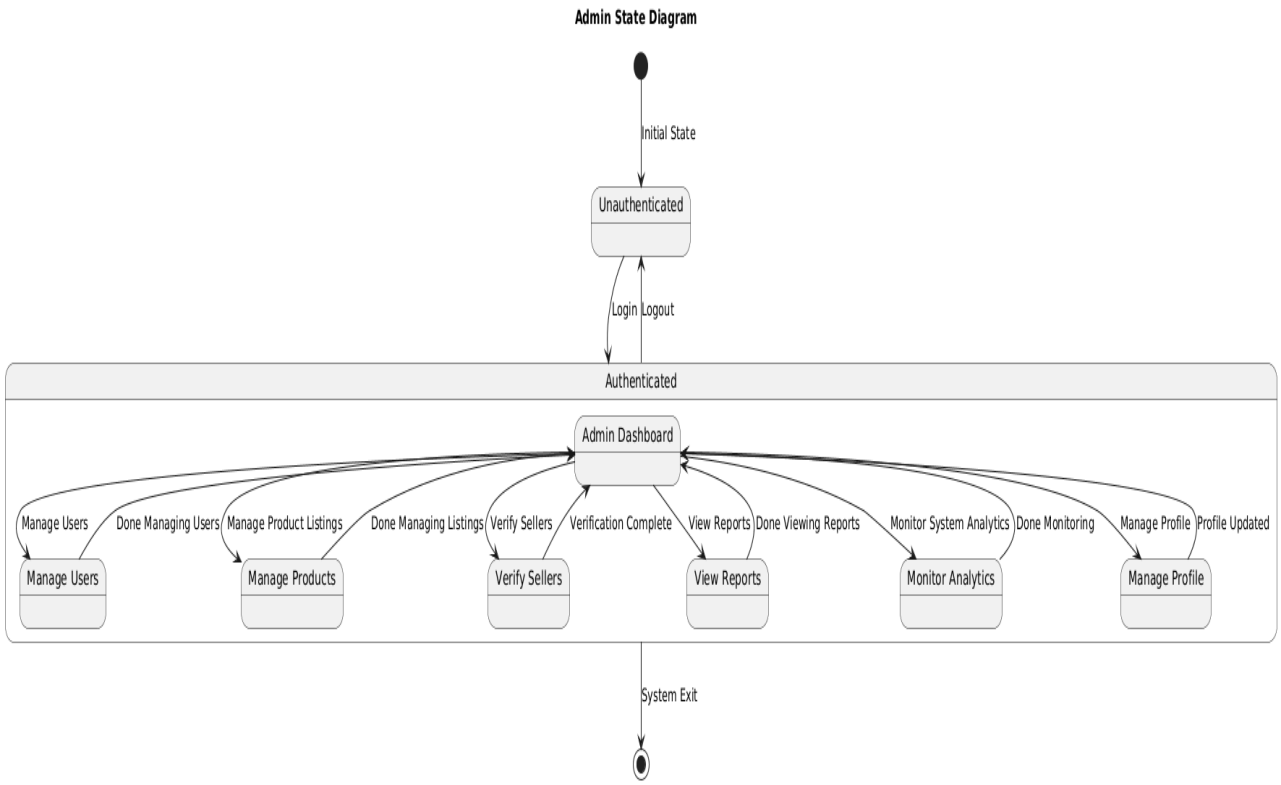


Figure 27 State Diagram AdminState

2.6.4.3. Activity Diagram

Activity Diagrams are used to explain the system's internal behavior. Similar to a flowchart or data flow diagram, an activity diagram visualizes a sequence of events or the direction of control inside a system.

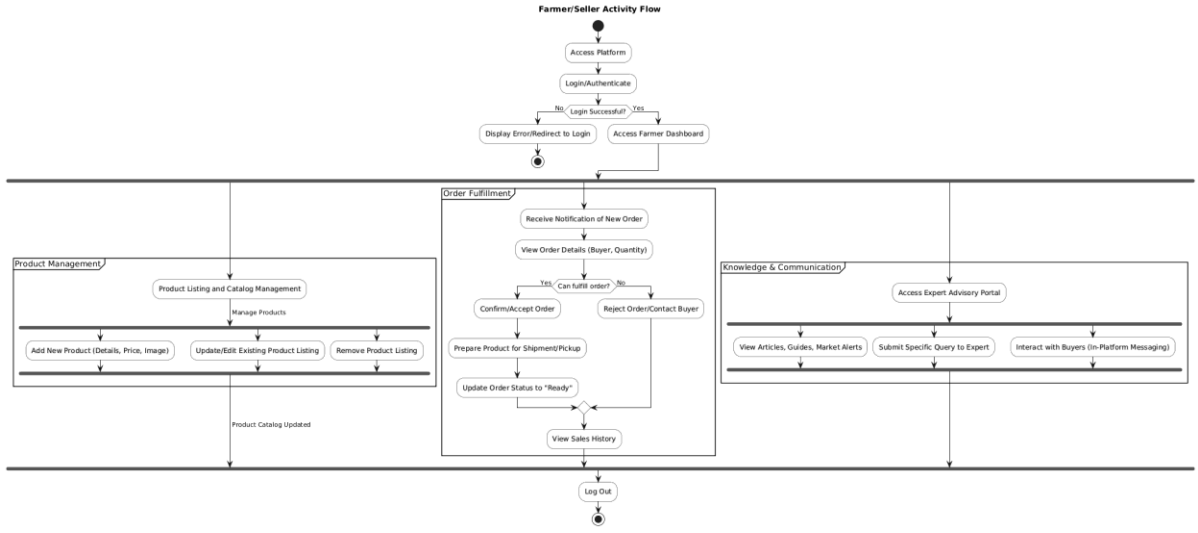


Figure 28 Activity Diagram Farmer Activity

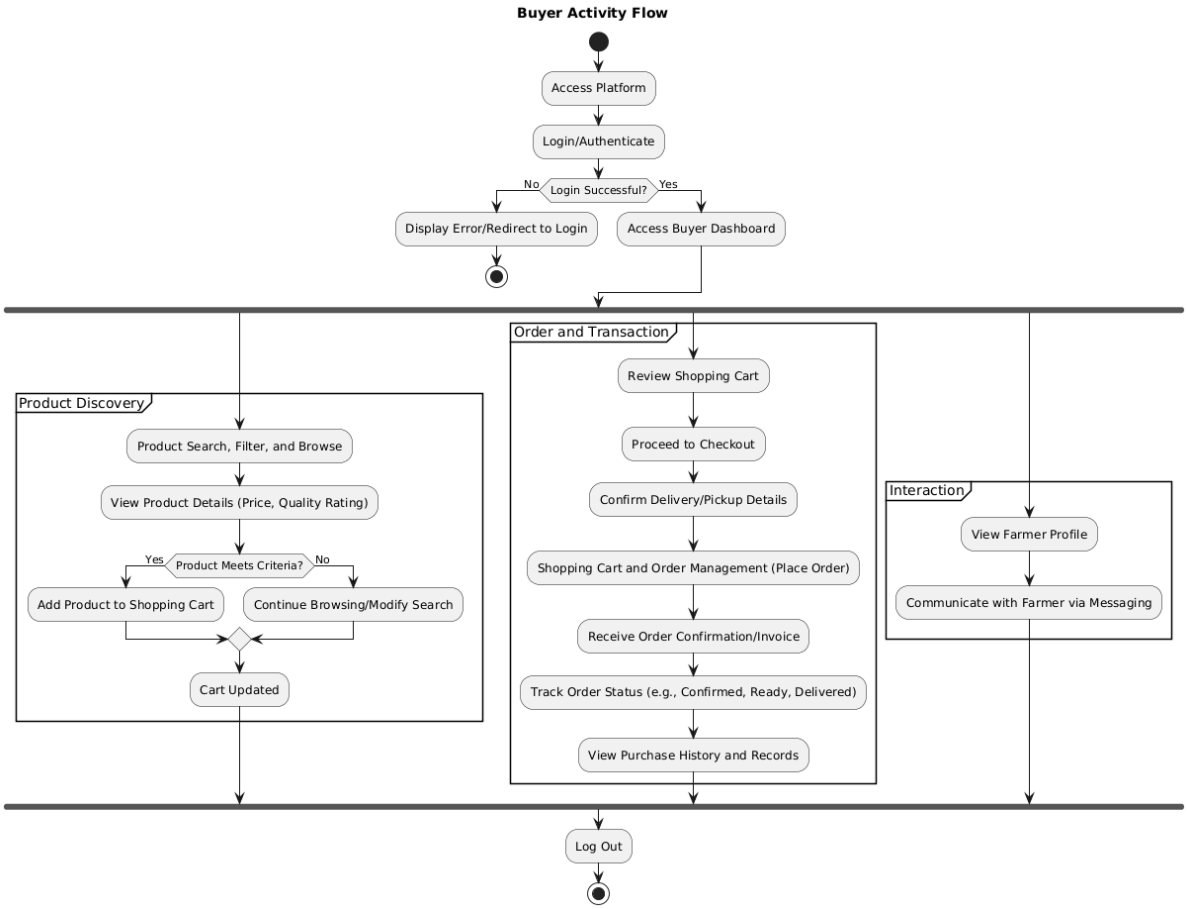


Figure 29 Activity Diagram Buyer Activity

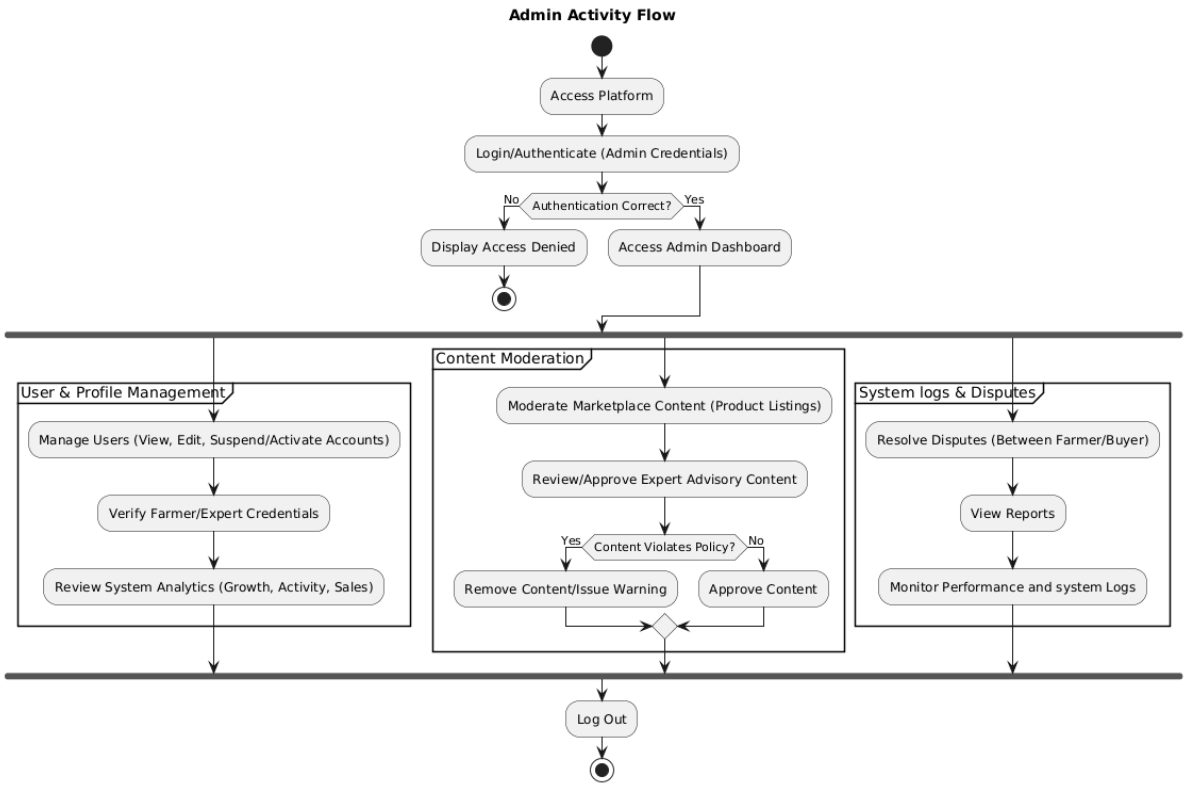


Figure 30 Activity Diagram Admin Activity

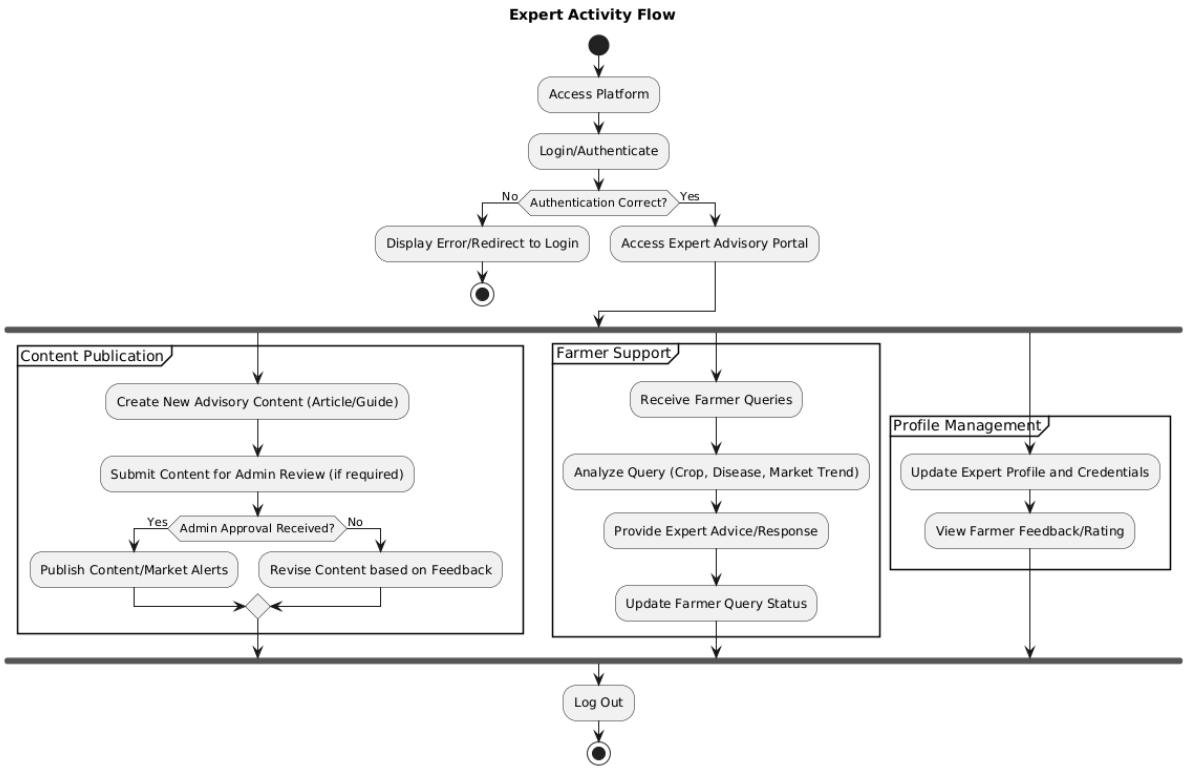


Figure 31 Activity Diagram Expert Activity

2.6.5. User Interface

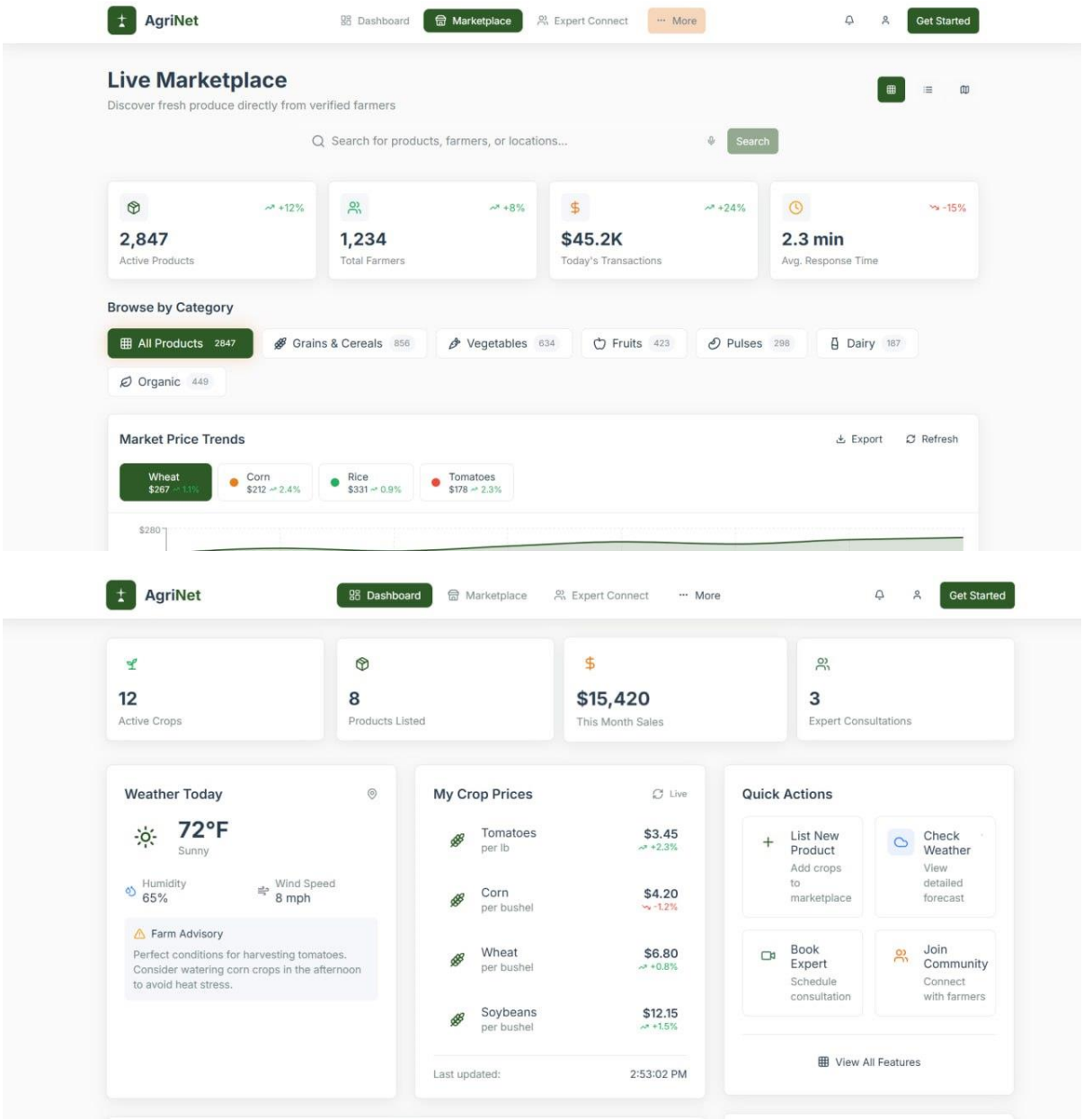


Figure 32 Dashboard interface diagram

3.CHAPTER THREE – SYSTEM DESIGN

3.1.Introduction

This chapter presents the system design that follows the requirement analysis and the initial modelling phase. It outlines the overall software architecture and highlights the key design objectives. These include the decomposition of the system into subsystems, the mapping of software components onto hardware resources, the management of persistent data, and the implementation of access control and security mechanisms.

Overview

System design refers to the process of specifying the essential elements of a systemincluding its components, modules, interfaces, and data structuresso that it can meet the defined requirements. It represents the transition from the analysis model to a detailed system design model that incorporates the non-functional requirements and constraints outlined in the problem statement and the requirements analysis document. Ultimately, system design provides a blueprint for how the new system will be constructed and implemented.

Purpose of the system design

The purpose of system design is to convert the requirements identified during the requirement analysis phase into a structured blueprint for building the actual system. During this stage, key decisions are made and detailed specifications are defined to guide subsequent phases such as implementation, testing, and maintenance. Overall, system design ensures that the system's architecture, components, and interactions are clearly outlined and aligned with the project's functional and non-functional requirements.

- **Transformation of Requirements**

Objective: Convert user and system requirements into a detailed, actionable system design.

Rationale: This process bridges the gap between high-level requirements and the final implementation, providing a clear roadmap that guides developers throughout system construction.

- **Architecture Definition**

Objective: Establish the overall structural framework and organization of the software system.

Rationale: A well-defined architecture organizes subsystems, components, and their interactions, forming a solid foundation for development and future expansion.

- **Subsystem Specification**

Objective: Describe the functionalities, responsibilities, and characteristics of each subsystem and component.

Rationale: Clear subsystem specifications ensure that developers can efficiently implement, integrate, and test individual components of the system.

- **Data Management Design**

Objective: Define the design of data structures, storage mechanisms, and retrieval processes.

Rationale: Proper data management ensures efficient storage, fast retrieval, and reliable manipulation of data in alignment with system requirements.

- **User Interface Design**

Objective: Develop an intuitive and user-friendly interface that supports smooth interaction with the system.

Rationale: A well-crafted interface enhances usability and user satisfaction, contributing significantly to the system's overall acceptance and effectiveness.

- **Security and Access Control**

Objective: Establish mechanisms for safeguarding system data and resources through secure authentication and authorization.

Rationale: Strong security controls protect sensitive information, restrict unauthorized access, and maintain the integrity and trustworthiness of the system.

- **Scalability and Performance Planning**

Objective: Plan for future system growth and ensure optimal performance under varying workloads.

Rationale: Incorporating scalability and performance considerations enables the system to adapt to increasing user demands and ensures smooth operation, even during peak usage periods.

Design Goal

The design goal of the system is to translate the requirements identified during the analysis phase into a structured architectural blueprint that guides the development of the Agrinet platform. It ensures that the system's components, modules, data structures, and interfaces are clearly defined so the implementation can proceed efficiently and accurately. The primary aim is to create a scalable, secure, and user-friendly system that meets both the functional needs of farmers, buyers, experts, and administrators, as well as the non-functional requirements such as performance, usability, and reliability. By establishing clear design goals, the system design phase provides a foundation for consistent implementation, testing, deployment, and long-term maintenance of the platform.

3.2.Current Software Architecture

In the Ethiopian agricultural context, there is **no existing digital software architecture** that supports the trading process between farmers, buyers, and agricultural experts in most rural and semi-urban areas. The current system operates almost entirely through **traditional,manual methods**, where farmers depend on local markets, brokers, personal networks, and word-of-mouth communication to sell their products. Buyers also rely on travelling to markets, contacting middlemen, or making phone calls to find the goods they need.

Because of this manual approach, the current environment lacks any structured software components such as centralized databases, user management modules, product catalog systems, or digital communication channels. There is also no automated record-keeping, price comparison mechanism, or organized way of accessing agricultural advisory services.

As a result, the existing architecture is essentially **non-digital**, characterized by fragmented communication, inconsistent pricing, and limited market reach. This creates a significant gap that the Agrinet system aims to fill by introducing a modern software architecture designed specifically for the Ethiopian agricultural market, supporting local conditions such as low internet connectivity, mobile-first usage, and diverse user literacy levels.

3.3 Proposed Software Architecture

3.3.1 Overview

The proposed software architecture for the Agrinet platform introduces a **modern, scalable,and mobile-friendly digital system** designed to transform the agricultural trading process in Ethiopia. Unlike the current manual environment, the new architecture provides a structured technological foundation that supports direct farmer–buyer interaction, transparent product listings, reliable communication, and access to expert agricultural knowledge.

The architecture follows a **multi-layered design**, separating the system into clearly defined components to improve maintainability, performance, and scalability. The platform consists of:

- **Client Layer:** Aweb interface optimized for Ethiopian farmers and buyers, who primarily access the internet through smartphones.

- **Application/Service Layer:** RESTful APIs that manage all core functionalities such as user authentication, product catalog management, orders, chat, expert advisory content, and admin operations.
- **Business Logic Layer:** Independent subsystems responsible for processing requests, enforcing rules, handling transactions, and executing the system's functional requirements.
- **Data Management Layer:** A centralized relational database managing users, products, orders, chats, reports, and expert content, along with cloud storage for images and media.
- **Infrastructure Layer:** Cloud-based servers and services designed to handle low-bandwidth environments, scalable traffic, and data security needs.

3.3.2.Subsystem Decomposition

1. User Management Subsystem

Responsibilities

- User Registration
- Login / Authentication
- Authorization (role-based access: Farmer, Buyer, Expert, Admin)
- Manage User Profiles

Interacts With

- Admin Subsystem
- Marketplace Subsystem

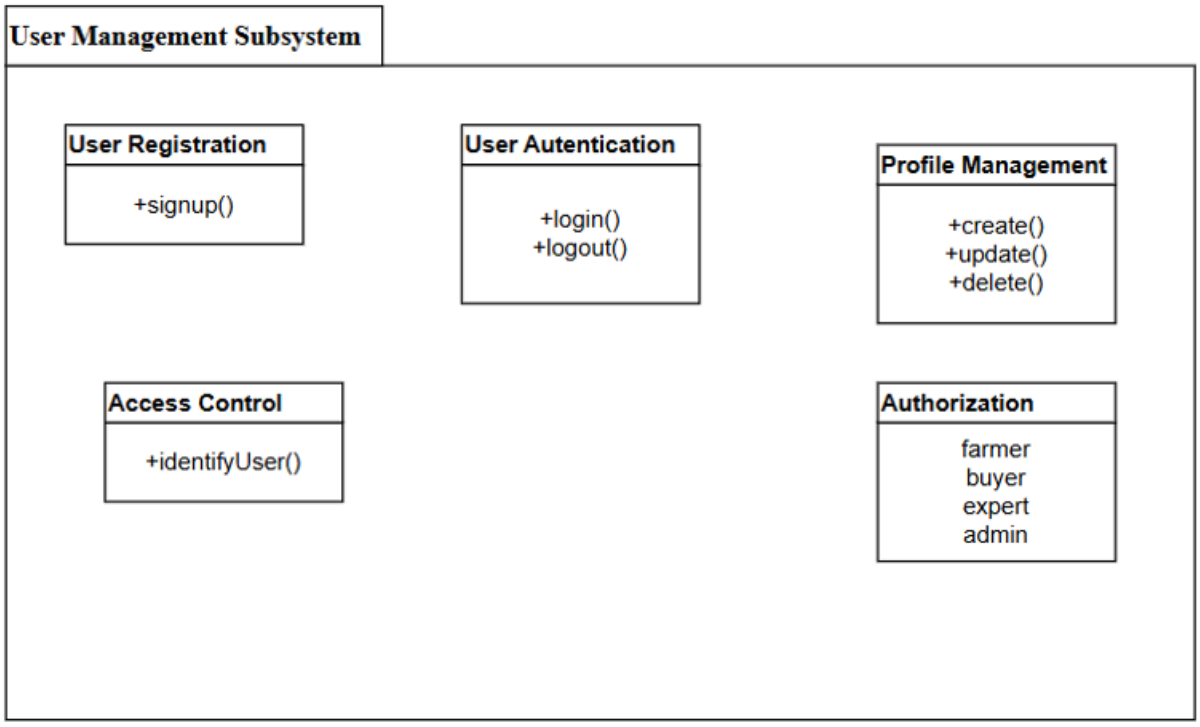


Figure 33 User Management Subsystem

Farmer Management Subsystem

Responsibilities

- Manage Farmer Profiles
- Manage Verification Status
- Allow product posting, editing, deletion
- View Orders received
- Access expert advisory

Interacts With

- Marketplace Subsystem
- Order Management Subsystem
- Expert Advisory Subsystem

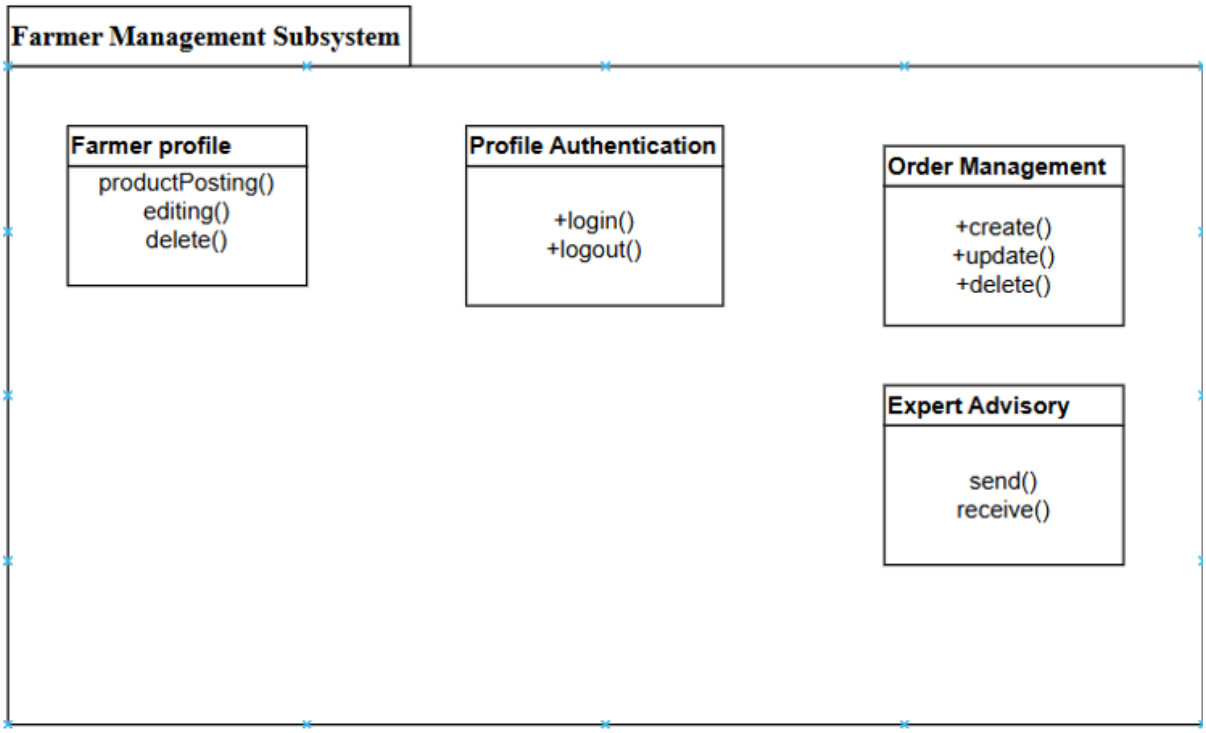


Figure 34 Farmer Management Subsystem

3. Buyer Management Subsystem

Responsibilities

- Manage buyer profiles
- Browse products
- Search & filter marketplace
- Add to cart
- Place orders
- View order history

Interacts With

- Marketplace Subsystem
- Order Management Subsystem

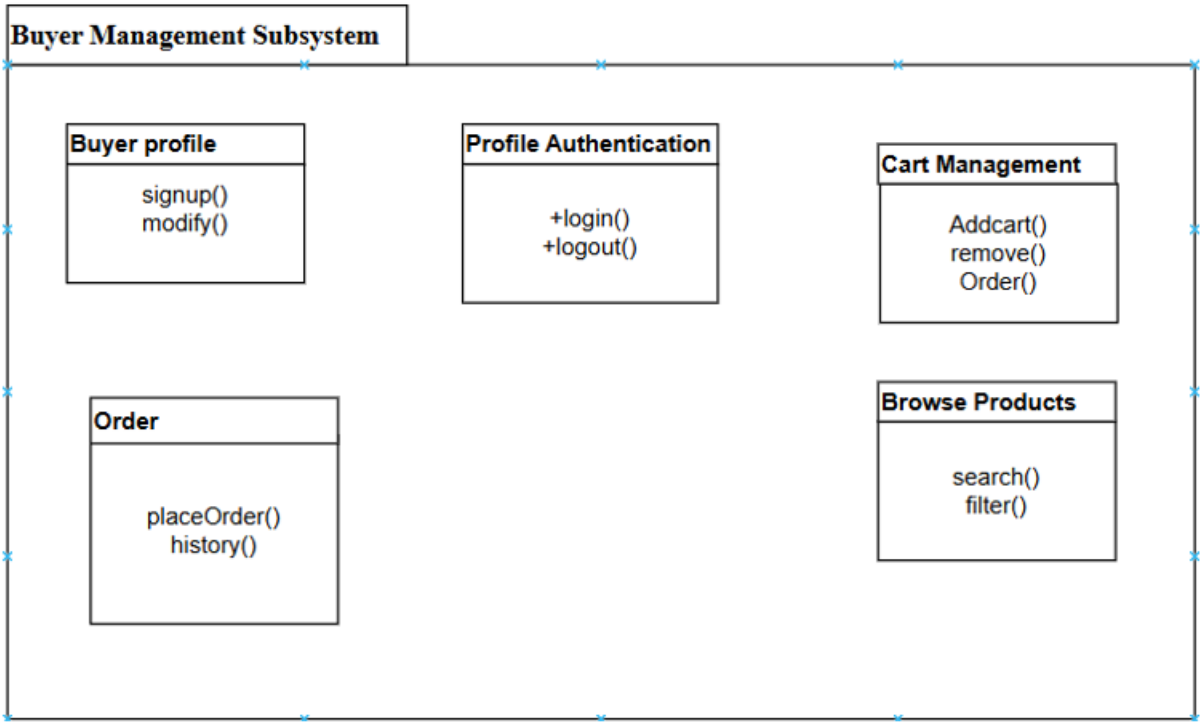


Figure 35 Buyer Management Subsystem

4. Marketplace Subsystem

Responsibilities

- Product Listing (Add, Edit, Delete)
- Product Catalog Display
- Search, Browse, Filter Products
- Product Details View
- Product Availability Management

Interacts With

- Farmer Subsystem
- Order Management Subsystem
- Admin Subsystem (moderation)

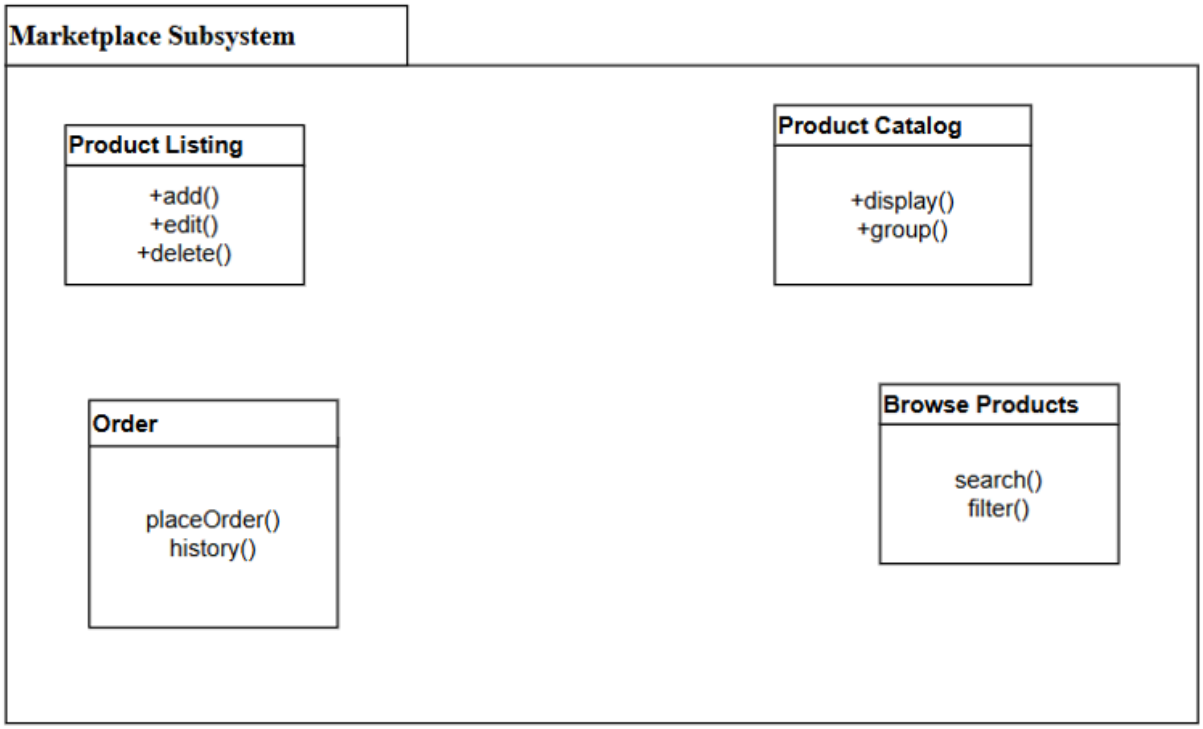


Figure 36 Marketplace Subsystem

5. Shopping Cart & Order Management Subsystem

Responsibilities

- Add to Cart
- Update/Remove Cart Items
- Order Placement
- Payment Processing (if implemented)
- Order Tracking (Pending, Completed, Cancelled)

- Order History

Interacts With

- Buyer Subsystem
- Farmer Subsystem
- Marketplace Subsystem

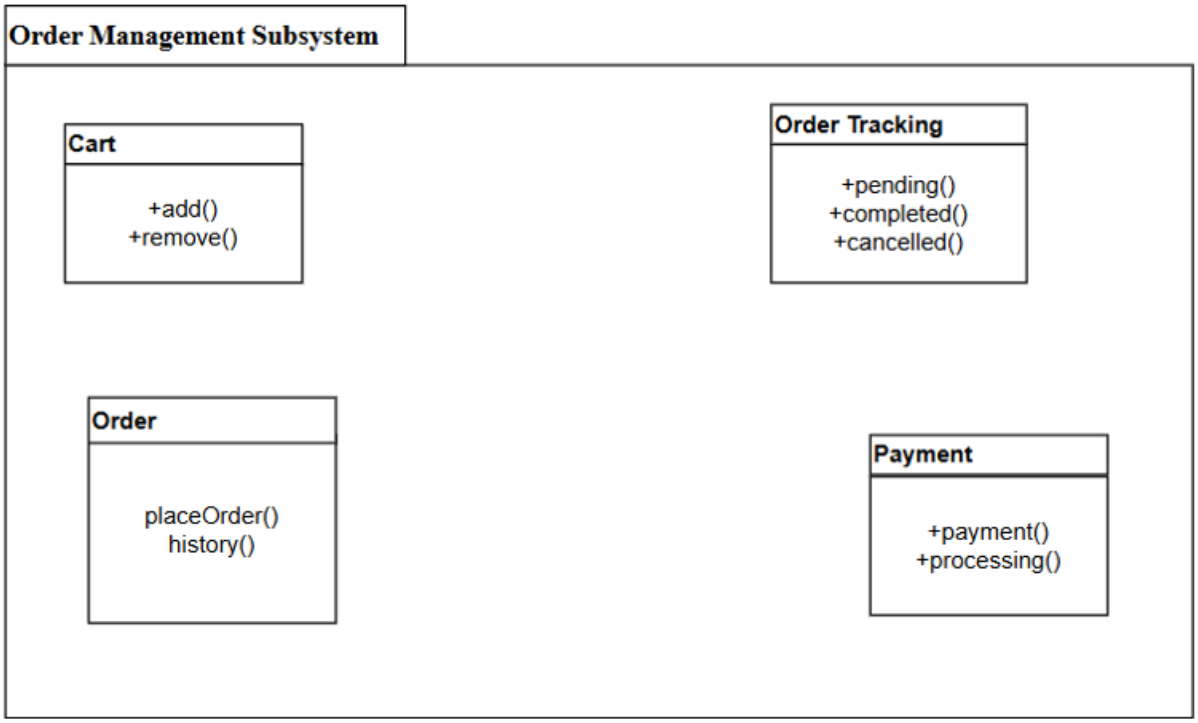


Figure 37 Order Management Subsystem

6. Expert Advisory Subsystem

Responsibilities

- Expert Login & Profile Management
- Publish Articles/Guides
- Respond to Farmer Queries
- Update Market Alerts

Interacts With

- Farmer Subsystem
- User Subsystem

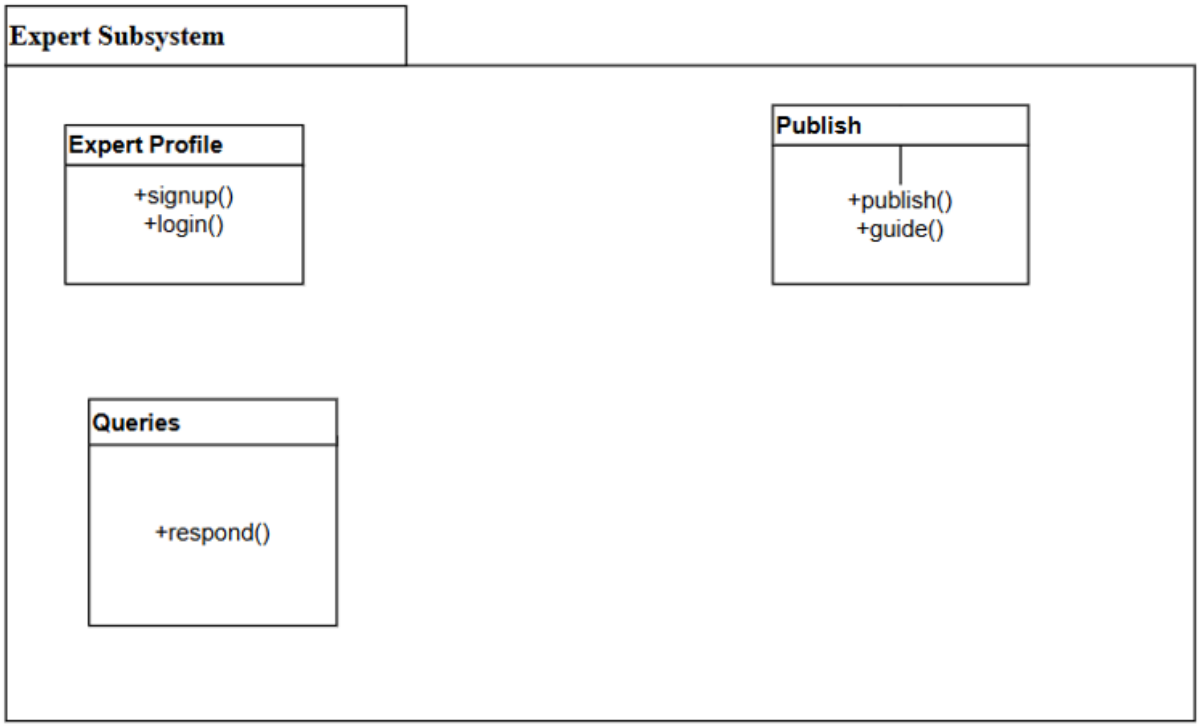


Figure 38 Expert Subsystem

8. Admin Subsystem

Responsibilities

- User Account Management (block/unblock/delete)
- User Verification (farmer / seller)
- Manage Products (approve/remove)
- Manage Reports
- View Analytics
- Monitor system activities

Interacts With

- All subsystems

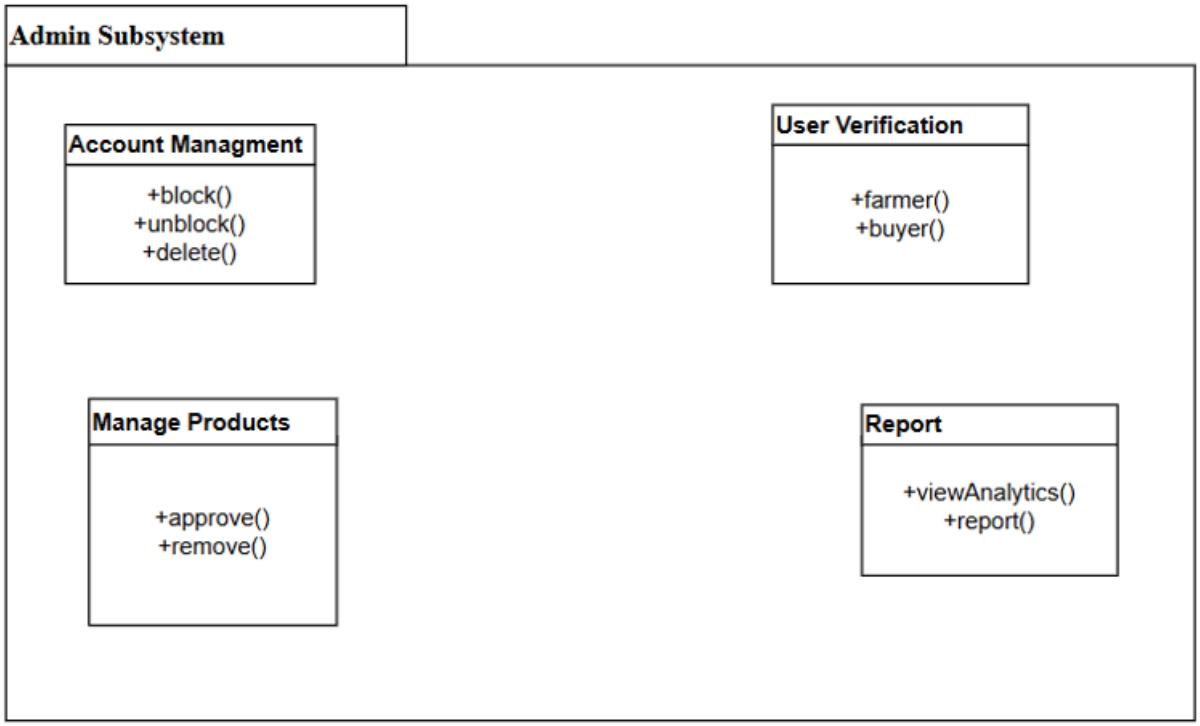


Figure 39 Admin Subsystem

The subsystem decomposition of the Agrinet platform provides a structured and modular breakdown of the system, ensuring that each major functionuser interaction, business processing, and data storageis clearly separated. This separation allows developers to better understand how different parts of the system operate independently while still contributing to the overall workflow. By dividing the system into meaningful subsystems such as User Management, Product Marketplace, Order Processing, Expert Advisory, and Admin Control, the architecture becomes more organized, maintainable, and scalable. Each subsystem can be developed, tested, and optimized individually without affecting other components, reducing complexity and improving development efficiency.

Furthermore, the decomposition aligns withreal-world operational needs by mapping responsibilities to logical units. This structure enhances system reliability, since issues within one subsystemsuch as product listing or chat messagingcan be isolated and resolved without impacting the rest of the platform. It also ensures flexibility for future growth, enabling new features to be added to specific subsystems without redesigning the entire system. Overall, subsystem decomposition strengthens the architectural foundation of The platform by promoting clarity, reusability, and long-term sustainability of the software.

3.3.2.1. Component Diagram

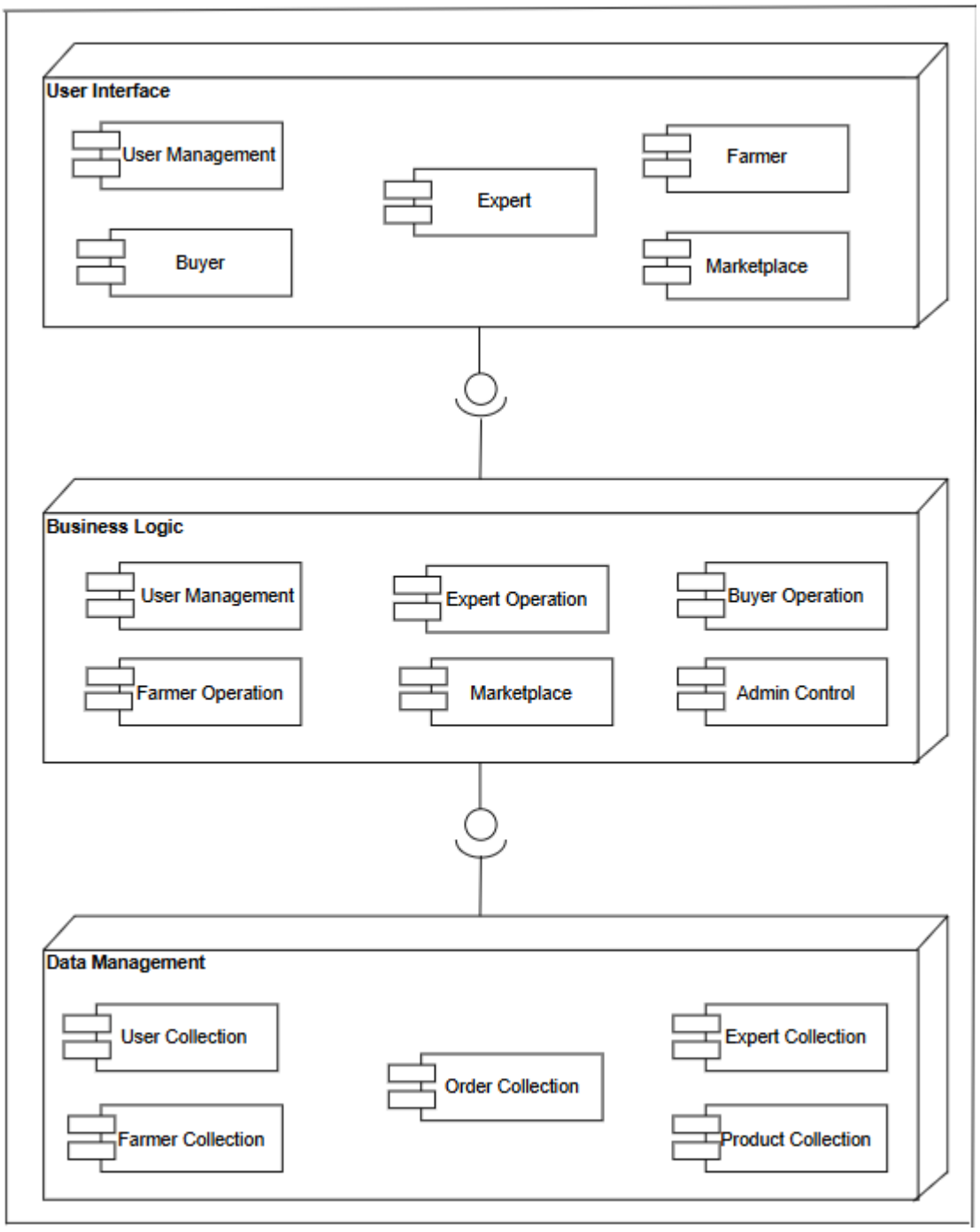


Figure 40 Component Diagram

3.3.3.Hardware/Software Mapping

The platform uses a multi-layer architecture where software components are systematically mapped onto the appropriate hardware infrastructure. This ensures efficient performance, scalability, and reliable operation across diverse environments, including rural settings with limited connectivity.

1. User Interface (UI) Layer
Software Components

- Web-based frontend (HTML, CSS, JavaScript, React/
- Mobile-first responsive UI framework
- User interaction modules:
    - Registration/Login screen
    - Product posting and browsing interfaces
    - Cart and order pages
    - Expert advisory UI
    - Admin dashboard UI

Mapped Hardware

- **Client-Side Devices** (from Requirement Analysis – Hardware Consideration)
    - Android/iOS smartphones with minimum **2GB RAM**
    - Tablets with touch-enabled screens
    - Desktop/laptop computers (Windows/Linux/macOS)
    - Basic hardware requirements:
        - **4GB RAM**, dual-core processor
        - Screen resolution supporting responsive layouts
    - Camera-equipped mobile phones for product image uploads
    - Devices with **3G or better** internet connectivity

2. Business Logic Layer
Software Components

This layer contains the functional processing modules described inSubsystem Decomposition:

- User Management Module
- Farmer Management Subsystem
- Buyer Management Subsystem

- Marketplace Subsystem
- Order & Cart Management
- Expert Advisory Service
- Reporting and Notification Service
- Admin Management & Moderation Logic

- Security modules (authentication, authorization, access control)

These modules enforce system rules, validation, workflows, and application behavior.

Mapped Hardware

- **Cloud-based Application Servers**
  - Virtual machines/containers running backend frameworks CPU: Multi-core cloud processors scalable on demand
  - RAM: 8–32 GB per instance
- **Load Balancers**
  - Distributes requests across multiple application nodes
- **Backup Power & Redundancy**
  - UPS, data center generators
  - Failover clusters

3. Data Management Layer

Software Components

- Centralized relational database (PostgreSQL)
- ORM layer for data access
- Cloud storage for product images, documents, articles
- Backup/archiving and data integrity services
- Logging and monitoring tools

The data dictionary and database schema (from Requirement Analysis) define the stored tables such as:

- Users
- Farmer profiles
- Products
- Orders & order items
- Cart
- Articles
- Market alerts

- Reports

Mapped Hardware

- **Cloud Database Servers**
  - High-availability replication
  - Minimum specs:
    - 4–16 GB RAM
    - SSD storage (50–500 GB depending on expected scale)
- **Storage Infrastructure**
  - Cloud-based object storage (S3, Azure Blob, etc.) for:
    - Images
    - Expert guides

4. API Gateway Layer

Software Components

- API Gateway Service:
  - Routes external requests to internal microservices
  - Request validation and traffic monitoring
- RESTful API Endpoints supporting:
  - User registration/authentication
  - Product listing and catalog operations
  - Order processing
  - Advisory content
  - Admin operations

Mapped Hardware

- **Cloud-Based API Gateway Hardware**
  - Hosted on cloud platforms
  - Virtualized compute nodes:
    - 4–8 GB RAM
    - Multi-core vCPU
  - Integrated with load balancers and backend servers

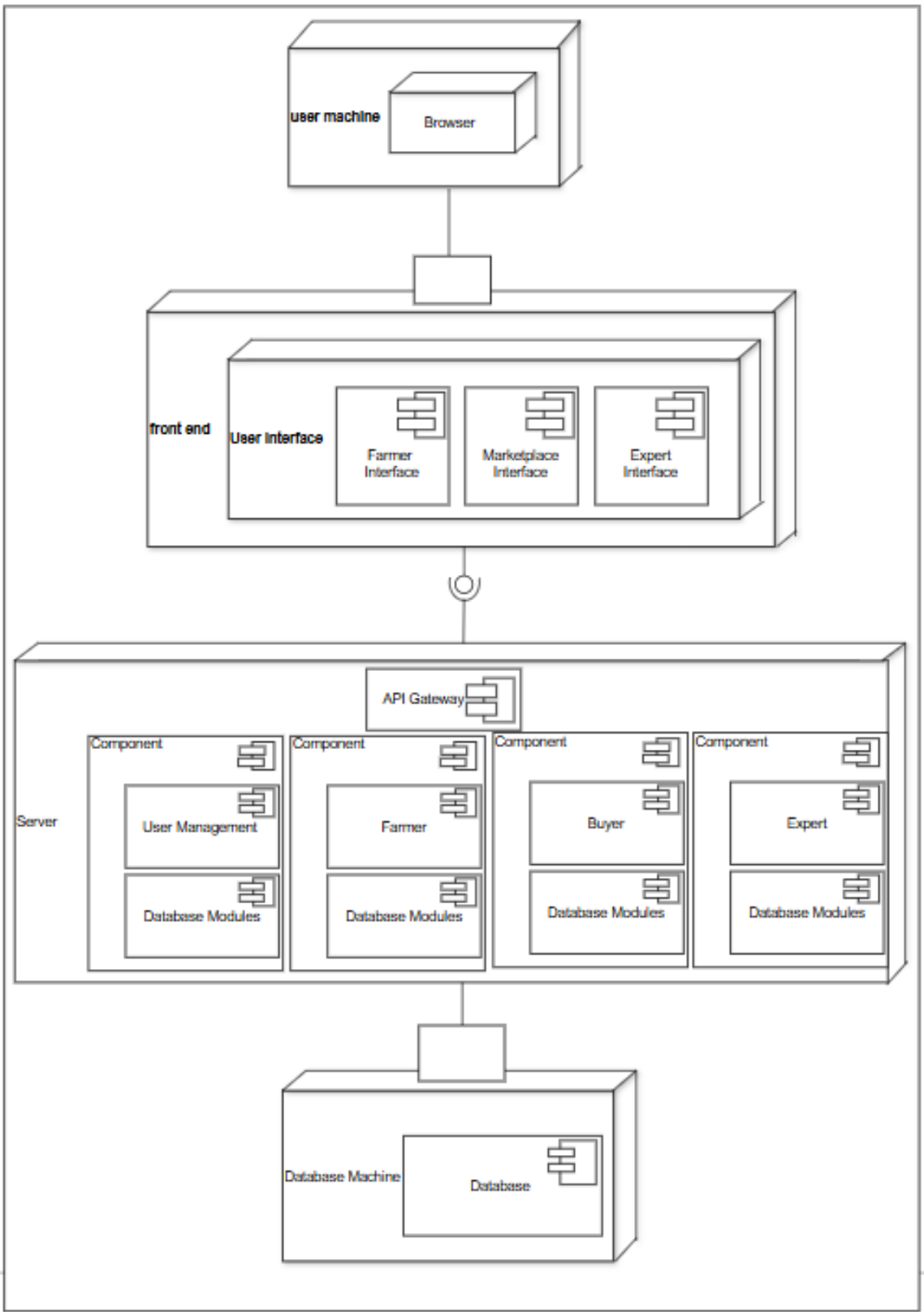


Figure 41 Hardware /Software Mapping Diagram

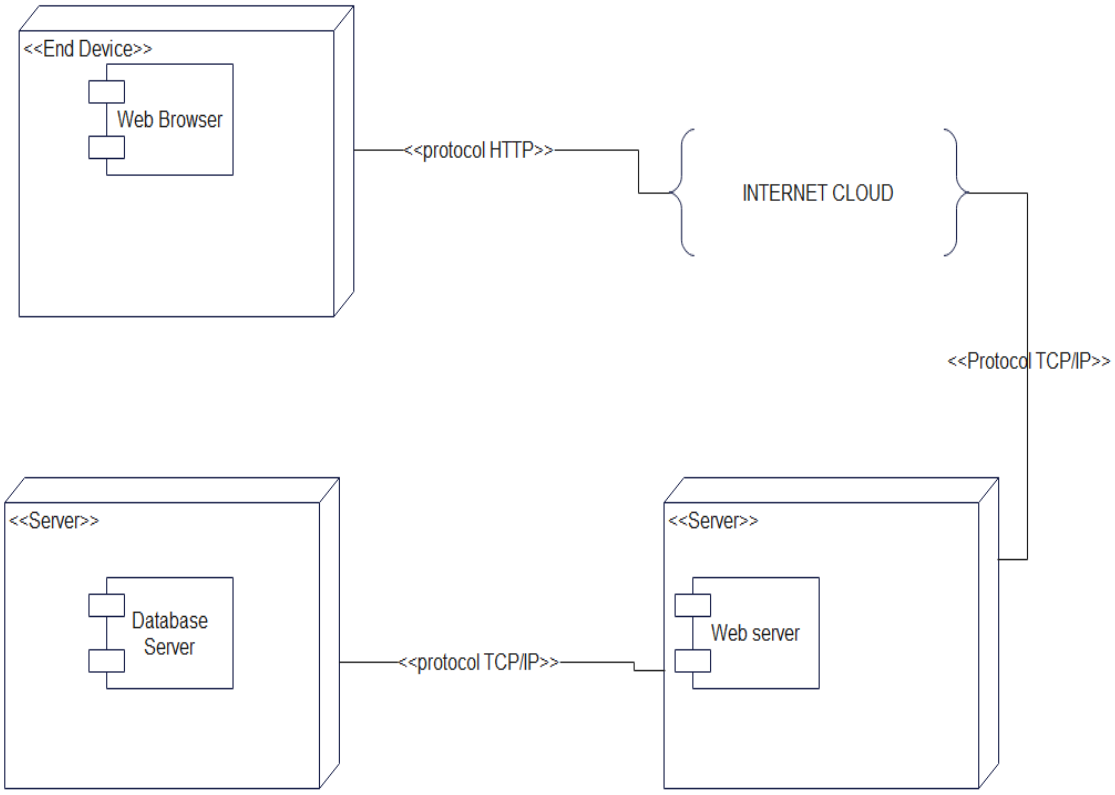


Figure 42 Deployment Diagram

3.3.4.Persistent Data Management

Persistent data management refers to the long-term storage of information and the supporting systems needed to maintain it. This part usually outlines the structure of the data, the choice of a database system, and details the specific information that must be kept beyond a single program session. These stored data values exist beyond any one instance of use.

Database Technology Selection: Our implementation utilizes a **multi-model database approach, primarily leveraging PostgreSQL**. While often categorized as a relational (SQL) database for its core strengths in ACID compliance and complex joins, PostgreSQL provides robust native support for the JSONB data type. This allows us to adopt a **document-oriented design pattern** within a reliable, transactional framework. Effectively, data is stored in flexible JSON documents while still benefiting from strong consistency, rich SQL queries, and full indexing capabilities.

Table 57 User

Field	Data Type	value
UserID	Integer	101
FullName	VARCHAR (100)	“LisanGebretensay”
Email	String	“lisan@agrinet.et”
PhoneNumber	VARCHAR (20)	“+251943343057”
Password	VARCHAR (255)	**********
UserRole	ENUM	“Farmer”
RegistrationDate	DATETIME	2025-09-15 10:30:00

Table 58 Farmer Profile

Field	Data Type	value
FarmerID	Integer	101
FarmerName	VARCHAR (100)	“Lisan Family Farm”
FarmerType	VARCHAR (50)	“Crop & Livestock”
VerificationStatus	ENUM	“Verfied”

Table 59 Buyer Profile

Field	Data Type	value
BuyerID	Integer	201
PreferredCatagory	VARCHAR (200)	“Coffee, Teff, Honey”

Table 60 Agricultural Expert

Field	Data Type	value
ExpertID	Integer	301
Qualification	VARCHAR (255)	“PhD in Agronomy”
Specialization	VARCHAR (100)	“Soil Management”
ExperienceYears	Integer	12

Table 61 Product

Field	Data Type	value
ProductID	Integer	5001
FarmerID(FK)	Integer	101
ProductName	VARCHAR (100)	“Coffee”
Price	DECIMAL (10,2)	1000
DatePosted	DATETIME	2025-09-15 11:30:00
Status	ENUM	“Available”

Table 62 Cart

Field	Data Type	value
CartID	Integer	7001
BuyerID(FK)	Integer	201
ProductID(FK)	Integer	5001
Quantity	Integer	3
DateAdded	DATETIME	2025-09-15 12:30:00

Table 63 Orders

Field	Data Type	value
OrderID	Integer	8001
BuyerID(FK)	Integer	201
FarmerID(FK)	Integer	101
TotalAmount	DECIMAL (10,2)	1352.5
OrderDate	DATETIME	2025-09-15 12:50:00
PaymentStatus	ENUM	“paid”
Status	ENUM	“Active”

Table 64 Order Items

Field	Data Type	value
OrderItemID	Integer	9001
OrderID(FK)	Integer	8001

ProductID(FK)	Integer	5001
Quantity	Integer	3
Price	DECIMAL (10,2)	1000.0

Table 65 Chat

Field	**Data Type**	**value**
ChatID	Integer	10001
SenderID(FK)	Integer	201
RecieverID(FK)	Integer	101
Message	TEXT	“which coffee type”
TimeStamp	DATETIME	2025-09-15 12:50:00

Table 66 Alert

Field	**Data Type**	**value**
AlertID	Integer	12001
ExpertId(FK)	Integer	301
Title	VARCHAR (150)	“Coffee Leaf Rust Alert”
Message	TEXT	“farmers in the region ……..”
TimeStamp	DATETIME	2025-09-15 12:55:00

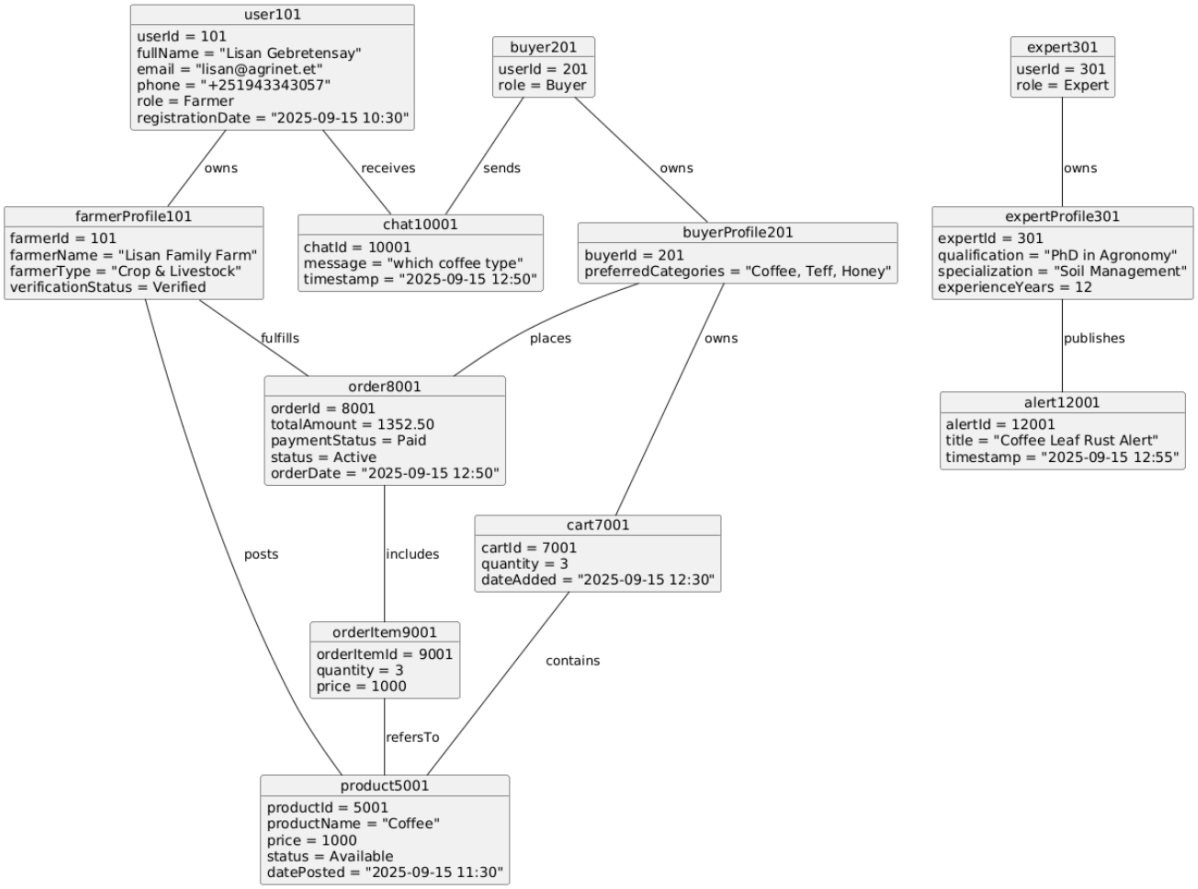


Figure 43 Persistent Data Management

3.3.5.Access Control and Security

Access control and security are critical components of the Agrinet platform, ensuring that system resources, user data, and transactions are protected against unauthorized access, misuse, and security threats. Given that the platform manages sensitive information such as user identities, product data, communications, and transaction records, strong security mechanisms are required to maintain trust, data integrity, and system reliability.

The platform implements **Role-Based Access Control (RBAC)** to regulate user permissions based on predefined roles. Each user is assigned a role at registration, and access to system features is determined by their respective role.

Table 67 access control and security

Subsystem	**Actor**	**Operation**
System	-User -Admin	-login -logout
User Interface Subsystem	- User -Admin	- Read (view pages, dashboard, content)
User Management Subsystem	-User *Admin	-Read Profile -Update Profile *Read *Update *Delete
Farmer Management Subsystem	-Farmer *Admin	-Read profile -Update profile -Post Product -View Orders *Verify Farmer *Suspend/Delete Farmer
Buyer Management Subsystem	-Buyer	-Read Profile -Update Profile -Browse Product
Marketplace Subsystem	-Farmer -Buyer - *Admin	-Add Product -Update Product -Delete Product -Read Product *Approve product *Remove Product
Shopping Cart & Order Management Subsystem	-Buyer	-Add to Cart -Update Cart

		-Delete Cart -Place Order
Expert Advisory Subsystem	-Expert	-Guides
Chat Subsystem	-User	-Read Message -Send Message

3.3.6.Subsystem Services

Subsystem services define the specific functionalities provided by each subsystem of the Agrinet platform. These services describe what each subsystem offers to other subsystems and to external Actors such as farmers, buyers, experts, and administrators. Clearly defining subsystem services improves modularity, supports maintainability, and enables independent development and testing of system components.

Table 68 Subsystem Services

Subsystem	**Service**	**Operation**	**Description**
Interface Subsystem	-User Management -Farmer -Buyer -Marketplace -Expert Advisory	-load () -unload ()	- These are interface services used to load and unload subsystem interfaces, enabling users to interact with the Agrinet system.
User Management Subsystem	-User Authentication -Profile Management -User Registration -Access Control	-login () -logout () - signup () - createUser() -updateUser() - deleteUser() - identifyUser()	-These services authenticate users, manage user profiles, handle registration, and control access based on roles.

Farmer Management Subsystem	-Farmer Profile -Farmer Verification -Product Posting	-createFarmerProfile() -updateFarmerProfile() -verifyFarmer() -postProduct() -updateProduct() -deleteProduct()	-These services manage farmer information, verify farmer accounts, and allow farmers to post and manage agricultural products.
Buyer Management Subsystem	-Buyer Profile -Product Browsing -Order Placement	-createBuyerProfile() -browseProduct() -placeOrder() -viewOrder()	-These services support buyer activities such as browsing products and placing orders.
Marketplace Subsystem	-Product Catalog -Product Search -Product Moderation	-addproduct() -updateProduct() -searchProduct() -approveProduct()	-These services manage product listings, searching, filtering, and administrative moderation of marketplace content.
Cart & Order Management Subsystem	-Cart Management -Order Processing	-addToCart() -updateCart() -removeCart() -createOrder()	-These services handle shopping cart operations, order creation.
Expert Advisory Subsystem	-Profile Management -Advisory Management	-createProfile() -publishArticle() -updateArticle()	-These services enable experts to publish agricultural advice, guides, and market alerts.
Chat Subsystem	-Discussion Management -User Management	-sendMessage() -readMessage()	-These services allow users to communicate, discuss topics, and interact within the Agrinet community.

3.4.Detailed Class Diagram

The detailed class diagram illustrates the static structure of the Agrinet system by showing the main classes, their attributes, methods, and relationships. It provides a low-level view of the system design and serves as a blueprint for implementation. The diagram focuses on core entities such as users, roles, products, orders, communication, and notifications, and demonstrates how these entities collaborate to support the system's functionality.

The Agrinet system follows an object-oriented design approach where responsibilities are clearly separated among classes. Inheritance is used to represent different user roles, while associations and aggregations define interactions between users, products, and transactions.

3.4.1.User Class

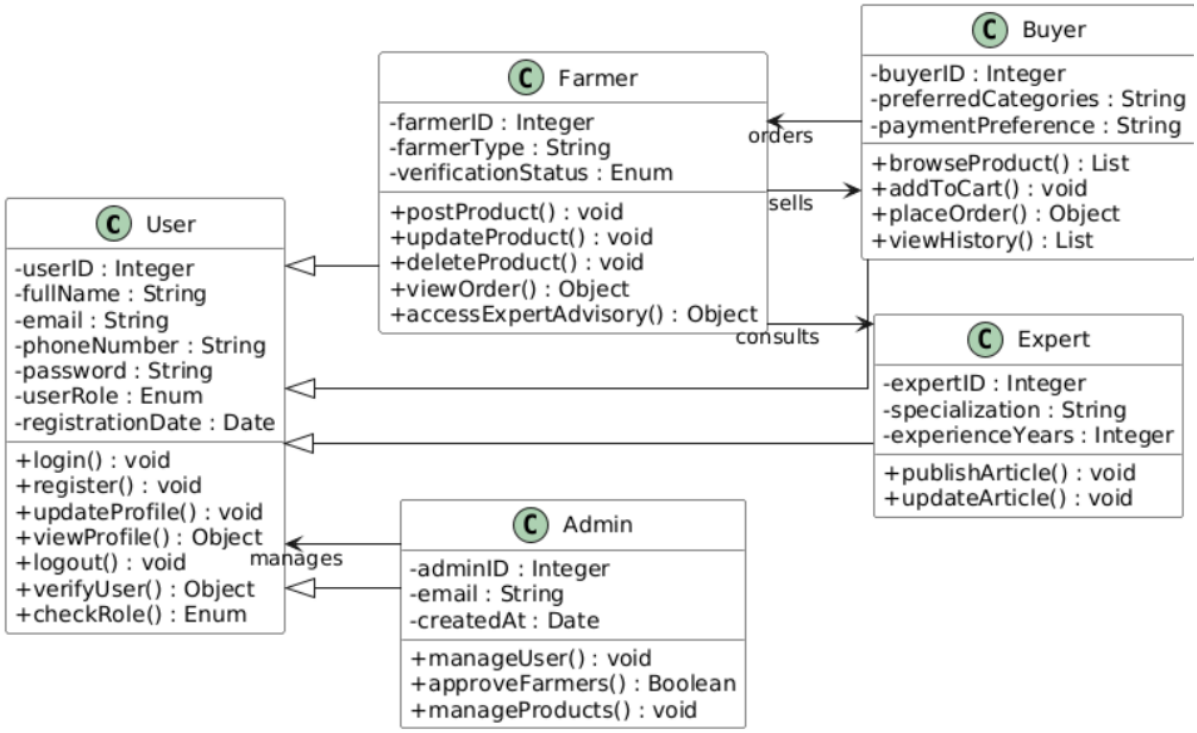


Figure 44 User Class

Table 69 User class attribute

Attribute Name	**Data Type**	**Visibility**	**Description**
UserID	Integer	private	A unique number of the user.
FullName	String	private	Full legal name of the user.
Email	String	private	Email address of the User.
PhoneNumber	String	private	Contact phone number of the user.
Password	String	private	Encrypted password of user account.
UserRole	Enum	private	Role of the user(Farmer,Expert)
RegistrationDate	Date	private	The date on which the User registered.

Table 70 User Class Operation

Operation	**Return Type**	**Description**
Login ()	void	Used to Authenticate and login in to the system.
Register ()	void	Used to register a new user.
updateprofile ()	void	Used to update User information.
viewProfile()	object	Used to view user profile detail.
logOut()	void	Used to log out from the system.

verifyUser()	object	Used to process user verification.
checkRole ()		Used to determine the role of the logged-in user.
Get ()	object	Getter method for user attributes.
Set ()	void	Setter method for user attributes.

3.4.2. Farmer class

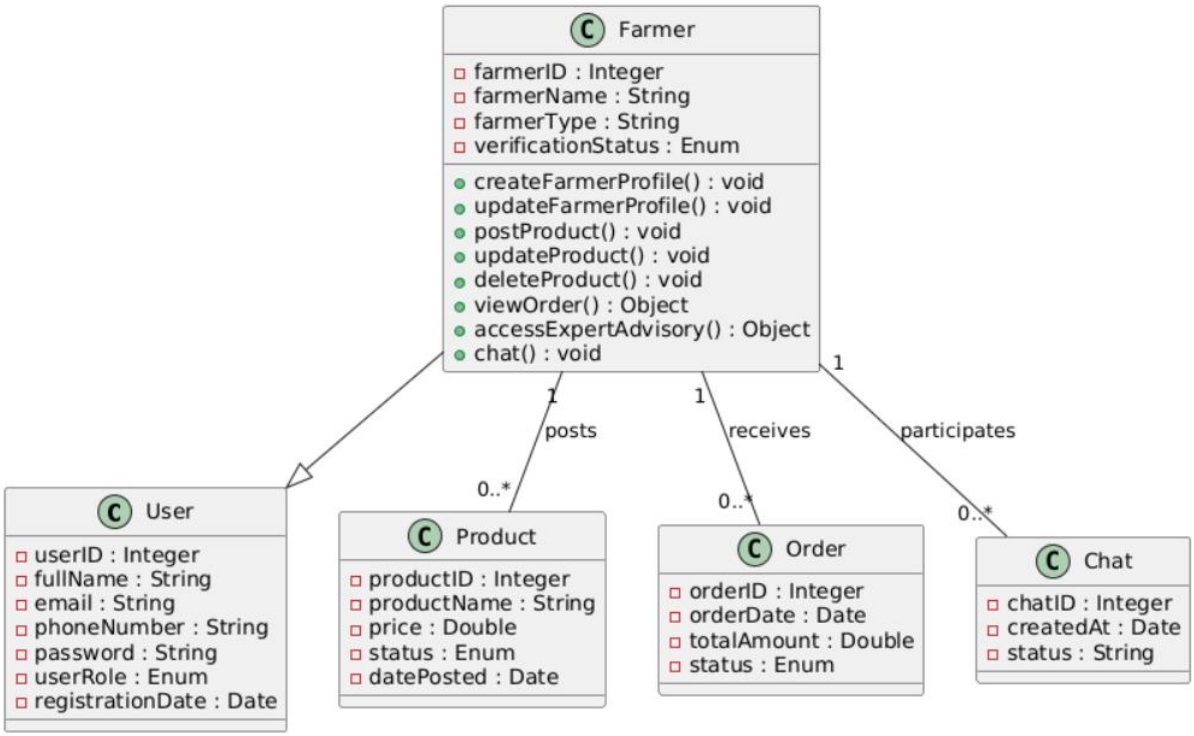


Figure 45 Farmer Class

Table 71 Farmer class Attributes

Attribute Name	**Data Type**	**Visibility**	**Description**
farmerID	Integer	private	A unique number of the farmer.
farmerName	String	private	Full legal name of the farmer.

farmerType	String	private	Type of farming.
verficationStatus	Enum	private	Indicate verification state.

Table 72 Farmer class operation

Operation	**Return Type**	**Description**
createFarmerProfile()	void	Used to create farmer profile in the system.
updateFarmerProfile ()	void	Used to updatefarmer profile information.
postProduct()	void	Used to post agricultural product to the marketplace.
updateProduct ()	void	Used to modify existing product information.
deleteProduct()	void	Used to remove a product form a marketplace.
viewOrder()	object	Used to view orders received form buyers.
accessExpertAdvisory()	object	Used to access agricultural expert guidance.
Chat ()	void	Used to communicate with buyers through the chat system.

3.4.3 Buyer Class

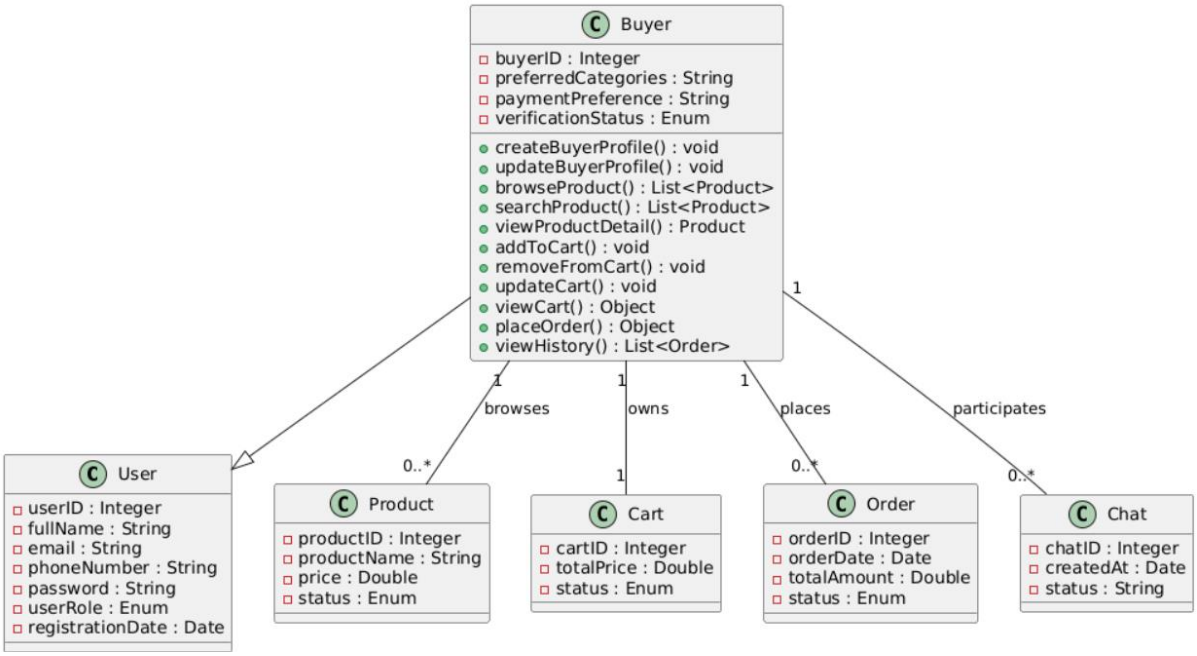


Figure 46 Buyer Class

Table 73 Buyer class Attributes

Attribute Name	**Data Type**	**Visibility**	**Description**
buyerID	Integer	private	A unique number for the buyer.
Productid (FK)	Integer	private	A unique number for product.
preferredCategories	String	private	Product categories preferred by the buyer.
paymentPreference	String	private	Preferred payment method for the buyer.
verficationStatus	Enum	private	Indicate verification state.

Table 74 Buyer Class Operation

Operation	Return Type	Description
createBuyerProfile()	void	Used to create buyer profile in the system.
updateBuyerProfile ()	void	Used to updatebuyer profile information.
browseProduct ()	List<product>	Allows the buyer to browse all available agricultural products in the market place.
searchProduct()	List<product>	Enable searching and filtering products by category or price.
viewProductDetail()	Product	Displays detail information about selected product.
addToCart()	void	Add a selected product and quantity to the buyer's shopping cart.
removeFromCart()	void	Remove a selected product from the shopping cart.
updateCart()	void	Modifies quantity of product already added to the cart.
viewCart()	object	Displays all product currently in the buyer's cart.
placeOrder()	object	Confirms the cart items, process payment.
viewHistory()	List<order>	Allows the buyer to view all previously places order.

3.4.4. Marketplace class

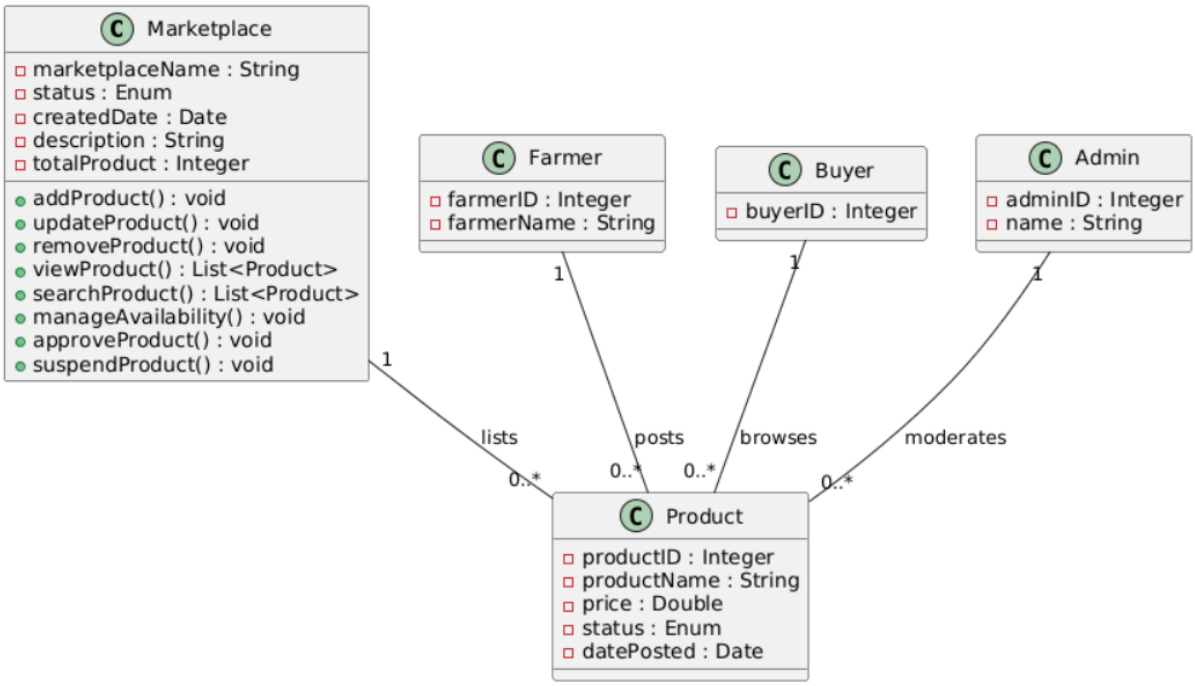


Figure 47 Marketplace Class

Table 75 Marketplace class attributes

Attribute Name	**Data Type**	**Visibility**	**Description**
Marketplace Name	String	private	Name of the marketplace.
Status	Enum	private	Current status of the market place.
createdDate	Date	private	Date when the marketplace was created.
Description	String	private	Brief description of the marketplace and its purpose.
totalProduct	Integer	private	Total number of products currently listed in the marketplace.

Table 76 Marketplace Class Operation

Operation	Return Type	Description
addProduct()	void	Add a new agricultural product to the marketplace listing.
updateProduct()	void	Updates details of an existing product in the marketplace.
removeProduct()	void	Remove a product from a marketplace.
viewProduct()	List<product>	Displays all the product available in the marketplace.
searchProduct()	List<product>	Search product by name or category.
manageAvailablity()	void	Updates product availability status.
approveProduct()	void	Approve product before it comes visible to the buyer.
suspendProduct()	void	Temporarily disable a product listing.

3.4.5 Cart & Order Class

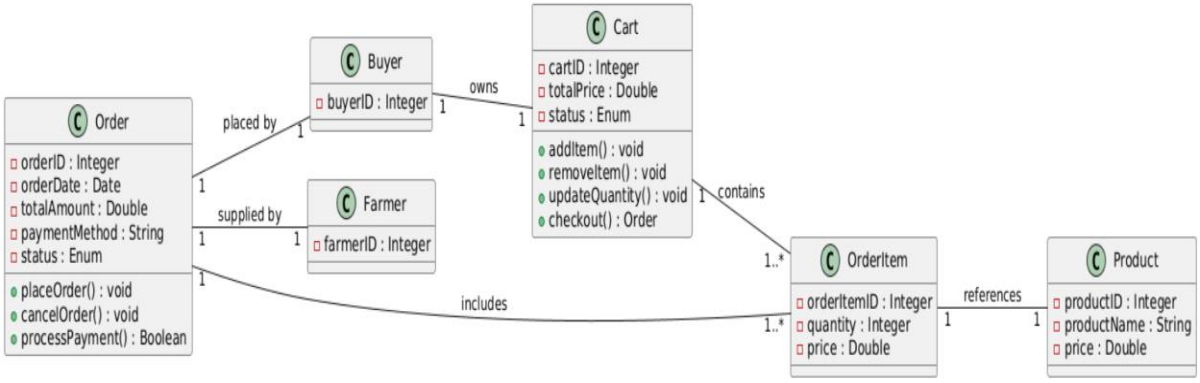


Figure 48 Cart & Order Class

Table 77 Cart class attributes

Attribute Name	Data Type	Visibility	Description
cartID	Integer	private	Unique identifier for the shopping cart.
buyerID(FK)	Integer	private	Reference to the buyer who owns the cart.
createdDate	Date	private	Date when the cart was created.
totalPrice	Double	private	Total price of all items in the cart.
Status	Enum	private	Current cart state.

Table 78 Order class attributes

Attribute Name	Data Type	Visibility	Description
orderID	Integer	private	Unique identifier for the order.
buyerID(FK)	Integer	private	Buyer who place order.
farmerID(FK)	Integer	private	Farmer supplying the product.

orderDate	Date	private	Date when the order was placed.
totalAmount	Double	Private	Total cost of the orders.
paymentMethod	String	private	Payment method used for the order.

Table 79 Cart Class Operation

Operation	**Return Type**	**Description**
addItemt()	void	Add a selected product with a specified quantity to the shopping cart.
removeItem()	void	Remove a selected product form the shopping cart.
updateQuantity()	void	Update the quantity of the existing product in the cart.
viewCart()	object	Displays all items currently added to the buyer's cart.
calculateCart()	double	Calculate the total price of all items in the cart.
clearCart()	void	Remove all items form the cart after checkout cancellation.
Checkout()	object	Initiate the checkout process and prepares for the cart order creation.

Table 80 Order class operation

Operation	**Return Type**	**Description**
placeOrder()	void	Creates a new order form the

		cart items and confirms the purchase.
cancelOrder()	void	Cancel an active order before it is completed.
viewOrder ()	object	Display information of specific order.
processPayment()	boolean	Process payment for the order and confirms the payment status.

3.4.6. Expert Advisory class

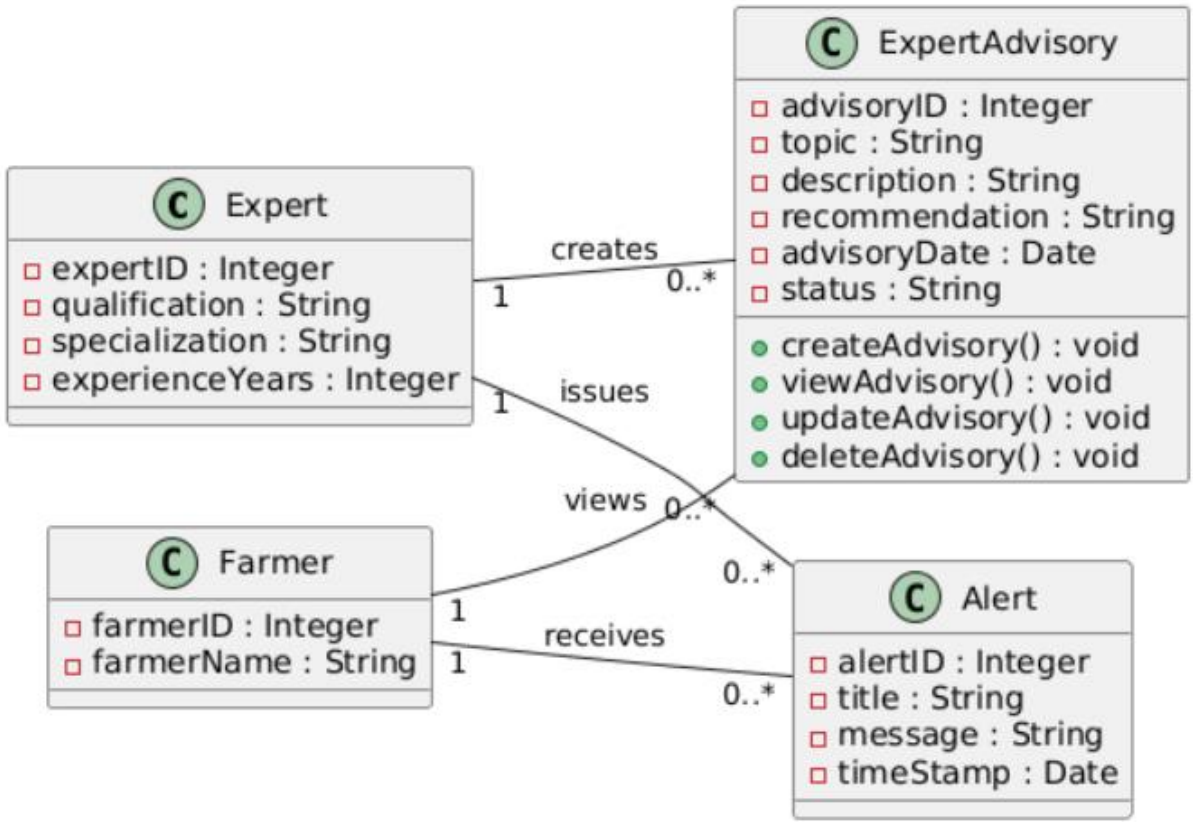


Figure 49 Expert Class

Table 81 Expert Advisory Class Attributes

Attribute Name	Data Type	Visibility	Description
advisoryID	Integer	private	Unique identifier for the export advisory.
topic	String	private	Subject or topic provided by the expert.
Description	String	private	Date when the cart was created.
Recommendation	String	private	Recommended solution or guidance given by the expert.
advisoryDate	Date	private	Date created or issued.
Status	String	private	Current status of the advisory.

Table 82 Expert Advisory Class Operation

Operation	Return Type	Description
createAdvisory()	void	Creates a new expert advisory record based on the topic, description and recommendation provided by the expert.
viewAdvisory()	void	Retrieves and displays the details of a specific advisory to farmers.
updateAdvisory()	void	Update an existing by modifying its content.
deleteAdvisory()	void	Removes an advisory record from the system when it is no longer relevant or required.

3.4.6. ChatClass

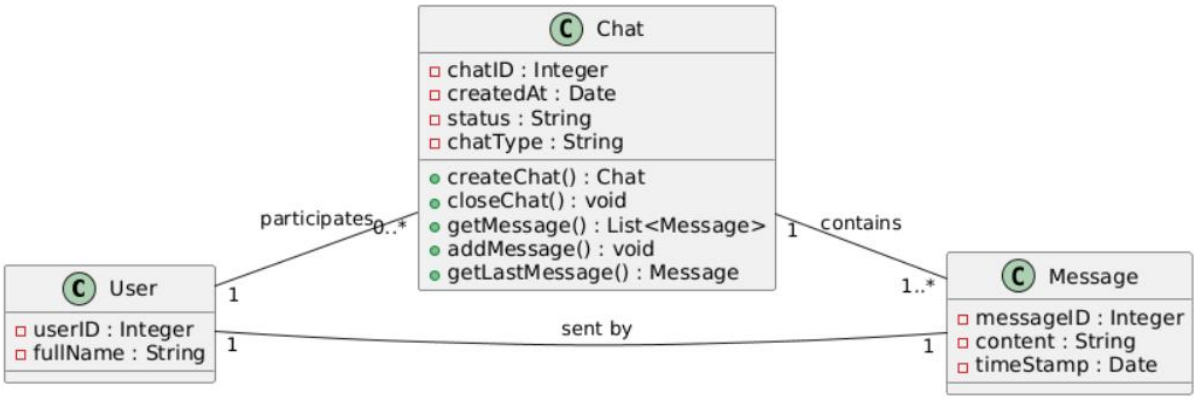


Figure 50 Chat Class

Table 83 Chat class attributes

Attribute Name	Data Type	Visibility	Description
chatID	Integer	private	Unique identifier for the chat.
createdAt	Date	private	Date and time when the chat was created.
Status	String	private	Current chat status.
chatType	String	private	Type of chat.

Table 84 Chat class operation

Operation	Return Type	Description
createChat()	Chat	Create a new chat session.

closeChat()	void	Close an active chat.
getMessage()	List<message>	Retrieves all message in the chat.
addMessage()	void	Add a new message to the chat.
getLastMessage()	Message	Returns the most recent message.

3.4.7. Admin class

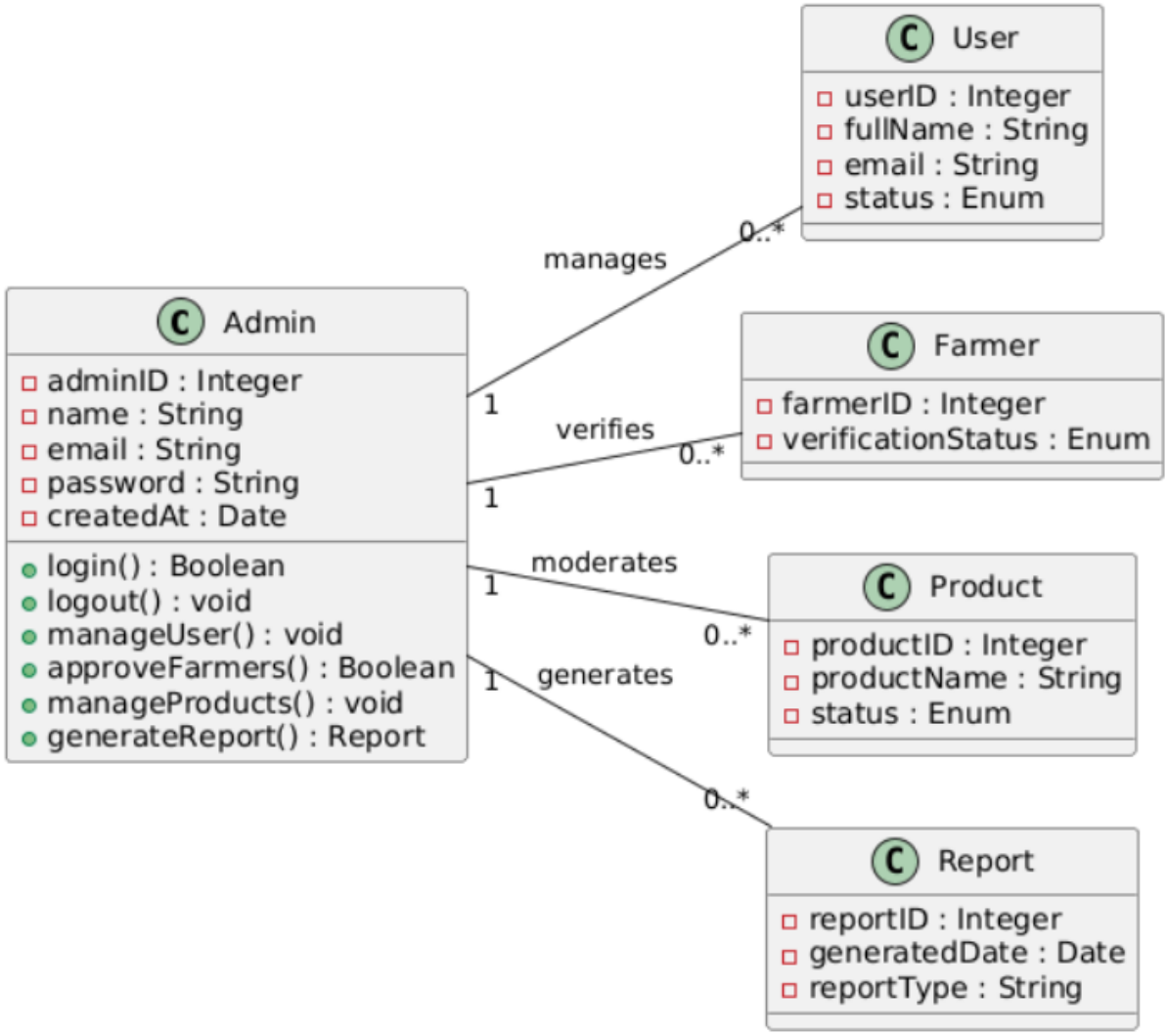


Figure 51 Admin Class

Table 85 Admin class attributes

Attribute Name	Data Type	Visibility	Description
adminID	Integer	private	Unique identifier for the admin.
Name	String	private	Full name of the admin.
Email	String	private	Add email address.
Password	String	private	Encrypted login password.
Created At	Date	private	Date the admin account was created.

Table 86 Admin class operation

Operation	Return Type	Description
login()	Boolean	Authenticate admin credential.
logout()	void	Logout the admin.
manageUser()	void	Activate, suspend and remove user.
approvalFarmers()	Boolean	Approve farmer registration.
manageProducts()	void	Add, update and remove products.
generateReport()	Report	Generate system report.

3.5 packages

Packages provide a way to organize related classes and components by grouping them under a common namespace. They combine similar elements into one structured unit, making the system easier to manage, understand, and maintain by improving code organization and clarity.

1. User Management Package

- **Purpose**: Handle user registration, authentication, authorization, and profile management for all user roles.
- **Components**:
    - User registration & login
    - Role-based access control (Farmer, Buyer, Expert, Admin)
    - Profile management
    - Account verification
    - Password reset

2. Farmer Management Package

- **Purpose**: Manage farmer-specific functions and data.
- **Components**:
    - Farmer profile management
    - Farmer verification system
    - Product posting interface
    - View orders received
    - Access to expert advisory

3. Buyer Management Package

- **Purpose**: Manage buyer-specific functions and marketplace interactions.
- **Components**:
    - Buyer profile management

- Product browsing & searching
- Shopping cart operations
- Order placement
- Order history tracking

4. Marketplace Package

- **Purpose**: Central hub for product listings and discovery.
- **Components**:
    - Product catalog display
    - Search, filter, and browse functionality
    - Product details view
    - Product availability management
    - Product moderation (admin)

5. Order & Cart Management Package

- **Purpose**: Handle the complete order lifecycle.
- **Components**:
    - Shopping cart management
    - Order placement & processing
    - Payment processing integration
    - Order tracking (pending, completed, cancelled)
    - Order history for buyers and farmers

6. Expert Advisory Package

- **Purpose**: Provide agricultural knowledge and support.
- **Components**:
    - Expert login & profile management
    - Article/guide publishing
    - Responding to farmer queries
    - Market alerts system
    - Advisory content management

7. Chat Package

- **Purpose**: Enable communication between users.
- **Components**:
    - Real-time messaging between farmers and buyers
    - Chat history
    - Notifications for new messages

8. Admin Package

- **Purpose**: System administration and moderation.
- **Components**:
    - User account management
    - Farmer/buyer verification
    - Product listing moderation
    - Report management
    - System analytics dashboard
    - Activity monitoring

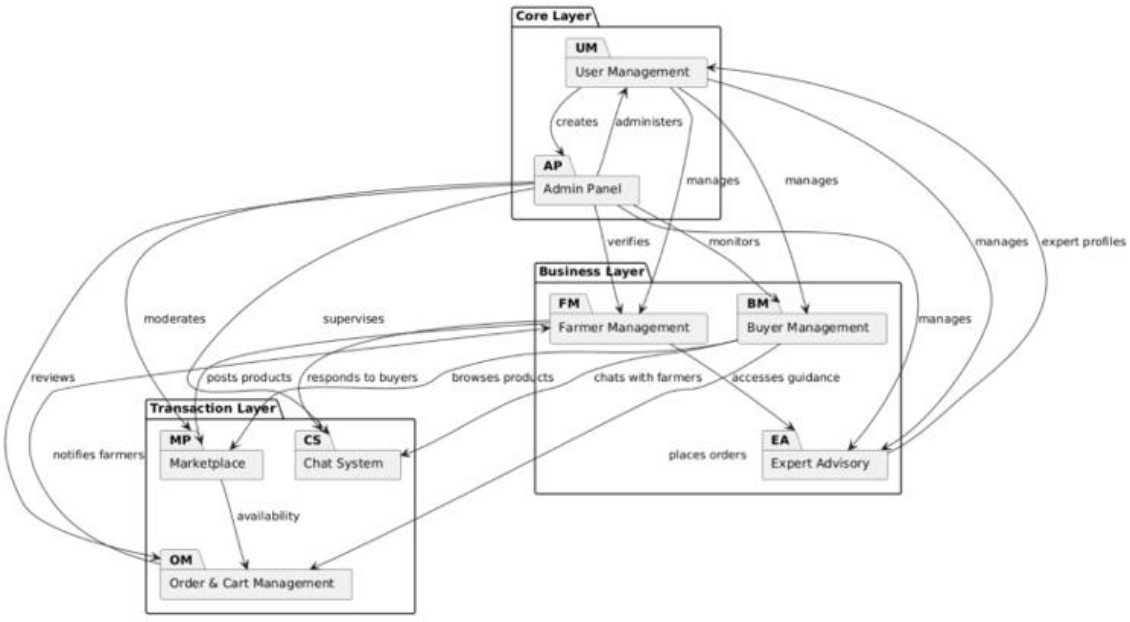
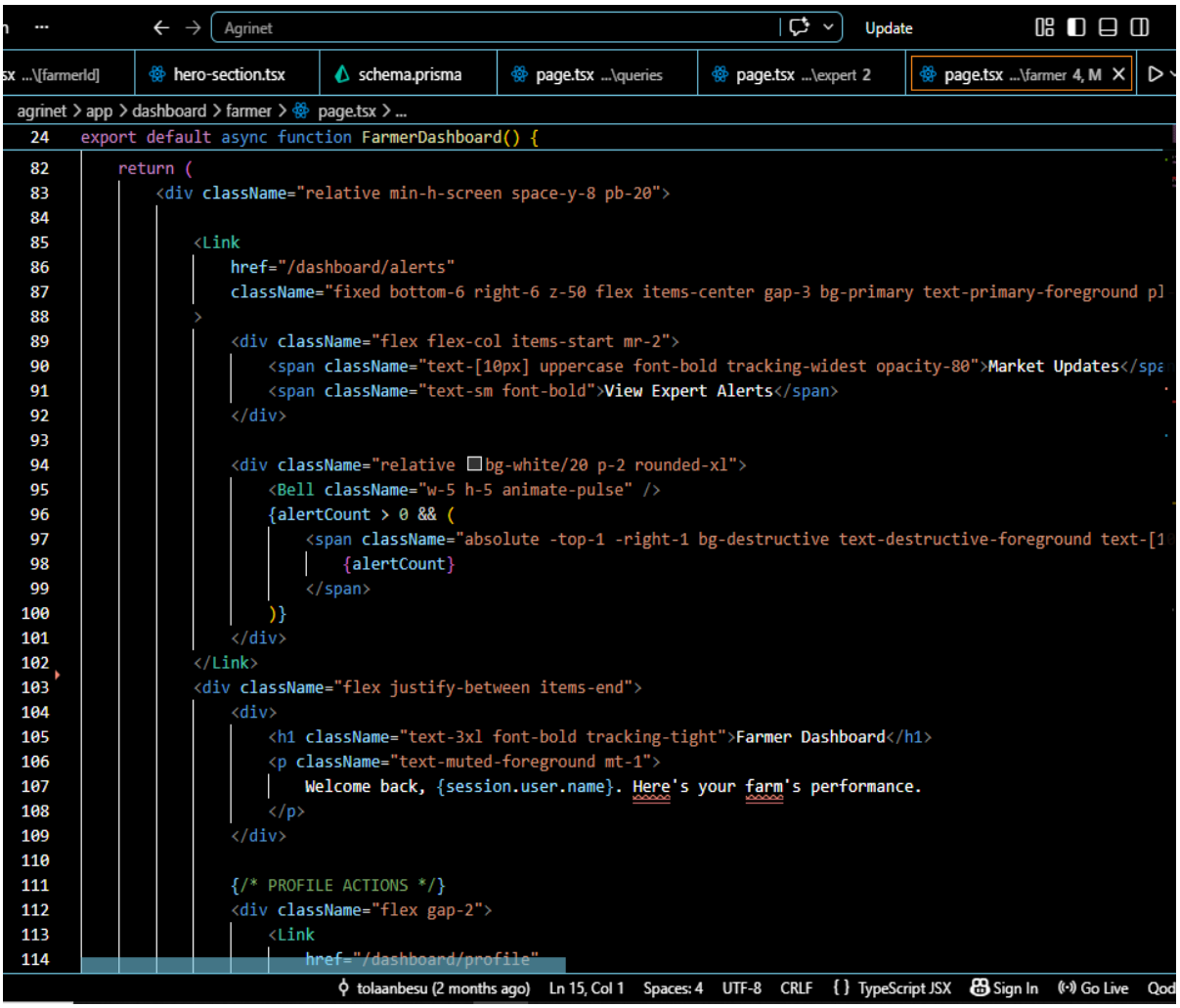


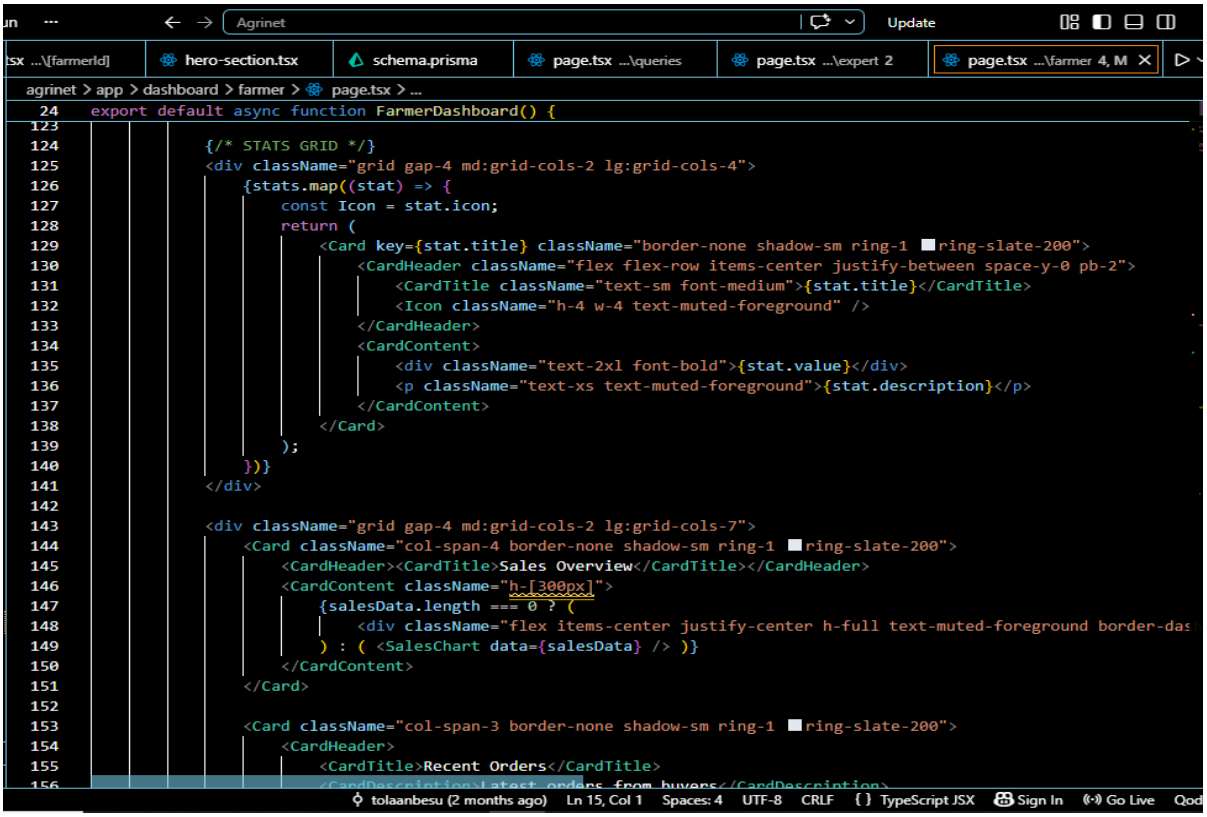
Figure 52 Package Diagram

4. CHAPTER FOUR-IMPLEMENTATION AND TESTING

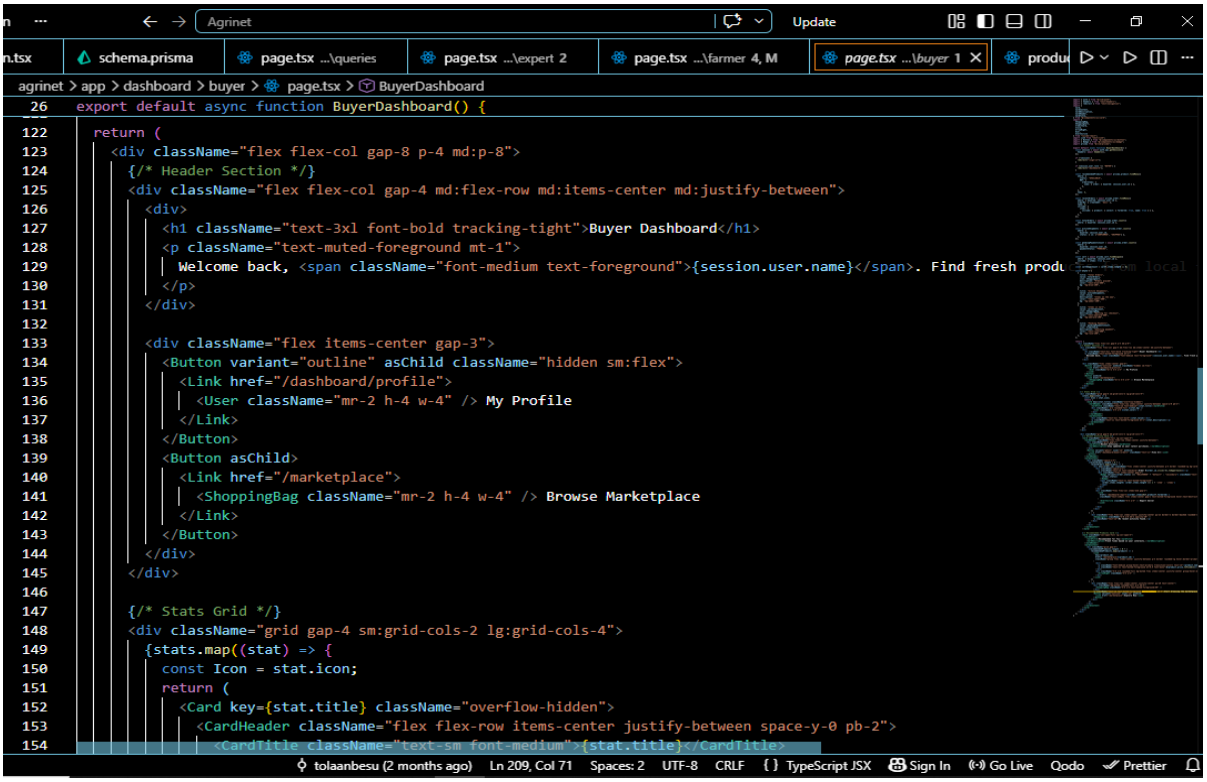
4.1 Implementation

4.1.1 sample code





Farmers page.tsx


Agrinet
Update
schema.prisma
page.tsx ...\queries
page.tsx ...\expert 2
page.tsx ...\farmer 4, M
page.tsx ...\buyer 1
agrinet > app > dashboard > buyer > page.tsx > BuyerDashboard
export default async function BuyerDashboard() {
return (

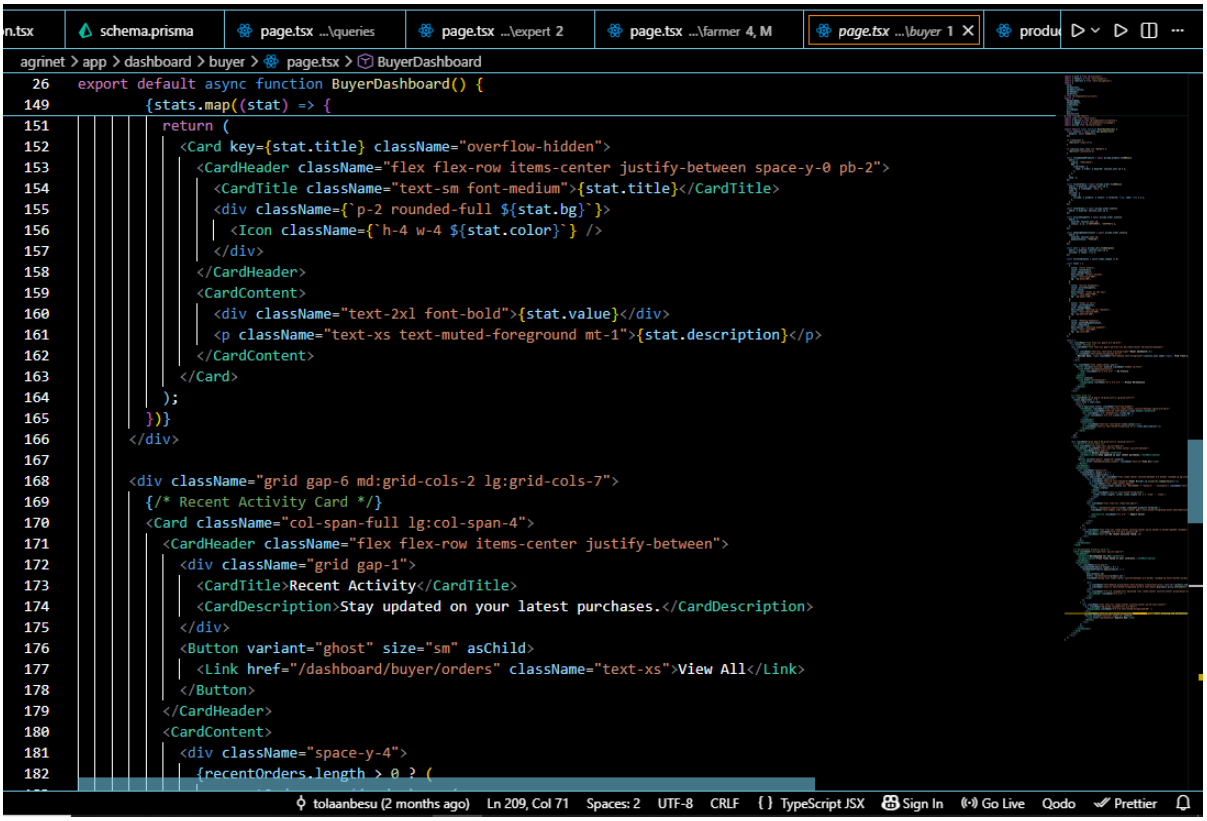
{/* Header Section */}

<h1 className="text-3xl font-bold tracking-tight">Buyer Dashboard</h1>

Welcome back, {session.user.name}. Find fresh produ

<Button variant="outline" asChild className="hidden sm:flex">
<Link href="/dashboard/profile">
<User className="mr-2 h-4 w-4" /> My Profile
</Link>
</Button>
<Button asChild>
<Link href="/marketplace">
<ShoppingBag className="mr-2 h-4 w-4" /> Browse Marketplace
</Link>
</Button>
{/* Stats Grid */}

{stats.map((stat) => {
const Icon = stat.icon;
return (
<Card key={stat.title} className="overflow-hidden">
<CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
<CardTitle className="text-sm font-medium">{stat.title}</CardTitle>
tolaanbesu (2 months ago)
Ln 209, Col 71
Spaces: 2
UTF-8
CRLF
TypeScript JSX
Sign In
Go Live
Qodo
Prettier


schema.prisma
page.tsx ...\queries
page.tsx ...\expert 2
page.tsx ...\farmer 4, M
page.tsx ...\buyer 1
agrinet > app > dashboard > buyer > page.tsx > BuyerDashboard
export default async function BuyerDashboard() {
{stats.map((stat) => {
return (
<Card key={stat.title} className="overflow-hidden">
<CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
<CardTitle className="text-sm font-medium">{stat.title}</CardTitle>
<div className={`p-2 rounded-full ${stat.bg}`}>
<Icon className={`h-4 w-4 ${stat.color}`} />

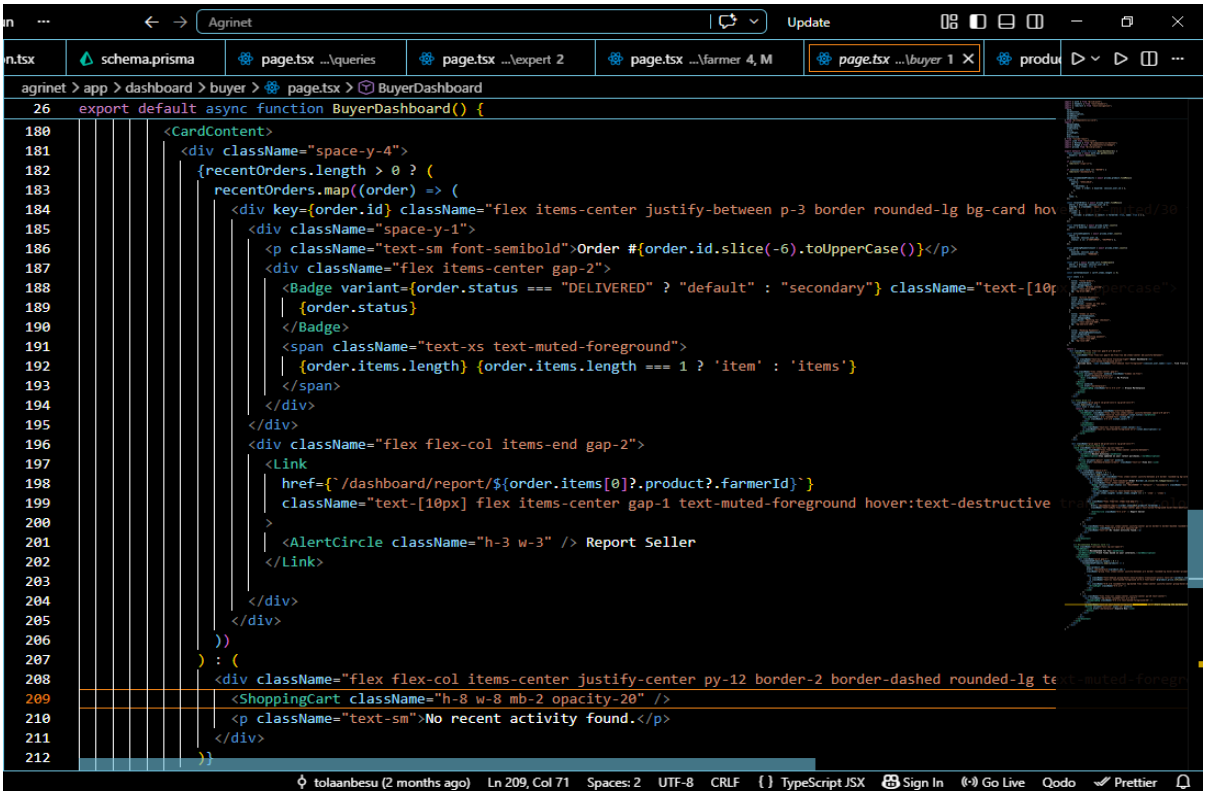
</CardHeader>
<CardContent>
{stat.value}
{stat.description}
</CardContent>
</Card>
);
})}

{/* Recent Activity Card */}
<Card className="col-span-full lg:col-span-4">
<CardHeader className="flex flex-row items-center justify-between">

<CardTitle>Recent Activity</CardTitle>
<CardDescription>Stay updated on your latest purchases.</CardDescription>

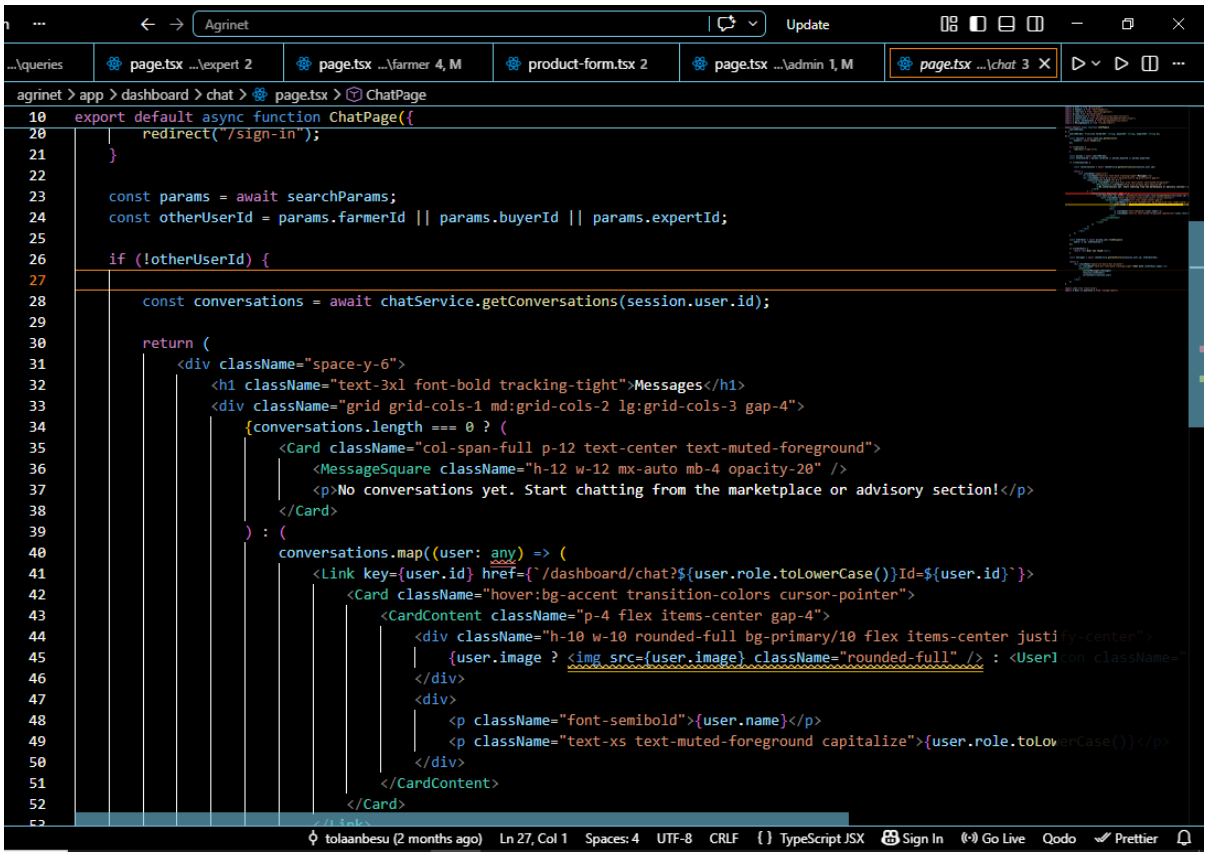
<Button variant="ghost" size="sm" asChild>
<Link href="/dashboard/buyer/orders" className="text-xs">View All</Link>
</Button>
</CardHeader>
<CardContent>

{recentOrders.length > 0 ? (
tolaanbesu (2 months ago)
Ln 209, Col 71
Spaces: 2
UTF-8
CRLF
TypeScript JSX
Sign In
Go Live
Qodo
Prettier

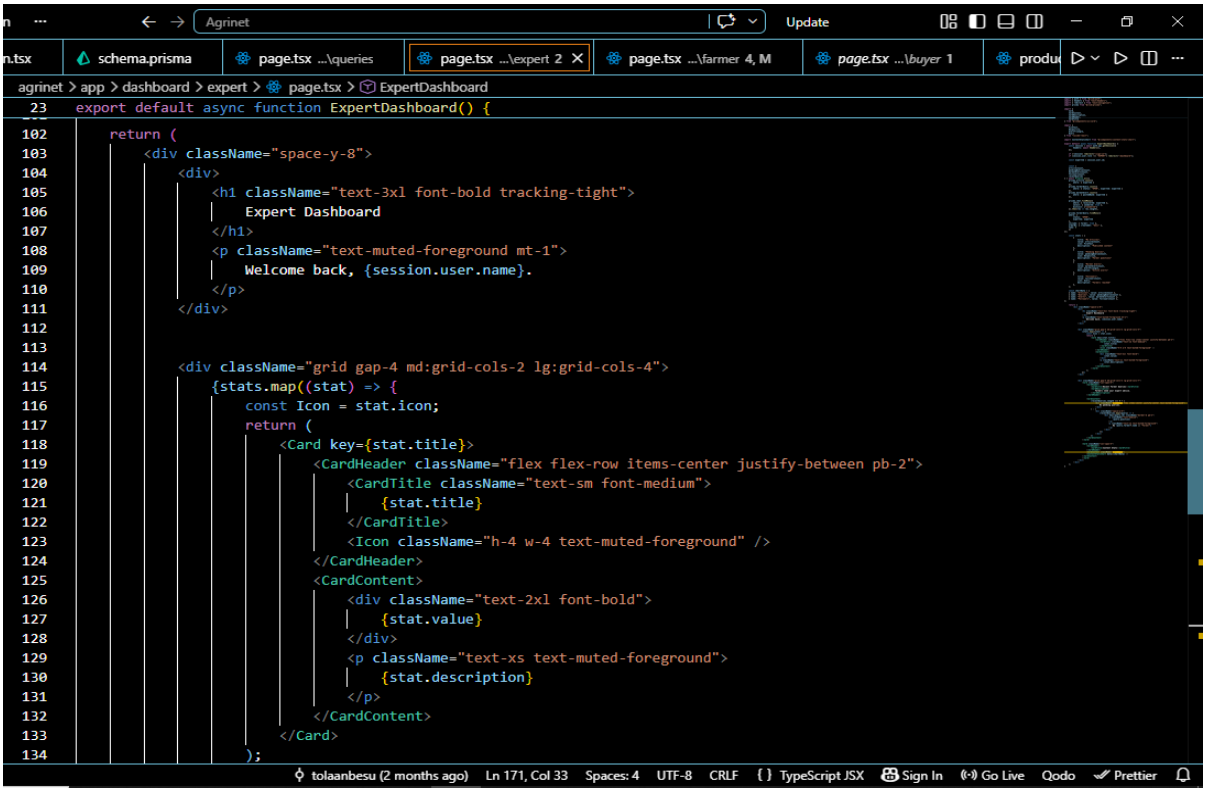


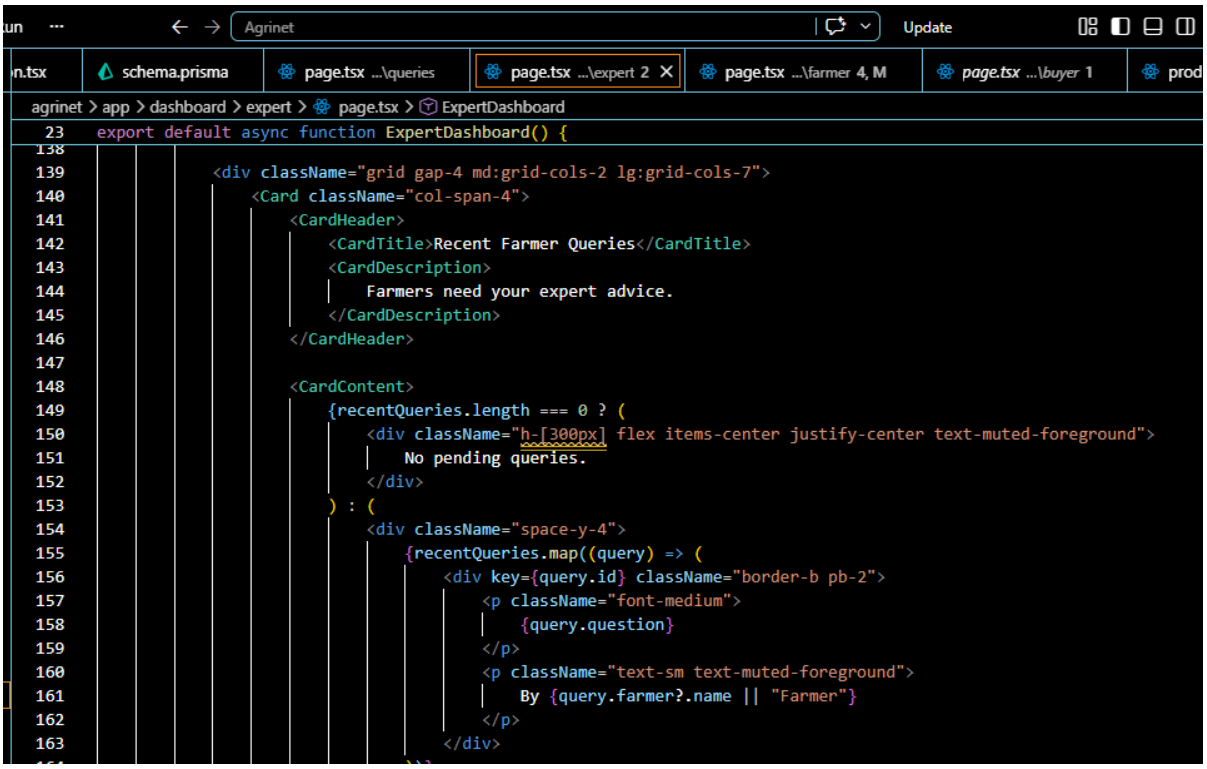
Buyers page.tsx

Admins page.tsx

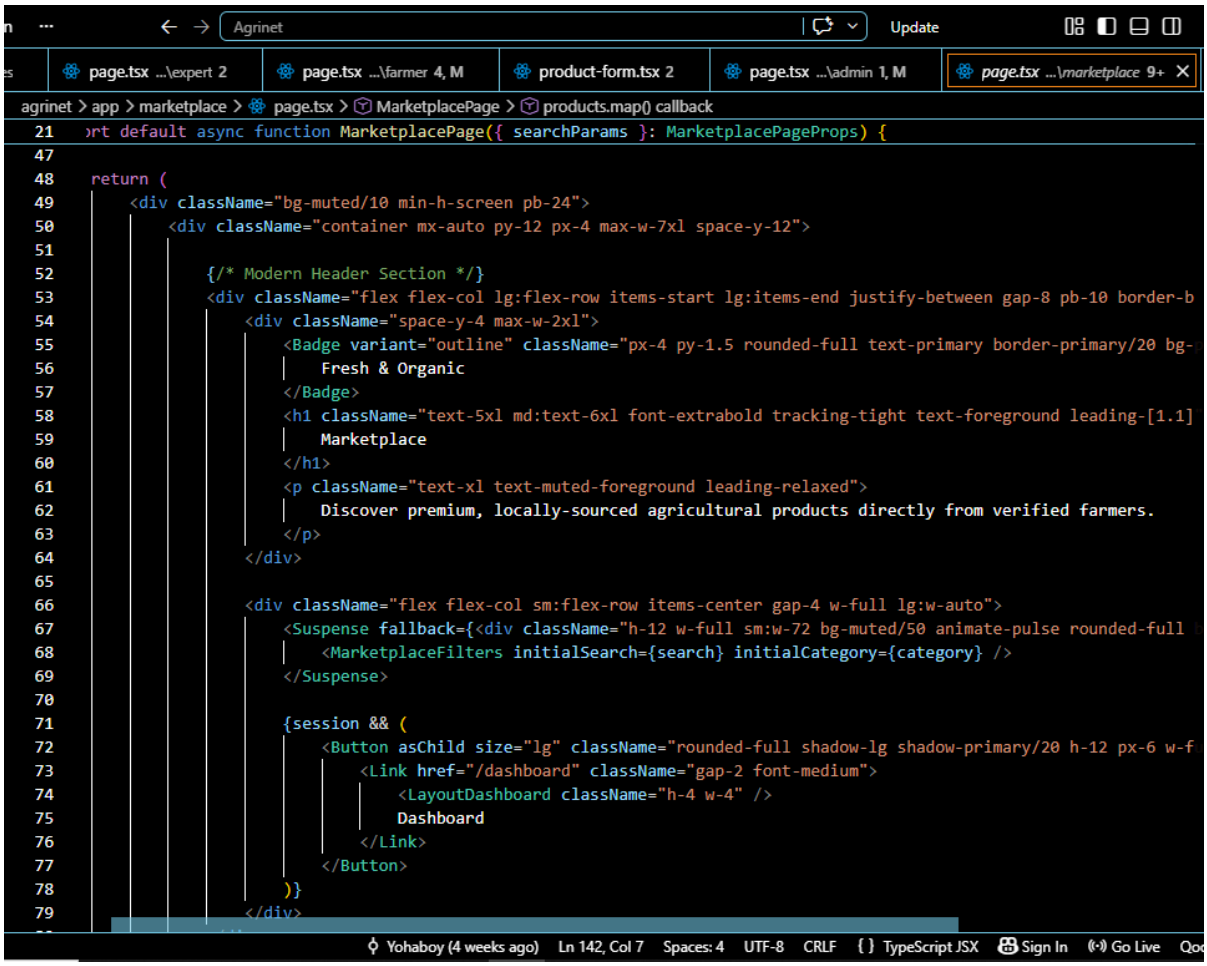


Chat page.tsx





Expert page.tsx


Agrinet
Update
page.tsx ...\expert 2
page.tsx ...\farmer 4, M
product-form.tsx 2
page.tsx ...\admin 1, M
page.tsx ...\marketplace 9+
agrinet > app > marketplace > page.tsx > MarketplacePage > products.map() callback
ort default async function MarketplacePage({ searchParams }: MarketplacePageProps) {
return (

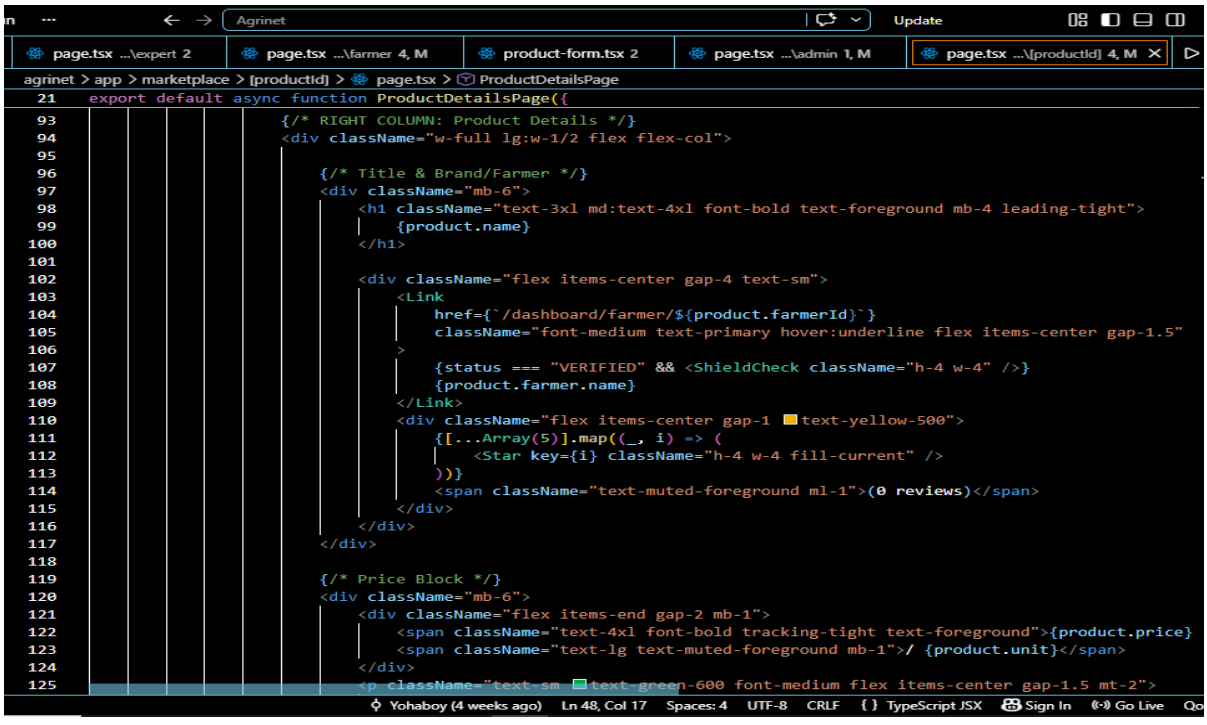
{/* Modern Header Section */}
<div className="flex flex-col lg:flex-row items-start lg:items-end justify-between gap-8 pb-10 border-b

<Badge variant="outline" className="px-4 py-1.5 rounded-full text-primary border-primary/20 bg-
Fresh & Organic
</Badge>
<h1 className="text-5xl md:text-6xl font-extrabold tracking-tight text-foreground leading-[1.1]
Marketplace
</h1>

Discover premium, locally-sourced agricultural products directly from verified farmers.

<Suspense fallback={<div className="h-12 w-full sm:w-72 bg-muted/50 animate-pulse rounded-full
<MarketplaceFilters initialSearch={search} initialCategory={category} />
</Suspense>
{session && (
<Button asChild size="lg" className="rounded-full shadow-lg shadow-primary/20 h-12 px-6 w-f
<Link href="/dashboard" className="gap-2 font-medium">
<LayoutDashboard className="h-4 w-4" />
Dashboard
</Link>
</Button>
)}

Yohaboy (4 weeks ago) Ln 142, Col 7 Spaces: 4 UTF-8 CRLF { } TypeScript JSX Sign In Go Live


Agrinet
Update
page.tsx ...\expert 2
page.tsx ...\farmer 4, M
product-form.tsx 2
page.tsx ...\admin 1, M
page.tsx ...\[productId] 4, M
agrinet > app > marketplace > [productId] > page.tsx > ProductDetailsPage
export default async function ProductDetailsPage({
{/* RIGHT COLUMN: Product Details */}

{/* Title & Brand/Farmer */}

<h1 className="text-3xl md:text-4xl font-bold text-foreground mb-4 leading-tight">
{product.name}
</h1>

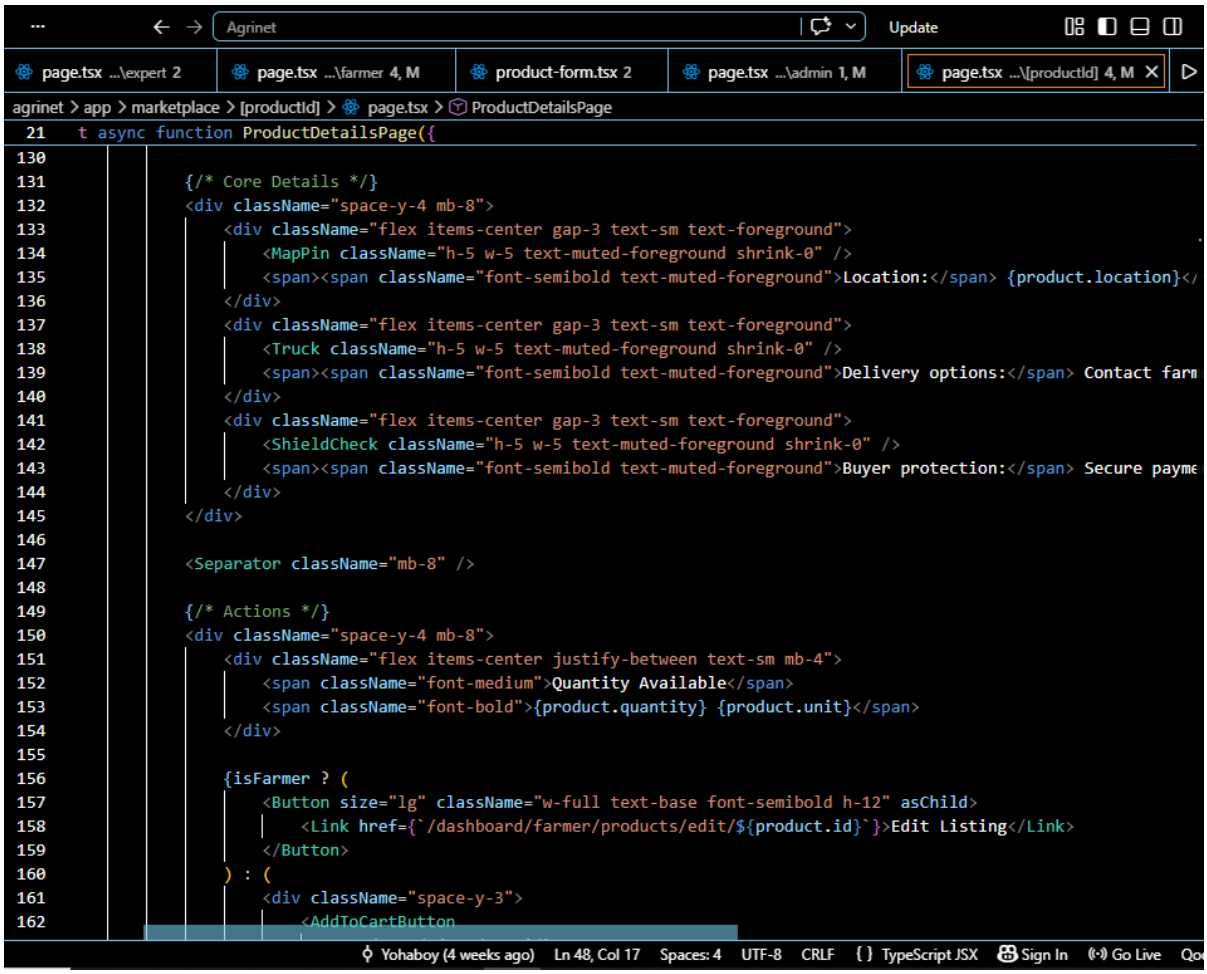
<Link
href={`/dashboard/farmer/${product.farmerId}`}
className="font-medium text-primary hover:underline flex items-center gap-1.5"
>
{status === "VERIFIED" && <ShieldCheck className="h-4 w-4" />}
{product.farmer.name}
</Link>

{[...Array(5)].map((_, i) => (
<Star key={i} className="h-4 w-4 fill-current" />
))}
(0 reviews)

{/* Price Block */}

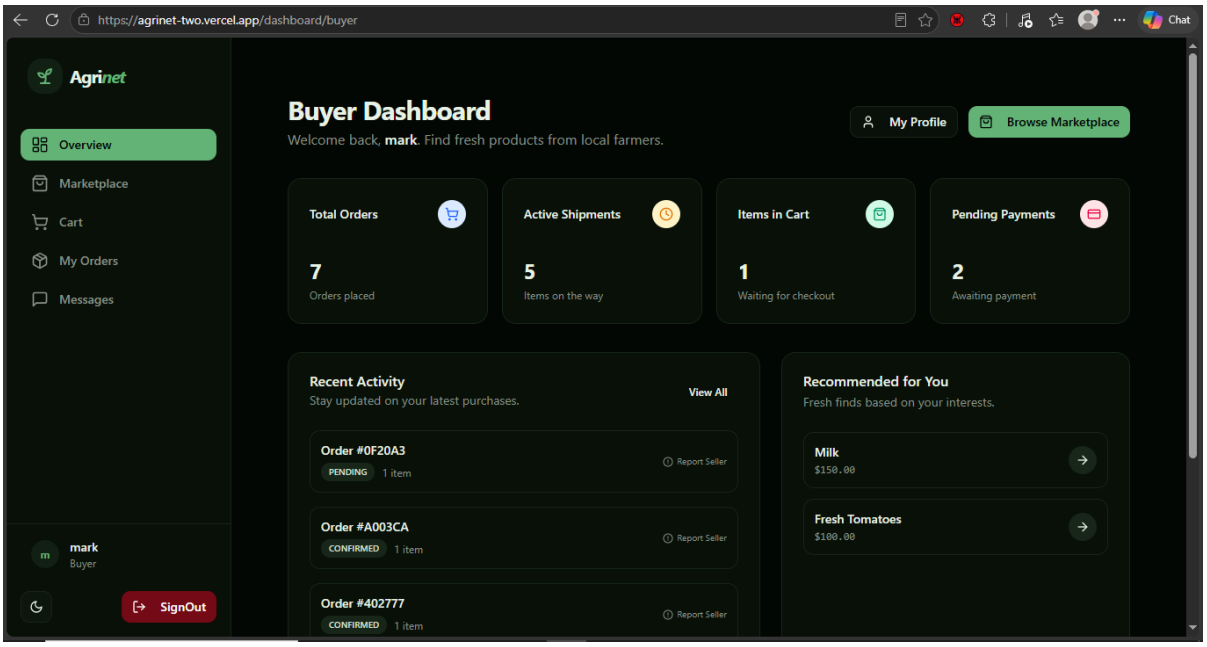
{product.price}
/ {product.unit}

Yohaboy (4 weeks ago) Ln 48, Col 17 Spaces: 4 UTF-8 CRLF { } TypeScript JSX Sign In Go Live

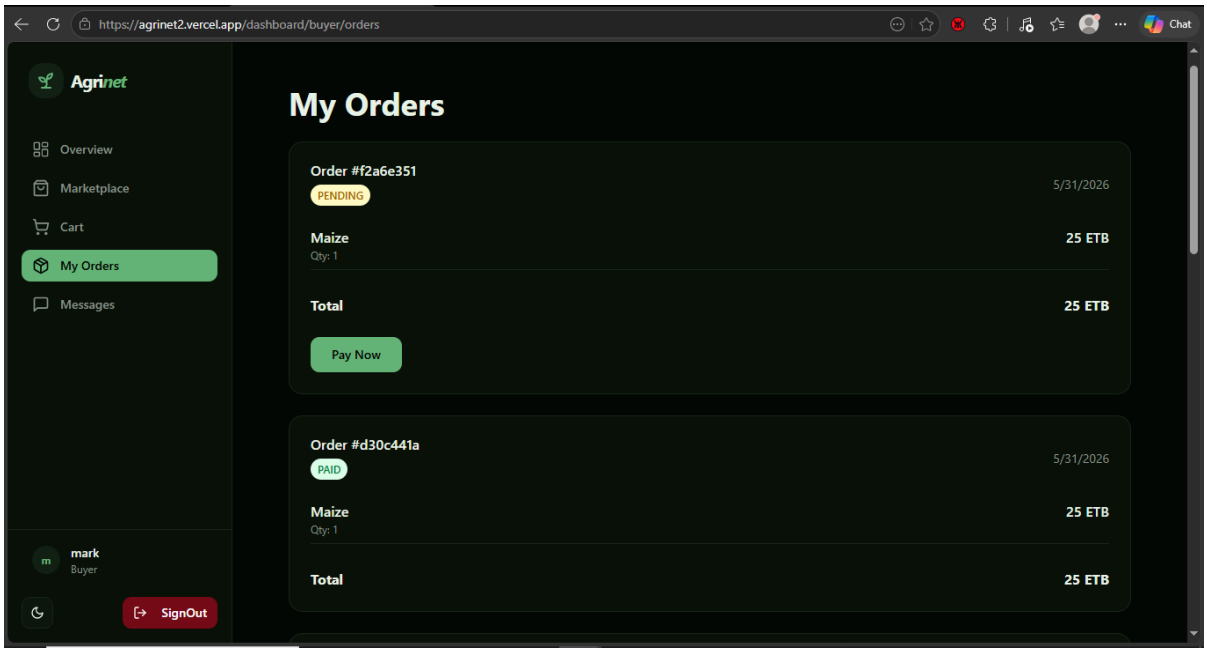


MarketPlace page.tsx

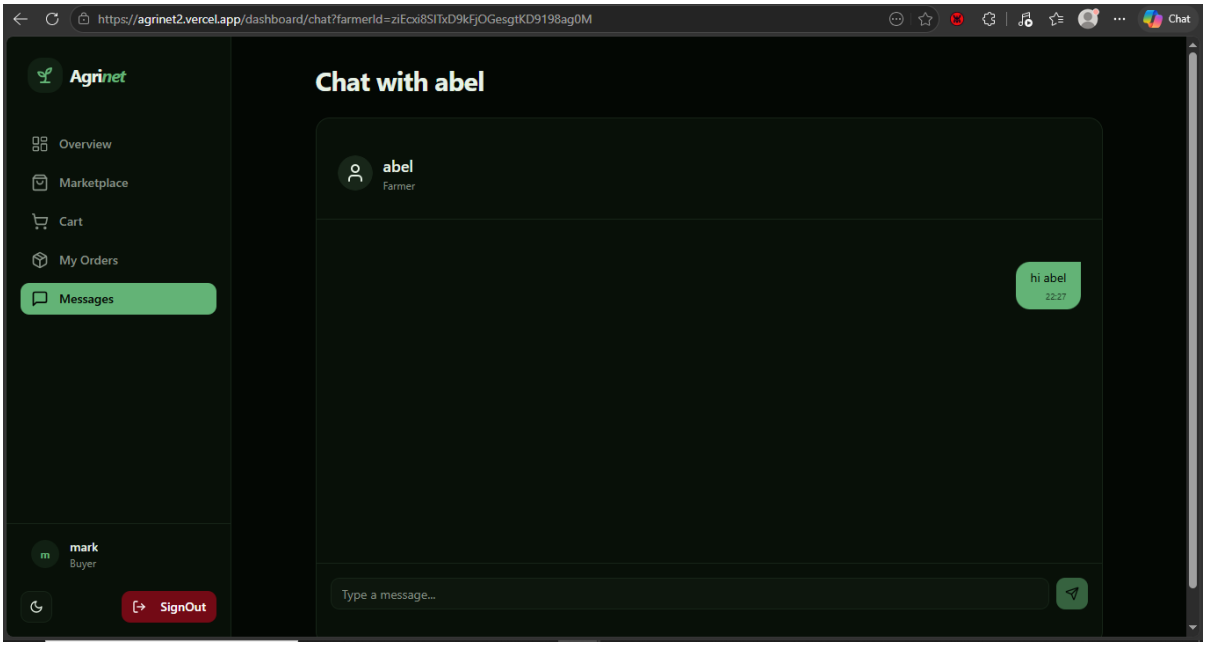
4.1.3 Screen Images



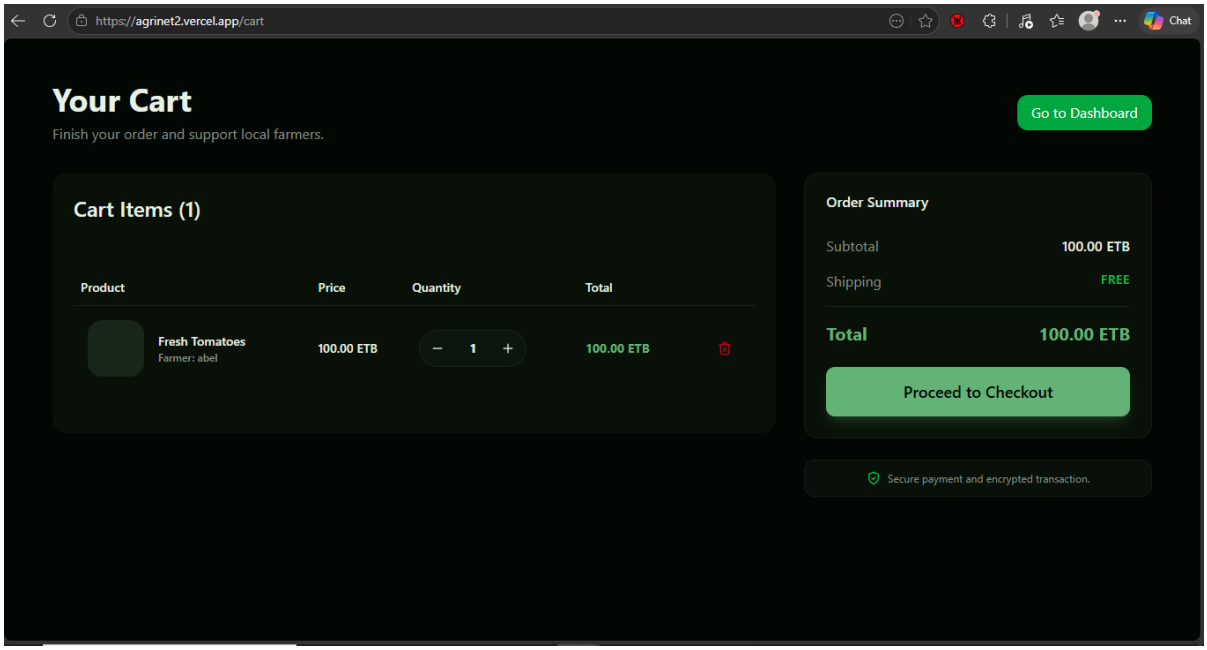
Buyers_dashboard



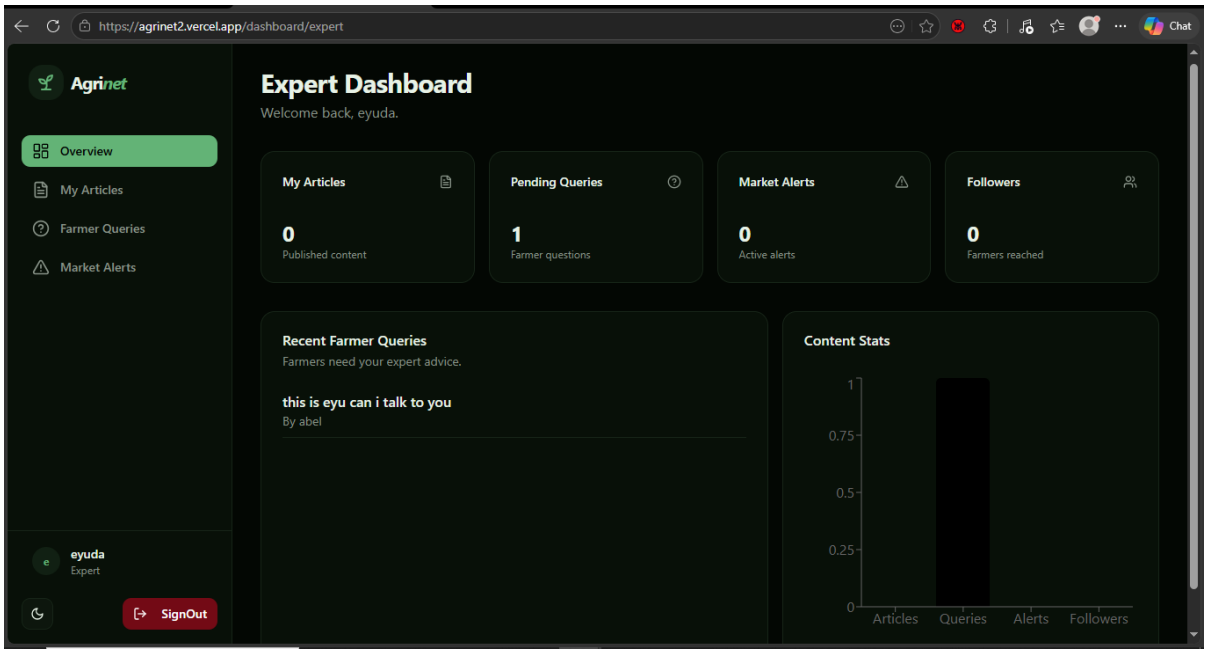
Buyer's Order Page



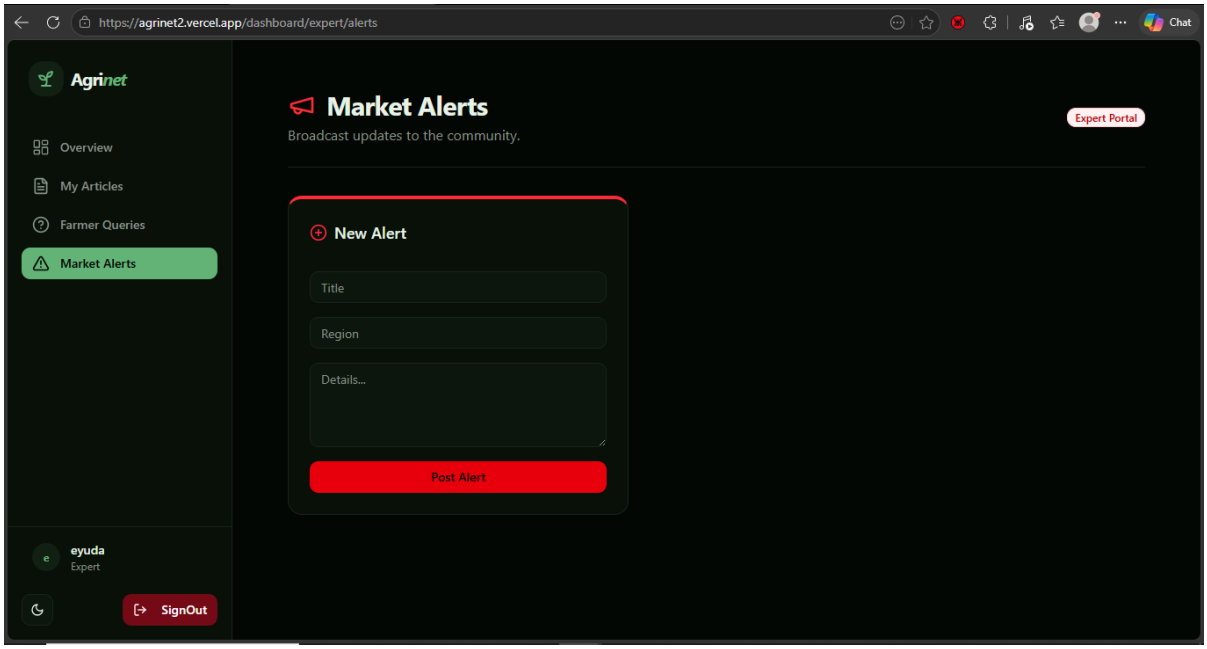
Buyer's chat page



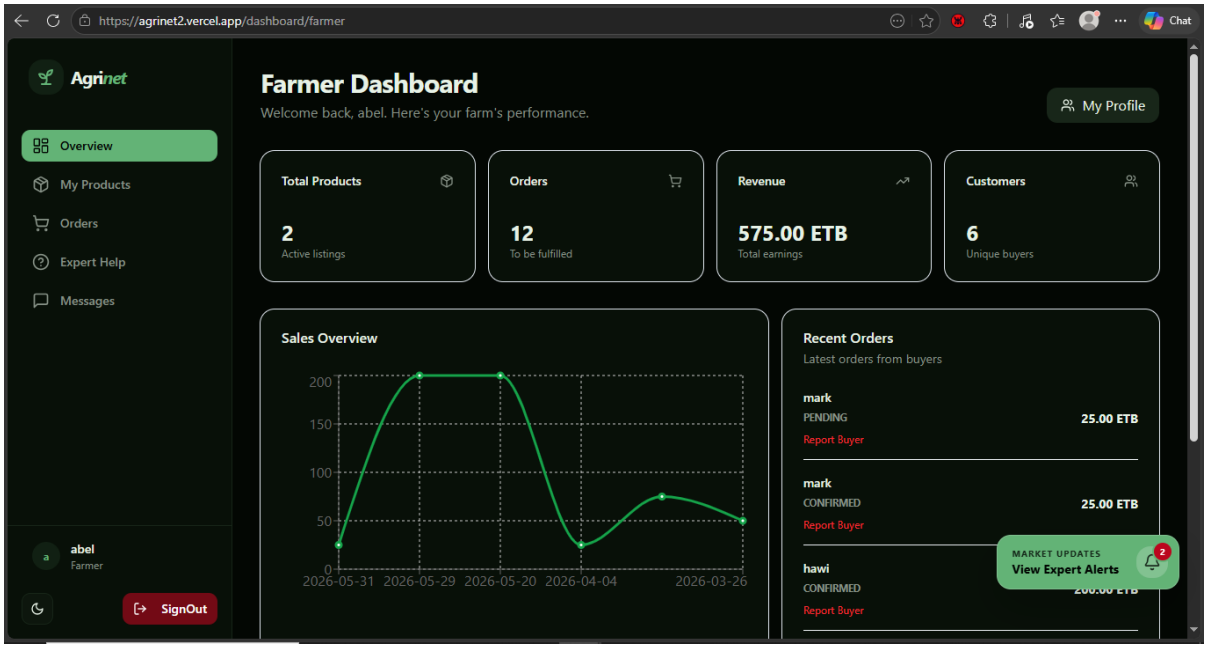
Buyer's Cart page



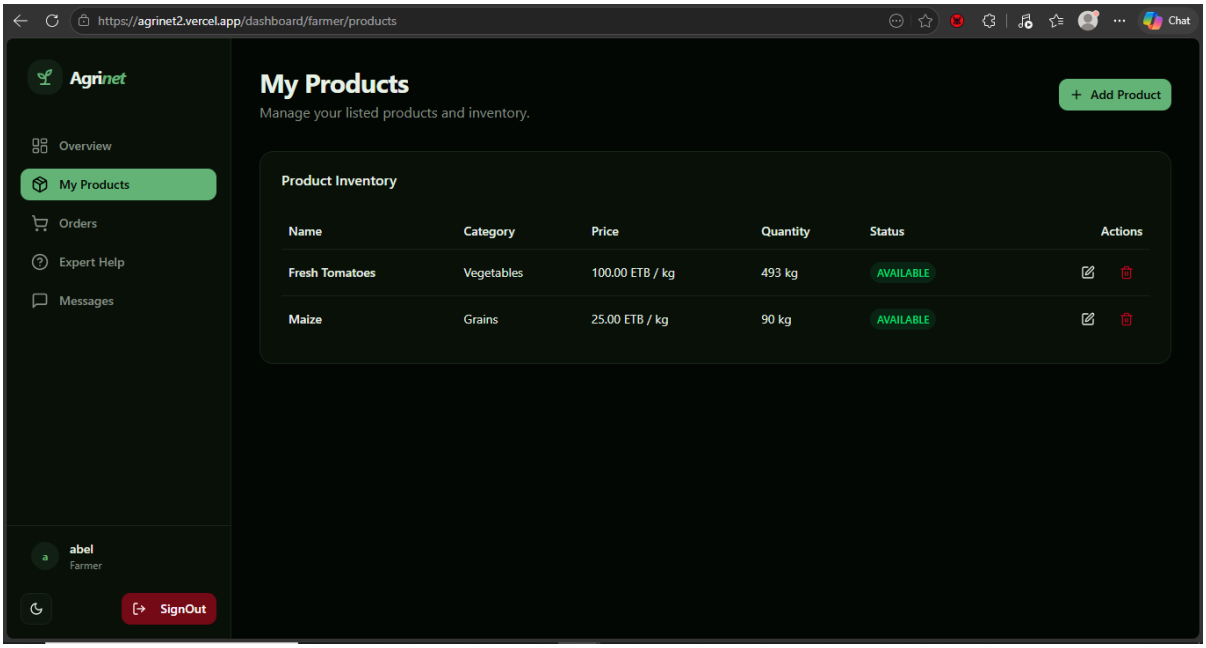
Expert Dashboard



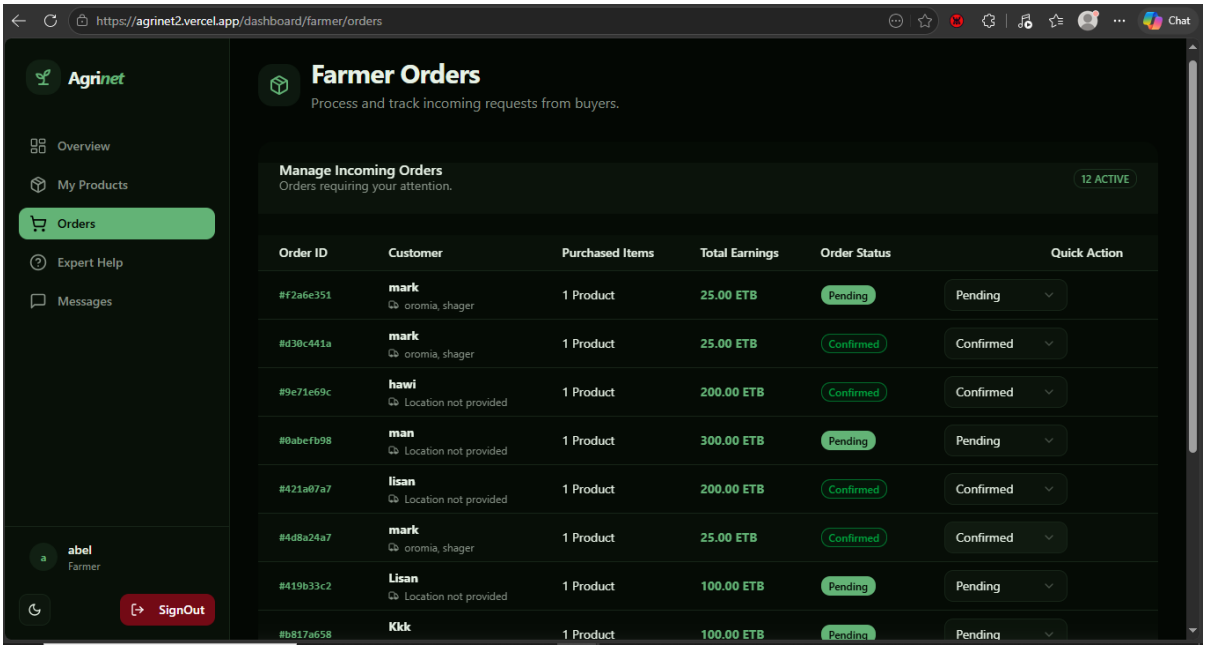
Expert alert page



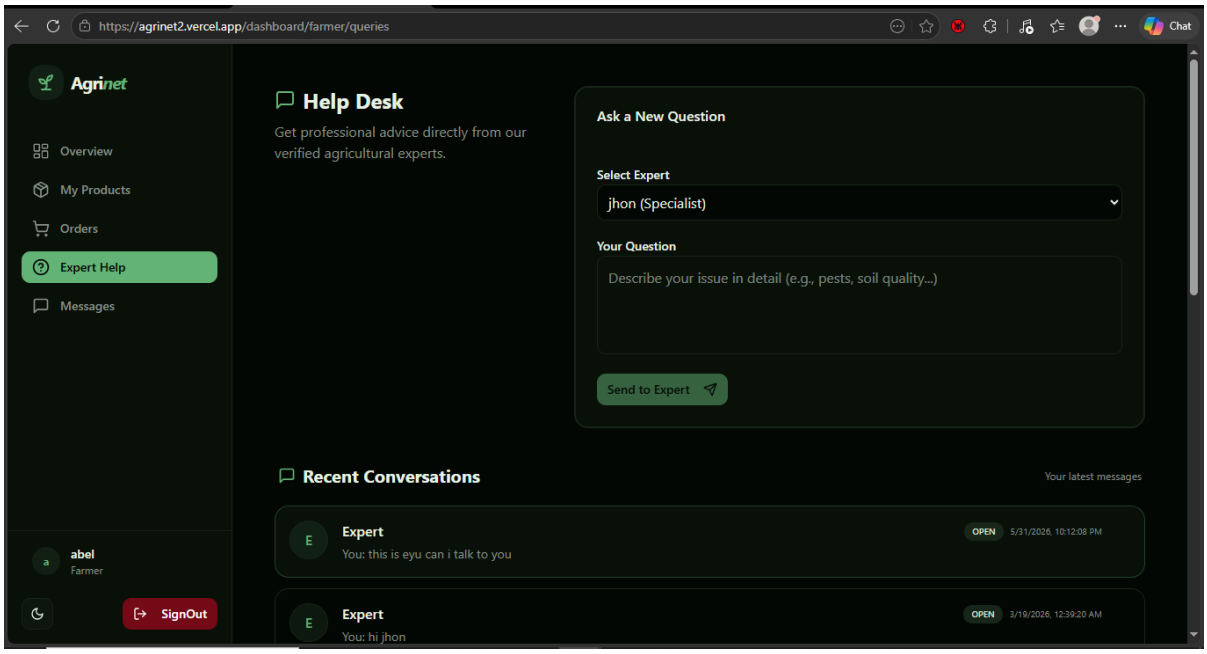
Farmer's Dashboard



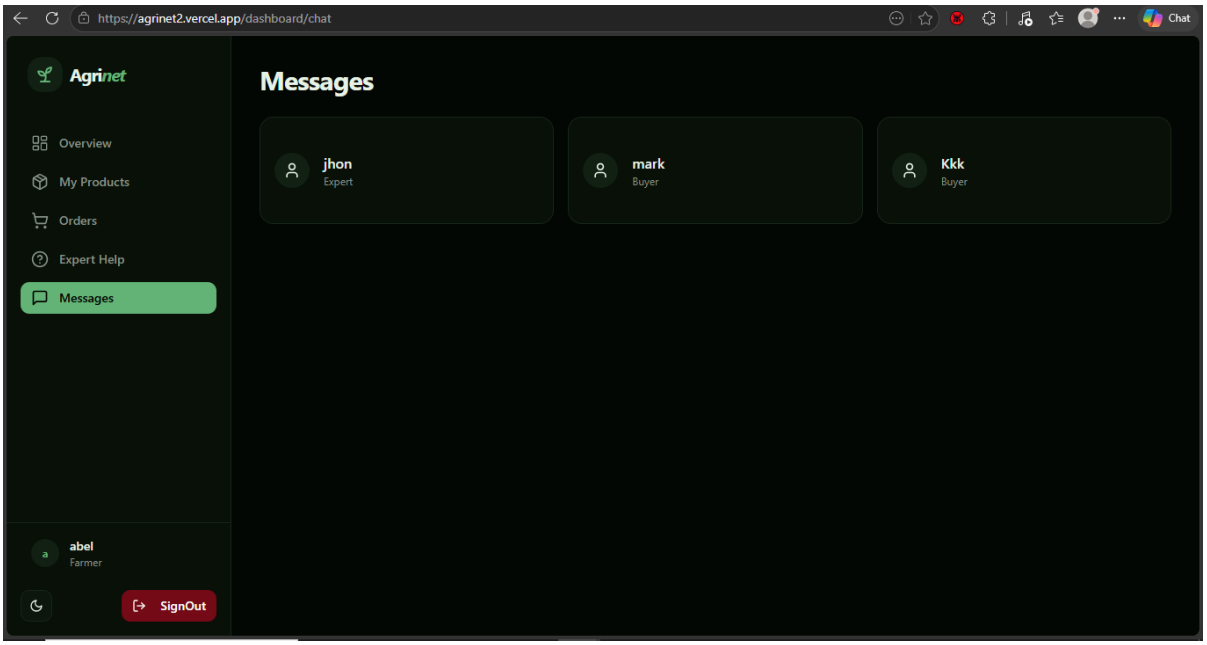
Farmer's product page



Farmer's Order page



Farmer expert page



Farmer's message page

REFERENCES

• Online Diagraming Platform. https://draw.io/

• Diagraming Software. Wondershare EdrawMax

• UML Diagrams Explained. https://miro.com/diagramming/what-is-a-uml-diagram/

• AAU. undergraduate final project template

• Banhazi, Thomas M. (2016). Subsystem decomposition.

• ResearcherGate https://www.researchgate.net/figure/Subsystem-decomposition-shown using-a UML-componentdiagram_fig1_311634500